RAVE-SPIN XR

PORTABLE XR COMPANION APP FOR DJS

FINAL PROJECT REPORT

TU858
BSC IN COMPUTER SCIENCE

CÉSAR HANNIN
C21415142

BRYAN DUGGAN
SUPERVISOR

SCHOOL OF COMPUTER SCIENCE

TECHNOLOGICAL UNIVERSITY, DUBLIN

18/04/2026

DECLARATION

I hereby declare that the work described in this dissertation is, except where otherwise stated, entirely my own work and has not been submitted as an exercise for a degree at this or any other university.

Signed:

César Marc Réné Hannin

18/04/2026

ACKNOWLEDGEMENTS

The completion of this project would not have been possible without the support and guidance of several individuals.

I want to extend the most heartfelt thank you to my supervisor, Dr. Bryan Duggan, as all of this was possible due to his continued support and passion for music. It is thanks to his insights and expertise, particularly with the technology used in this project, which shaped the direction and quality of my work.

I am also sincerely thankful to my family, whose constant encouragement, unwavering support, and restocking of snacks, which greatly helped me to stay motivated throughout these past few months, and to Django the Dog, and Lily the cat, who made sure I was up early each day and effectively mitigated any spikes of stress.

I would like to also say thank you to the TUD DJ-Society and to the several Irish DJs that helped give vital feedback to the project.

PREFACE - GLOSSARY

Throughout this report and this entire project, several terms will be used to refer to different components and equipment. As these terms can mean different things to different readers depending on their background, here's a table to clarify the terminology that'll be used for this report.

XR – EXTENDED REALITY

XR *(Extended Reality)* is an umbrella term for technologies that blend virtual spatial elements with the physical real-world environment [1], enabling users to observe and, in some cases, interact with digital content within real-world contexts.

XR encompasses AR *(Augmented Reality)*, MR *(Mixed Reality)* and VR *(Virtual Reality)* and as this project encompasses features of both AR and MR, the term XR will be used to help avoid confusion.

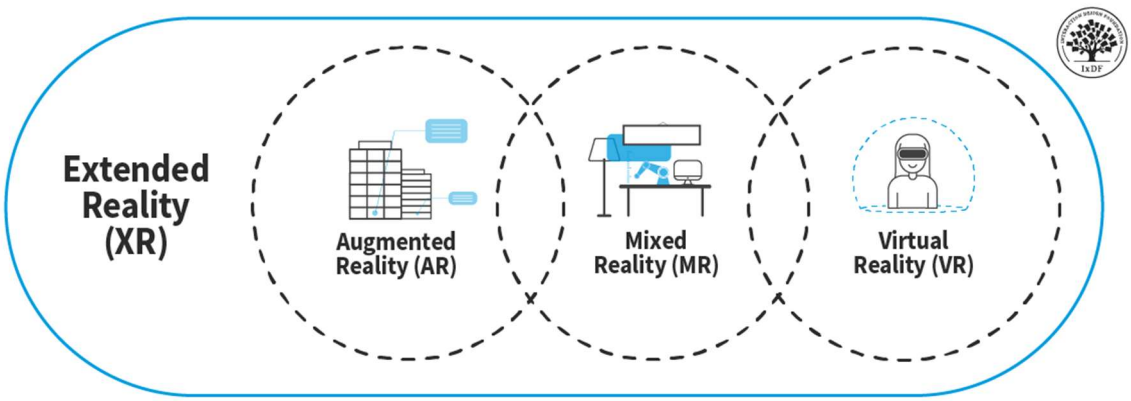


Figure 0-1 Diagram showcasing differentiation between XR, AR, MR, and VR

(DJ) CONTROLLER

A Controller refers to an all-in-one piece of equipment. Typically combines two or more virtual “decks” with the jog-wheels and a central mixer section. These decks have songs loaded in them for playback. Usually includes built-in Performance pads and basic effects controls. For most use cases, it functions as a near complete general solution for mixing and manipulating tracks.

Figure 0-2 Picture of a Pioneer DDJ-FLX4

(XR) CONTROLLERS

Many XR headsets come with their own controllers to use for user interactions with the system.

Because Rave-Spin XR will rely solely on user hand-tracking, and not the controllers of the headset, the term “Controller” will refer to a **DJ Controller**, and when referring to the hand-held controllers that come with an XR Headset, this will be clearly stated.

Figure 0-3 Picture of Meta Quest 3S Controllers. Not used by RaveSpin

Figure 0-4 Picture of Meta Quest 3S Headset (Used for RaveSpin testing)

PERFORMANCE PAD

A pad-based performance controller consisting of multiple padded buttons, each mapped to a specific sample, loop, or effect.

Pressing a pad triggers the associated action, allowing rapid, rhythmic triggering during a performance.

Figure 0-5 Picture of a Pioneer DDJ-XP2

JOG WHEELS AND SPIN WHEELS

A large spinnable wheel that lets users “scratch” the music.

This is where the music jumps backwards or forwards in during playback, depending on the scale of the spin taken by the user.

The proper name is a “Jog wheel,” *(Others may also call this part a “Platter” but this term is never used in Rave-Spin XR)*, but in the code for this project, it is referred to as a “Spin Wheel” on numerous occasions.

All terms refer to the same component that spins with the music.

Figure 0-6 Picture of the Pioneer CDJ-3000

SETUP AND SESSION

The **Setup** refers to the configuration of equipment used by a DJ. In this project, the setup has the infrastructure in place to be changed later at runtime to accommodate a number of various other components.

Figure 0-7 Picture of a DJ Setup

The **Session** refers to the period of time that this setup is being actively used by a DJ, during which time they could be mixing, performing, recording, etc...

BPM / TEMPO

In songs, there is always going to be a certain speed that songs can be, or BPM *(Beats per minute)* or Tempo as it may also be called.

Throughout this project, BPM and Tempo will refer to the same thing, the set speed of a song.

EQ (EQUALISER)

When audio is being played, what we hear is the result of adding together many different sound waves, each at their own frequency and with their own amplitude. When you add all the frequencies of a song snippet together, the result is the complex sound that reaches your ears.

The way these frequencies are distributed is what gives different types of music their feel. Bass-heavy music such as hip-hop or EDM *(Electronic Dance Music)* has most of its energy sitting in the lower frequencies. These are the slow, powerful vibrations you can sometimes physically feel in your chest at a concert. Higher frequencies are still there, but they are quieter and contribute less to the overall sound. This is why bass-heavy music feels deep and heavy.

The opposite is true for music that sounds bright or sharp. Instruments like violins, violas, or the higher notes on a piano produce most of their energy in the higher frequencies. The lower frequencies are still present but are not as prevalent.

Figure 0-8 Spectrum of music with heavy bass.

Figure 0-9 Spectrum of music with higher notes / treble

What an EQ (Equaliser) does is manage this balance. An EQ can emphasise any band of frequencies, adding more focus to the bass, mid-tones, or high tones of a song.

HIGH / LOW PASS FILTER (COLOUR FX)

The main idea behind a high pass filter, and a low pass filter, is to only allow frequencies above or below a certain threshold to pass through.

The **high pass filter** will only allow frequencies **above** a set threshold to pass through.

The **low pass filter** will only allow frequencies **below** a set threshold to pass through.

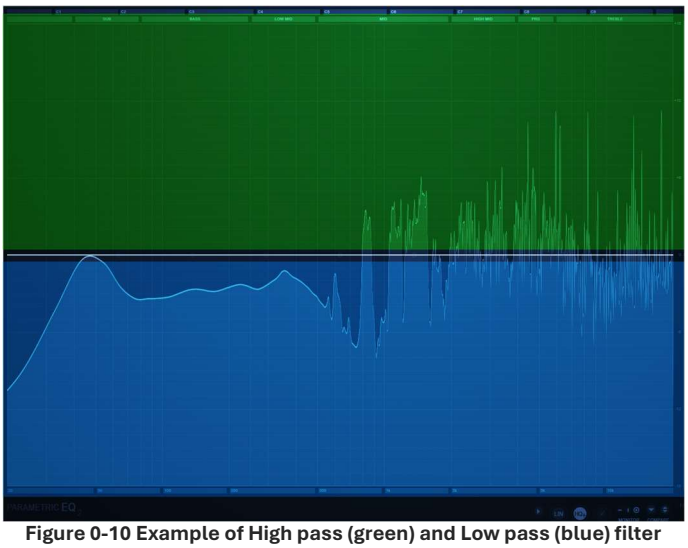


Figure 0-10 Example of High pass (green) and Low pass (blue) filter

A Colour FX is a special parameter that ranges from 0.0, to 1.0 *(inclusive)*. A value of 0.5 means no filter is active.

A value below 0.5 specifies the threshold for a low pass filter. Inversely, a value above 0.5 specifies the threshold for a high pass filter. Another name for this is **"CFX"** *(Colour FX)*. Having both a low pass filter and a high pass filter active at the same time is counterintuitive as they operate in juxtaposition. A single CFX slider is preferable as it operates the 2 filters with one slider, enabling and disabling the filters as needed with a simple single slider.

ABSTRACT

The idea behind Rave-Spin XR, is through the use of an XR headset, such as the Meta Quest 3S, give experienced DJs *(Disk Jockeys)* the opportunity to make and mix music, without the need for the usual physical equipment being present, thus allowing them to mix and play on-the-go.

With the included tooltips, beginner guides on YouTube and the Official RaveSpin XR website [2], and its inherent ease-of-use, Rave-Spin XR allows beginners to try out DJing for themselves and learn the various techniques and components without the required financial investment in the relevant hardware. With the growing interest in electronic music and people learning to DJ and mix, this will allow people to try out the artform for themselves and gauge whether it's a passion they would like to pursue. Studies have shown that XR can be a very effective method for teaching which would be ideal for people who want to practice DJing with this application.[3]

The application is loaded onto the users' headset. All audio processing, hand-tracking, rendering, File-IO, metadata extraction from songs, is all locally run on the users' headset. There is explicitly no online requirement.

Sessions may be recorded by the user, saving all output to the users' headset, where it can be easily shared, or easily extracted via USB to the user's computer using the included tools.

The initial goal of Rave-Spin XR was to make an application that was free and open source, that any DJ, regardless of expertise, could pick up and use. It is also a proof-of-concept that manipulation of audio in such a manner could be done on current-generation low powered mobile devices with acceptable performance.

Experienced DJs can mix and play as they would with the real-life equivalent hardware, with little to no onboarding, with quality-of-life features that are only possible in XR, such as easily interactable and navigable menus in 3D space, and later on in future iterations, the ability to switch out hardware at runtime.

Beginner DJs can try play and mix with DJ equipment without the need to make the financial investment to acquire it. They could also learn, practice, and see for themselves if it's an artform they would like to explore further.

TABLE OF CONTENTS

I. INTRODUCTION

Rave-Spin XR is an XR-based DJ application designed to provide a portable, immersive alternative to traditional DJ equipment. Conventional DJ controllers and mixers are typically large, heavy, and expensive, making them difficult to transport and inaccessible for many beginners, like the Pioneer DDJ-FLX10 with its €1,689 price tag and 6.7KG weight making it difficult to carry [4]. High-end setups can cost thousands of euros and often require a solid grounding in music theory and DJ techniques before a user can meaningfully engage with them.

This project addresses these barriers by emulating the DJ environment in XR *(Extended Reality, sometimes called Augmented Reality)*. Rave-Spin XR allows experienced DJs to emulate a full performance DJ setup using only a headset and hand tracking, enabling them to practise or perform without carrying physical hardware.

At the same time, the system supports complete beginners by providing an accessible, low-cost entry point into DJing that does not require investing in physical gear. Online tutorials have also been created and published online for public viewing.

In addition, the application includes descriptions of all components and tooltips to make the learning process more engaging. These are here to help teach core DJ skills and concepts in an interactive way, helping users build confidence and competence over time. The goal of Rave-Spin XR was to combine the flexibility of XR with intuitive interaction design to create a portable DJ experience that is both practical for experienced users and approachable for newcomers.

A. PROJECT BACKGROUND

Effective DJing is a combination of musical, technical, and performance skills. Some individuals enter the space with a strong foundation in music theory, while others begin with little to no prior knowledge, which can significantly limit their ability to progress. In addition to understanding elements such as rhythm and harmony, aspiring DJs must also learn to operate a range of hardware and software tools, each with its own workflows and terminology, pros and cons, and potential stock availability issues.

While it's possible to teach yourself everything you may need using online resources, the whole learning environment can be quite fragmented at times. Platforms such as YouTube provide several tutorials, but they're sometimes tied to specific, commonly used controllers or software. Given the wide variety of DJ equipment available, many devices might receive limited coverage, leaving beginners at a loss if they can't find enough information or tutorials on their chosen equipment which can be confusing and discouraging. Studies have shown that XR has been a great tool for enhanced learning [3], which is exactly what would benefit a tool like Rave-Spin XR.

For experienced DJs, the challenge is different but related. Professional-grade setups are often physically large, heavy, and expensive, making them impractical to transport regularly, especially for practice or casual use. The Pioneer DDJ-FLX10, the gold standard of DJ controllers [5], [6], [7], is 40cm long and 6.7kg, it's a challenge to move just that alone. A system that can emulate familiar DJ workflows in an XR environment has the potential to reduce reliance on the physical hardware, enabling DJs to rehearse using only a headset, without the logistical overhead of carrying full equipment.

B. PROJECT DESCRIPTION

This project focuses on emulating the workflow used by the Pioneer DDJ-FLX4. The user interacts with a DJ Controller in 3D space using just their hands alone. XR Controllers are not required. The DJ Controller used in the final version is based on the Pioneer DDJ-FLX4 because it's a highly popular entry-level DJ controller that includes most features an aspiring DJ could want [8]. By targeting this controller first, the system can support many of the interactions and workflows that any DJ will encounter with most other controllers, making both support for adding a new DJ controller in the future very quick, and letting the user very quickly and easily switch to another DJ Controller with very little onboarding.

Rave-Spin provides an XR-based environment in which users can learn and practise DJ skills using a virtual model of a DJ Controller that is based on the DDJ-FLX4 before committing to purchasing physical equipment. The controller itself typically costs in the region of €200–€330, which represents a significant upfront expense for many potential learners. By offering an interactive XR alternative, the project seeks to lower this financial barrier. Users can get familiar with the layout, experiment with mixing techniques and follow structured tutorials without needing to buy hardware in advance.

If, after using the system, a user decides that DJing is not for them, they can simply stop using the application without having made any financial investment. Conversely, if they decide to progress further, their experience in XR should transfer directly to the physical controller, supporting a smoother and more confident transition into using real-world DJ equipment.

C. PROJECT AIMS AND OBJECTIVES

The aim of this project was to build a fully functional XR replication of a DJ Controller that was heavily inspired by and based on the Pioneer DDJ-FLX4. This DJ Controller could then be used by DJs to mix their tracks in much the same way they would with the real equivalent physical hardware. As the project progressed, additional features such as tutorials on how to DJ effectively were to be introduced, with further functionality planned for later iterations.

The following features were identified as what was needed to achieve the minimum viable product:

- Accurate 3D rendition of a DJ Controller with the same workflow and features of the Pioneer DDJ-FLX4 and an accompanying XR User Interface.
- The ability to **load audio tracks** from the device's local storage.
- **Reliable hand-detection** across all interactive controls, such as buttons, sliders, knobs, jog wheels, UI elements, etc...
- Real time **audio effects** such as reverb, delay, phaser, pitch-shifting, EQ adjustment, and more.
- The ability to **record and save audio** output from a DJ session for later playback.
- Support for **microphone input** when recording sessions.
- Working performance pads.
- Jog-wheel scratching.
- Match the tempos of different songs.
- Easily create and manage loops.

The following extended features were not needed for the minimum viable product but are planned for future iterations:

- **Interactive tutorials** covering core DJ skills and equipment setup and usage. These tutorials would incorporate gamification elements to make the learning process more engaging. They would also cover several other aspects too, like the more practical aspects of live sound, including speaker placement, wiring configurations, gain staging, and dB levels, all with the goal of helping users prepare reliable audio setups for real world gigs.
- **Local Multiplayer**, allow users to connect to other users' sessions on the same wireless network, allowing users to join sessions for collaborative mixing, remote teaching, and real time feedback.
- **Additional DJ equipment** beyond the DDJ-FLX4, including *(but not limited to)* CDJs, extra performance pads, and MIDI controllers.
- Integration with **third party music streaming platforms** such as Spotify, Apple Music, YouTube Music, and SoundCloud, allowing users to download tracks directly from their existing libraries and playlists to use in sessions.

D. PROJECT SCOPE

The scope of this project was shaped around two central goals:

1. To develop a **portable XR music tool** that emulates popular DJ equipment closely enough to be genuinely usable by experienced DJs without a steep adjustment period. While it was never intended to fully replace dedicated hardware, the system needed to function as a practical, portable alternative, particularly useful when travelling or when access to physical gear is limited.
2. To provide an **accessible learning environment** for beginners who might already own an XR headset, especially a cheap affordable one like the Meta Quest 3S, but who may not yet own any DJ hardware. Rave-Spin XR would introduce users to what a DJ does, how DJ workflows operate, and how to build core skills, all without needing to buy physical equipment upfront. If a user went on to invest in real hardware after using the system, the knowledge and muscle memory gained should transfer over naturally. If they decided DJing was not for them, they would have avoided any unnecessary expense.

To keep the project achievable within the two-semester development timeframe, the first few prototypes focused on replicating the Pioneer DDJ-FLX4. The intention was to implement one piece of equipment well and accurately, rather than attempt an exhaustive catalogue, all while leaving the infrastructure there to allow for the addition for more and to scale as time went on. The project is open source, so anyone who wishes to add support for their own equipment can do so with relative ease.

Features such as integrated light shows were considered during early planning but were excluded from the current scope, as they would not have contributed meaningfully to the core mixing experience and would have added significant visual complexity.

A major ongoing challenge throughout development was balancing performance with accuracy. The application needs to maintain a consistently high frame rate in XR while still providing precise, responsive emulation of real DJ controls.

Any noticeable lag or latency on button presses, fader movements, or jog wheel scratching just can't happen.

Some DJ devices, including the Pioneer DDJ-FLX4, rely on companion software interfaces for tasks like loading and managing tracks. To account for this, the project includes a modular UI layer called Librebox, which can be reused with different controllers or additional equipment wherever similar track management and configuration functionality is needed. It also doubles as the main menu interface for Rave-Spin XR. More information can be found on page 48, Section III.B.2, "Librebox".

Testing was carried out exclusively on the Meta Quest 3S. This headset was chosen because it was both available to borrow from the college, and also because of its price and popularity, meaning the results should be representative for the vast majority of the XR user base, as seen in Figure II-9 Market Share of XR Headsets in 2025 *(page 25)*. It also runs on Android and uses the OpenXR library, both of which are open standards meaning other headsets should work with no issue.

Rave-Spin XR was never designed to make existing DJ solutions obsolete. It's meant to complement them and give existing DJs more choice and flexibility with how they perform, and to let beginners break into the industry more easily by having an even lower barrier to entry.

E. UNIQUE SELLING POINT

What makes Rave-Spin XR unique is the fact that there's no other app exactly like it on the market for this price point *(or a lack of price point)*. For people who don't know how to DJ, or are learning from scratch, it is a fantastic learning tool, hopefully teaching more people about how fun and easy it can be to DJ and make the scene more varied and expressive. Alternatives like Tribe XR DJ Academy are good but they are not free.

For experienced DJs, it will allow them to record mixing sessions, even if they're away from their physical setups. They can go abroad with a small backpack and still mix and experiment with new tracks right away, as carrying around a XR headset is much more practical and portable than a physical DJ Controller.

It is completely free and open source, using **FOSS** *(Free open-source software)* tools to help accommodate this, with a great focus on code readability and documentation so that anyone else who wants to expand on it, such as adding more equipment from other brands may do so easily.

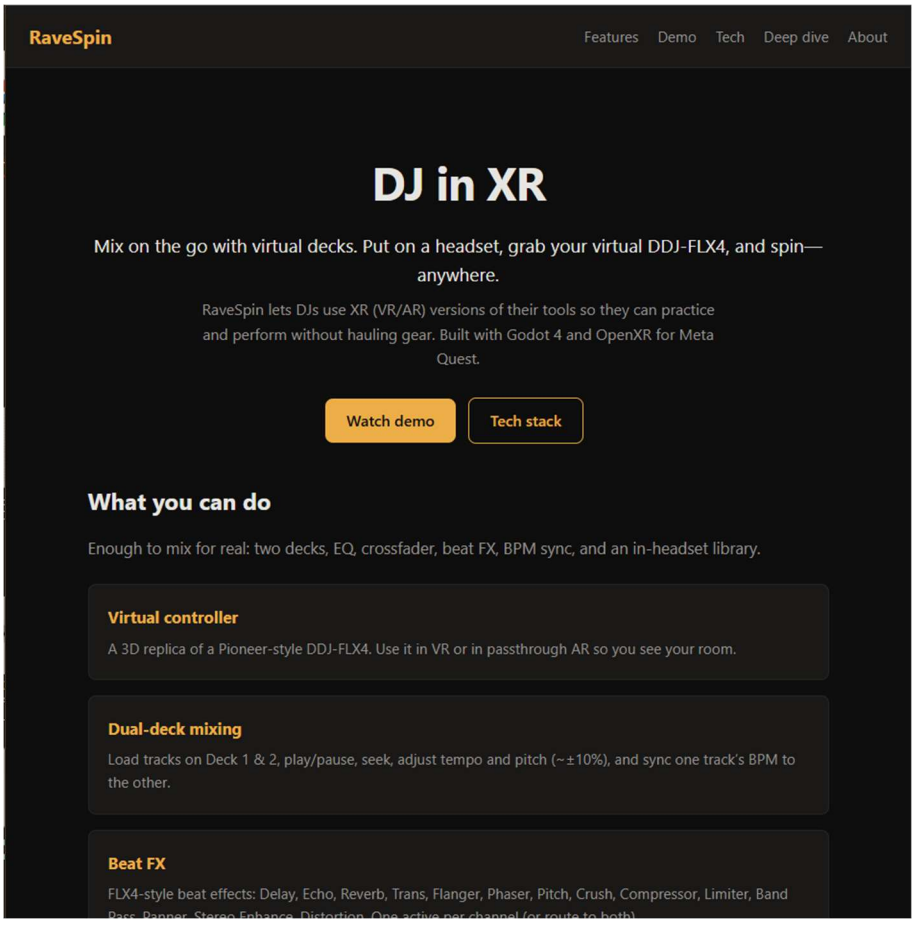


Figure I-1 Screenshot of RaveSpin Official Website

II. LITERATURE REVIEW

In this chapter I will discuss the research that was undertaken to assess the feasibility of Rave-Spin XR and the approaches considered during its design. It details existing solutions to the problems the project set out to solve, the technologies adopted to address them, and what prior final-year projects tackled similar goals or used similar approaches and what could be learned from them.

A. ALTERNATIVE EXISTING SOLUTIONS

1. TRIBE XR DJ SCHOOL

Tribe XR DJ School lets users learn and perform DJ sets using realistic virtual hardware based off real equipment [9]. Users can use various controls, effects, mix tracks, and it also comes with interactive tutorials, multiplayer sessions, and live classes with mentors. Users can also import their own music and stream performances via Twitch or other platforms. Tribe XR runs on Meta Quest, SteamVR, and PC VR systems. It shares many of the same goals of Rave Spin and it accomplishes them well.

Pros	Cons
Vast catalogue of effects and gamification elements for learning.	Only a paid subscription model. No option for a free plan, only a free 7-day trial
Highly rated and polished [10]	Many users have mentioned issues regarding latency.
Dedicated community to provide support and lessons	Equipment simulation is accurate, but users have reported issues with gestures not being recognised, making moving knobs and controls difficult sometimes.[10]
Cross platform, able to use SteamVR, Oculus, whichever XR headset the user wants	Steam version is still in early access nearly 6 years later
Import your own tracks	

Figure II-1 Screenshot #1 of Tribe XR DJ School


Figure II-2 Screenshot #2 of Tribe XR DJ School

2. ALIVE IN VR

Alive-In-VR is an XR interface for Ableton Live.[11] Rather than making audio itself, it sends commands to Ableton and lets you use the features that Ableton offers. In Alive-In-VR you can place blocks which represent clips, instruments, mixer controls, etc, around you in XR and interact with them. It doesn't try exactly replicate physical hardware, but it still gives you good control over your music.

Pros	Cons
Is a VR interface for Ableton Live. This allows it to use a mature, full-featured industry standard DAW. [11]	Does NOT generate audio itself. It's just a controller / interface for Ableton Live which itself requires a valid paid licence.
Uses easy to grab blocks around you in 3D space instead of trying to perfectly replicate real life equipment thus becoming much easier to control	No educational aspects. You're expected to fully know the software and terminology.
Great performance, very little reported latency [12]	Doesn't simulate a virtual audience or venue. It's more of a VR interface tool than a performance simulator.
	No community behind it, which means no community support if any issues arise.
	Users have complained about polish. Appears tacky.

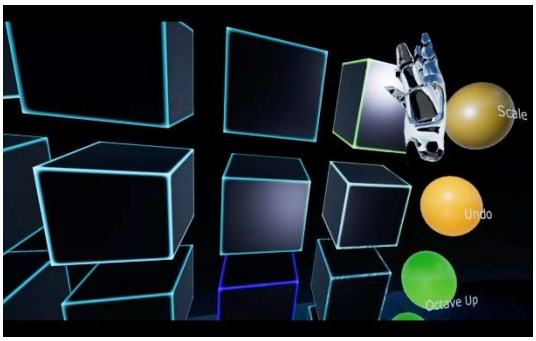


Figure II-3 Screenshot #1 of Alive-in-VR

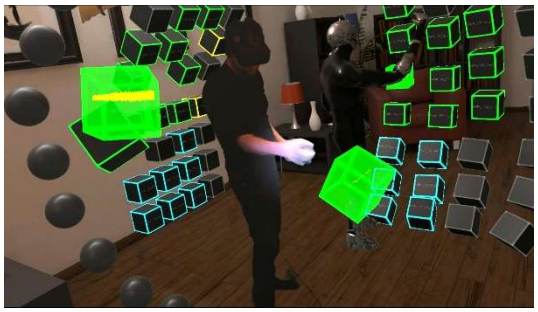


Figure II-4 Screenshot #2 of Alive-in-VR

3. VINYL REALITY

Vinyl Reality is a XR app that simulates vinyl-style DJing. It has 2 turntables, and 2-channel mixers with EQ, filters, gain, etc, and supports importing your own tracks. You can DJ in virtual environments with reactive crowds, lighting effects, and multiple camera angles. You can record your sessions or stream them and use custom or built environments. There's a short interactive tutorial to teach you the basic controls. It also has Rekordbox integration *(a very famous app for interacting with DJ controllers)*. [13], [14]

Pros	Cons
Great deal of realism and fidelity, able to work with turntables, mixer with EQ, filters, gain, etc.	Many users and the developer, have mentioned issues regarding latency. [14]
Import your own tracks.	It does not support elements such as sequencers or synthesizers with MIDI tracks.
Virtual cameras can show you on a screen to see how you look while DJing, this also includes several environments *(custom ones too)*, reactive crowds, and light shows.	Users have reported that some aspects are too oversimplified. I could not find anything that immediately stuck out so will require further investigation.
	Some prior information is required. Tutorials do not teach you from the ground up.

Figure II-5 Screenshot #1 of Vinyl Reality

Figure II-6 Screenshot #2 of Vinyl Reality

4. DJAY

Djay is a DJ app that lets users mix music and do live performance on VisionOS and Meta Quest. It comes with virtual 2-turntable and mixer interface, integrates music streaming, supports MIDI/controller hardware. Depending on the platform the user is using and subscription plan, it includes various effects, auto-mix, looping, crossfader, etc. [15], [16], [17]

Pros	Cons
Cross Platform support	Pro tier is paid which locks off many features such as gesture controls, different views, custom playlists, and more.
Import your own tracks *(Spotify, Apple Music, and Soundcloud integration included)*	Some features are disabled for certain streaming sources *(Apple Music, Spotify)* due to licensing.
In-depth mixing and channel separation, such as vocal separation	Some users have reported that they feel that development of new AI features has taken priority over core usability features, like addressing crashing on slightly older hardware.
Very user-friendly	Does not Emulate real-world Controllers, it only interacts with them via USB.
Very feature rich	
Highly rated and polished with what appears to be user-friendly UI / UX	
Free tier has many of the essentials and is sufficient	

Figure II-7 Screenshot #1 of DJAY

Figure II-8 Screenshot #2 of DJAY

5. CONCLUSION

These findings and preexisting solutions showed that the idea of Rave-Spin XR could be done. These preexisting systems allow users to interact with virtual turntables, mixers, sequencers, and more, all in XR environments. This gave me a great deal of confidence going into the project as I knew that technology had caught up to the point where real-time manipulation of music and audio effects was possible, all without major compromises, and all while running locally on consumer XR headsets.

B. TECHNOLOGIES RESEARCHED

1. HARDWARE

For hardware it's important that the headset is capable of XR, has a good user base, and should have abundant information surrounding development. It should also be compatible with open standards. The Meta Quest 3S for example, runs on the Android Operating System and is compatible with the OpenXR standard. This means that little to no extra development is required to get Rave-Spin XR to run on the headset compared to other versions. Any builds of Rave-Spin XR that run on the Meta Quest 3S will also run on PICO headsets too as both run on Android and comply with the OpenXR standard. [18]

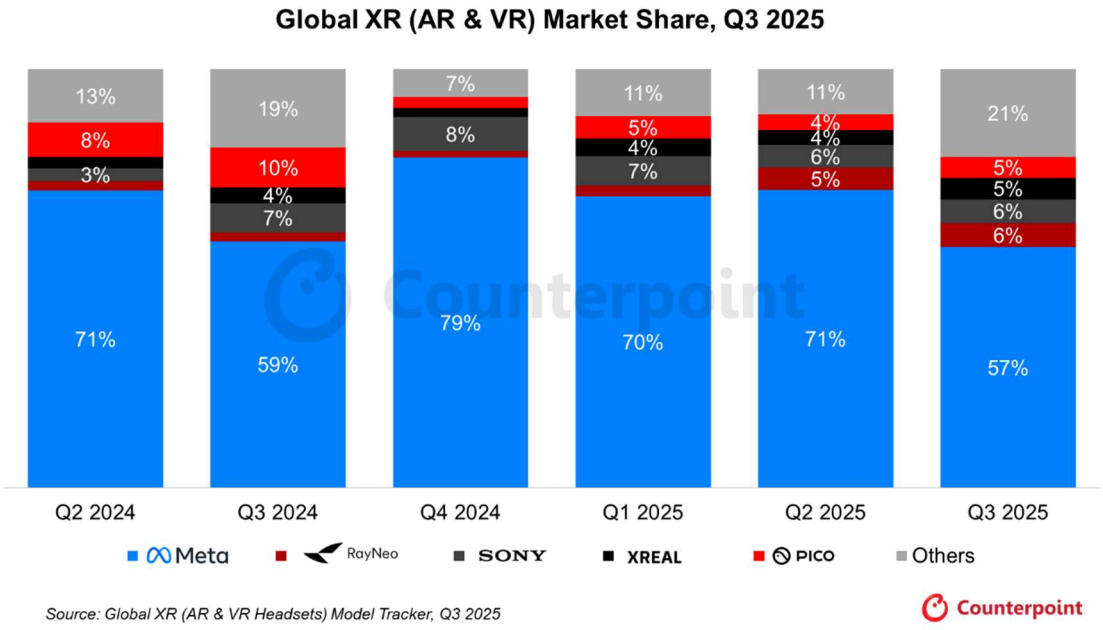


Figure II-9 Market Share of XR Headsets in 2025

One of the benefits of Godot *(the main toolkit used to develop Rave-Spin XR, will explore this further in section II.B.2.d) page 28)* is that when we are developing for one headset, thanks to several advancements in the world of XR, such as the adoption of the aforementioned new global standard of XR functionality called OpenXR, and most headsets being run by derivatives of Android, if we target one headset, there's a high chance it will work on others with no extra work.

Other headsets, such as Sony and RayNeo, are less explicit with their open Android and OpenXR support so extra work might be needed to target those platforms.

From the Figure II-9 Market Share of XR Headsets in 2025, we can see that the Meta platform is the most predominant XR headset platform. [19] Meta also have very affordable options, such as the Meta Quest 3S which is one of the most popular headsets on the market due to its fantastic price-to-performance ratio [20], and is part of the reason why it was used as the headset to develop the project on.

2. ENGINE

For projects such as these, they are created using an engine. An engine is a mature software framework and collection of tools that abstracts low-level systems such as rendering, physics, and audio processing, allowing developers to interact more easily with the underlying sub-systems.

For the project scope, it was unrealistic to attempt to try creating an engine to handle rendering and heavy-duty audio processing. I have made a game engine before in the past [21] and from first-hand experience know about the high difficulty of getting even simple things rendering and started.

As Rave-Spin XR is targeting low-powered android hardware, this meant I would have had to learn OpenGL *(the main Graphics library for Android)* from scratch, and how to do XR rendering on top of that. This would have been unfeasible for the timeframe. Although a custom solution would theoretically offer the best performance, as I would have complete control of thread groups and resource management, and I have previous experience with FMOD so audio manipulation wouldn't have been a major issue, to complete this in the timeframe, a preexisting and mature engine was required.

There are several engines in the market, and I had to compare each one to determine the most suitable one for this project. The top contenders are Unreal, Unity, Godot, Gamemaker, Flax, and several others.

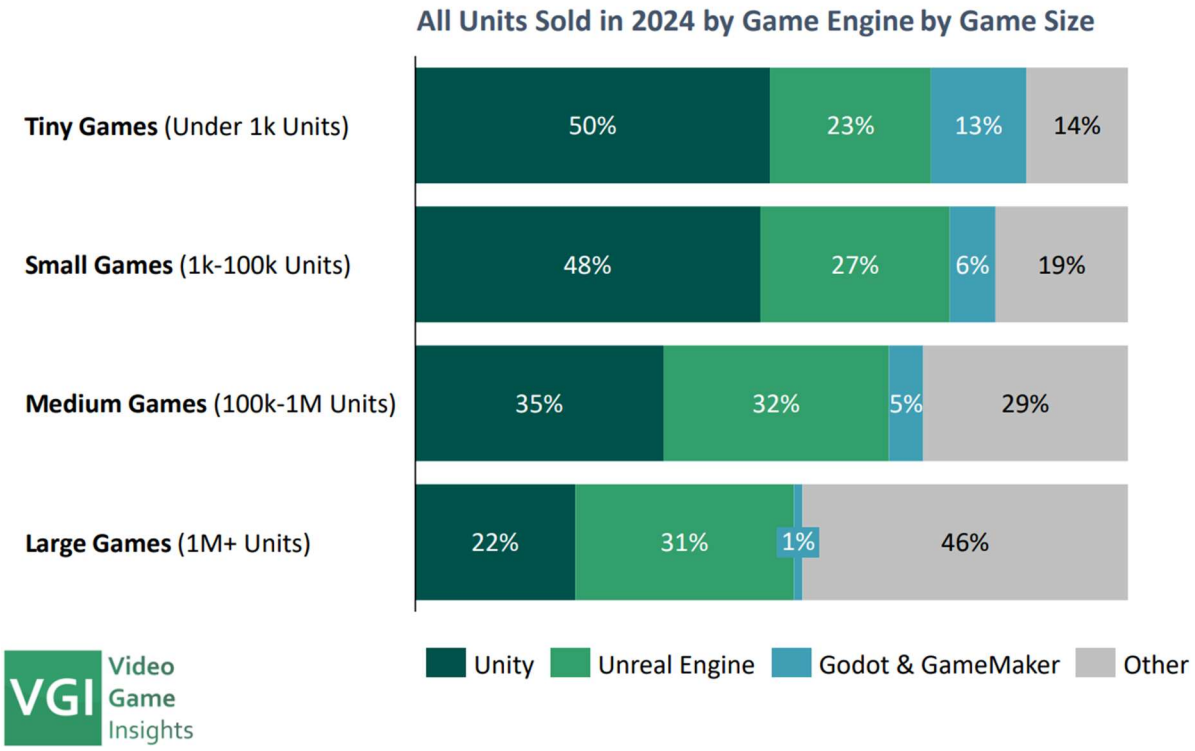


Figure II-10 Most popular Game-Engines used for release video games in 2024.

A) UNREAL ENGINE

I made a couple of projects before in Unreal Engine and have a great deal of experience with it. I am extremely comfortable with C++ and blueprints scripting.

I am also aware about the faults of this engine and where it may fall short.

The default 3D rendering pipeline needs to be changed to the forward rendering pipeline instead of the default deferred rendering pipeline so that it may run faster while using fewer resources [22]. I do not have as much experience with a forward rendered pipeline.

From a rendering perspective, Unreal Engine is the best contender. Unreal Engine also boasts its interoperability with VR. It's an industry standard engine and has great support for android. [23], [24], [25]

Although we can see from Figure II-10 Most popular Game-Engines used for release video games in , it is indeed one of the most popular engines today, I didn't choose unreal engine for only two main reasons:

- It is **not open source** [26]. Rave-Spin XR is FOSS so that others may add their own equipment and customise the program however they may wish. Unreal Engine, from my own experience does not have an easy mechanism to allow for advanced after-market user modifications. Unreal UGC *(User Generated Content)* exists [27], [28] but from previous experience it was limited in its capabilities. If I wanted to include full customisability, a "mod kit" would have to be developed, and this would take an unknown toll on the project timeframe. Licensing issues also exist for developing a "mod kit" and is uncommon for Unreal Engine titles.

- Previous **bad experiences with audio issues**. Unreal Engine default audio engine has fantastic audio capabilities, and it is highly sophisticated, especially for games. For this project, low latency, and multiple sounds being in memory and being able to be concurrently played at once is of utmost importance, and from previous projects I have run into issues with Unreal such as audio cut-offs, drops in performance when multiple FX were active, and more issues. Although these issues may have been fixed with newer engine versions or could have been fixed by editing the engine source code or implementing my own audio subsystem *(perhaps with FMOD)*, I believed this to be a major issue with this approach. If I had more time, I would have tried unreal engine, but for the timeframe provided, something that would have provided more certainty of working was preferred.

B) UNITY ENGINE

Unity is a game engine which also is widely used in the industry.

It has fantastic XR capabilities and can render easily on very weak hardware. It, however, is not open source, and the source code, unlike Unreal Engine and Godot, is completely hidden to free-tier users [29].

I have never used Unity before in the past so a learning curve would undoubtedly be present. I also found users complaining about audio latency and performance issues, particularly when a great number of FXs are active. [30], [31]

Like Unreal Engine, it is not open source, so it suffers from the same issues that stem from that. I did not choose Unity for these reasons.

C) FLAX ENGINE

Flax Engine is another game engine that also supports Android.

I have recently started using this engine and although it shows great promise, it still doesn't support VR.

The online Trello Board showing the engines progress specifically mentions VR/XR support as 20% completed.

It has stayed that way since 2016. [32]

I chose not to proceed with Flax Engine as it was not possible to say with absolute certainty that the project could be completed in the given timeframe with this software. The Flax Engine, for all its features and benefits, was not mature enough. Especially in the area of XR development.

D) GODOT ENGINE

Godot engine version 4.6 is the engine I decided to go for as it has very robust XR functionality, such as gesture controls, spatial awareness, and multiple libraries to extend it even more.

The default audio engine is also extremely modular and supports custom bus layout. The main thing which led me to choose it were the projects created by Bryan Duggan, such as his Godot audio sequencer, were able to easily handle multiple concurrent sounds, all playing on tempo, concurrently, with almost no performance loss. This proved that the default audio capabilities alone were capable enough to handle what Rave-Spin XR aimed to do.

Godot is also open source and features a rich ecosystem of plugins and tools which proved to be instrumental in the projects progress, such as the Godot-XR-Tools plugin [33], Rhythm notifier [34], and Godot XR Hand Pose Detector [35]. The community behind Godot is appears to be large and passionate, especially recently as many Unity developers have moved over to Godot in recent months. [36]

In future iterations, it may be possible to implement a plugin for Twitch / YouTube / Discord integration. If not, then it's possible to compile your own custom plugins to use with Godot, as I had done with the FFMPEG framework *(see section XIII.A "FFMPEG GD-Extension", page 184)*.

One issue was that I had never used Godot before, and I had run into many complications with its own scripting language "GD-Script." Although using C# and C++ is possible, it is advisable to primarily use GD-Script as it is more tightly integrated with the Godot Engine [37]. Debugging and documentation generation was better suited to GD-Script in my experience developing this project.

Figure II-11 Screenshot of RaveSpin being developed in the Godot Engine

3. BLENDER

To make the various 3D models and animations, I used Blender.

Like Godot, it's free and open source and has an incredible ecosystem of plugins and tools and the community is equally large and enthusiastic. Another feature of Blender which is of particular importance to this project is its ability to export to **GLTF** file format. This means that whatever names I give the various meshes and components remain persistent between Godot and Blender, and similar meshes can be "linked" instead of unique.

For example, this means that when we import a model into Godot that has 50 shapes, rather than Godot storing 50 individual shapes in memory so that it can render each one individually, it can instead keep just one shape in memory and render it 50 times. It's small savings like these that help with the performance of Rave-Spin XR on mobile hardware.

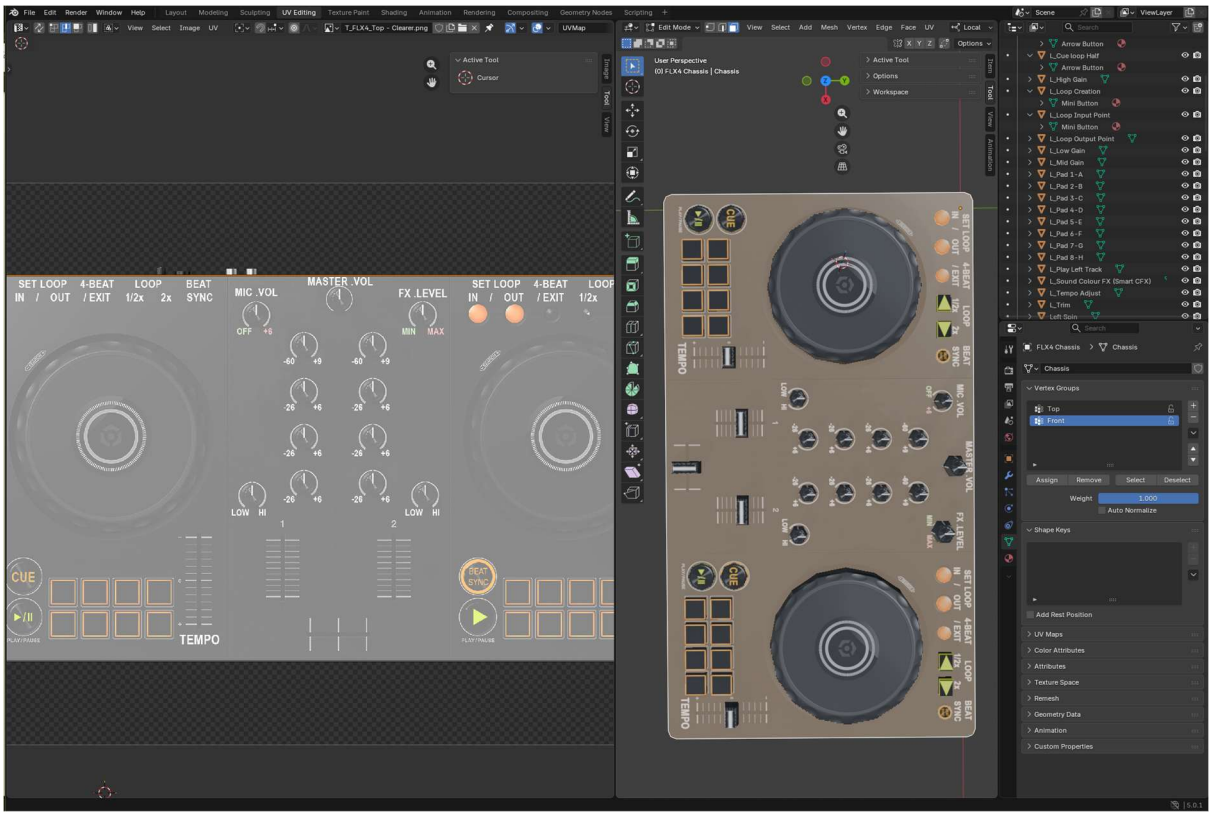


Figure II-12 3D models being created in Blender, to be imported into the Godot Engine

A) GLTF MODEL FORMAT

As mentioned previously, both Blender and Godot accept the GLTF file format for models. This is perfect for this project because GLTF retains the hierarchy or “scene tree” of a model and its sub-components [38].

If our model has multiples of the same mesh, like a button, rather than each mesh being a unique 3D mesh of a button that gets repeated everywhere, they can all share the same shared button mesh, leading to performance and storage savings. [39]

Figure II-13 Scene tree of a DJ Controller mesh in Blender

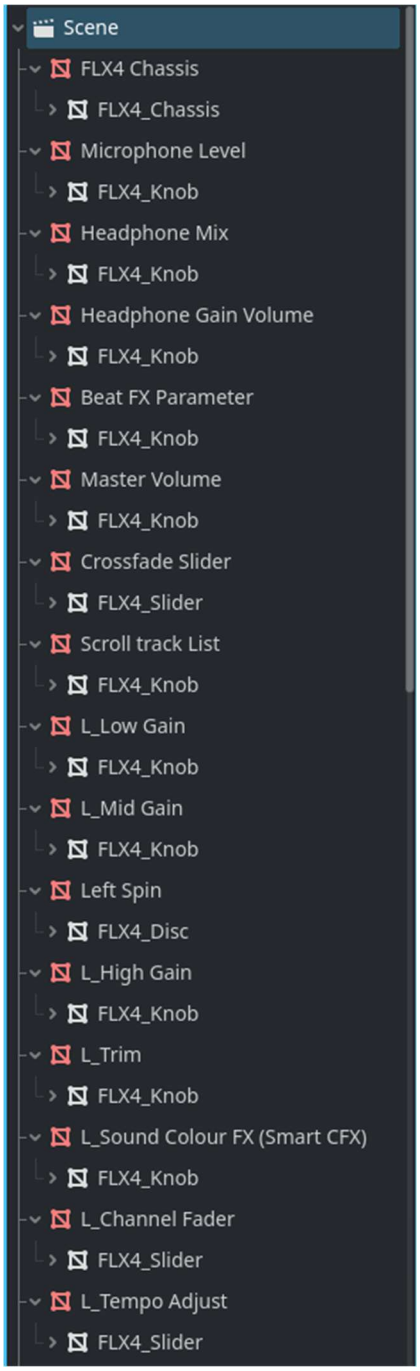


Figure II-14 Scene tree of the same DJ Controller mesh imported into Godot

4. FL STUDIO

For making sample tracks to use for use in the final version and for testing playback, I used FL Studio 20. It is the DAW *(Digital Audio Workstation)* that I have the most confidence and experience in.

Any other DAW is also fine as it is only .WAV, .MP3, or .OGG audio files that we would want.

5. FFMPEG

"A complete, cross-platform solution to record, convert and stream audio and video."

FFMPEG is a framework that anyone can download and use to analyse audio and video files and convert them into any other format that we may need. For the purposes of Rave-Spin XR, I used the FFProbe submodule that comes included with it, so I could analyse audio files, and extract information from that file such as its metadata which includes track title, album names, artists names, song BPM, sometimes even what genres could describe a song. [40]

Using the FFProbe submodule I could also generate waveforms of songs *(see Figure II-15)* which can be very useful for determining if a song needs more volume or less volume, or "gain" as it's called.

This is discussed further in Section II.C.4 "Waveforms" *(page 38)*.

Figure II-15 Waveform image of a Song generated using FFMPEG.

C. OTHER RESEARCH

In this section I will be discussing the extra research I had done regarding other components of the system and the development of Rave-Spin XR.

1. ADB (ANDROID DEBUG BRIDGE)

To effectively interact with our headset and deploy any changes, I used **ADB** (Android Debug Bridge). This is a tool that lets us talk directly to whichever android device we want, in this case, it was the Meta Quest 3S XR headset. Having this tool installed, Godot can automatically detect the headset and deploy the latest build of Rave-Spin XR onto the headset.

I could also call ADB directly to perform actions like extracting all recorded songs on the headset, like what we have with some of the pre-included dev tools *(see section IV.C.2, "Extract Recordings from Headset (Python Script)", page 125)*.

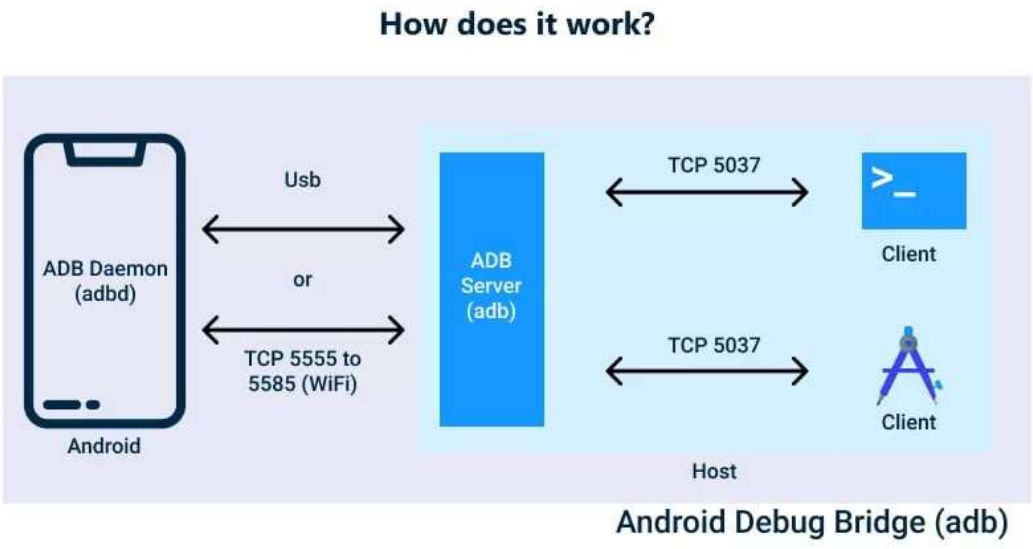


Figure II-16 Diagram of ADB overview.

2. GODOT PROFILER

One of the helpful tools that comes prepackaged with Godot is its inbuilt profiler. After understanding how to use it effectively, it proved to be an invaluable tool. The profiler works by checking how long each frame took to render, and specifically, how much time each function took to execute in that frame.

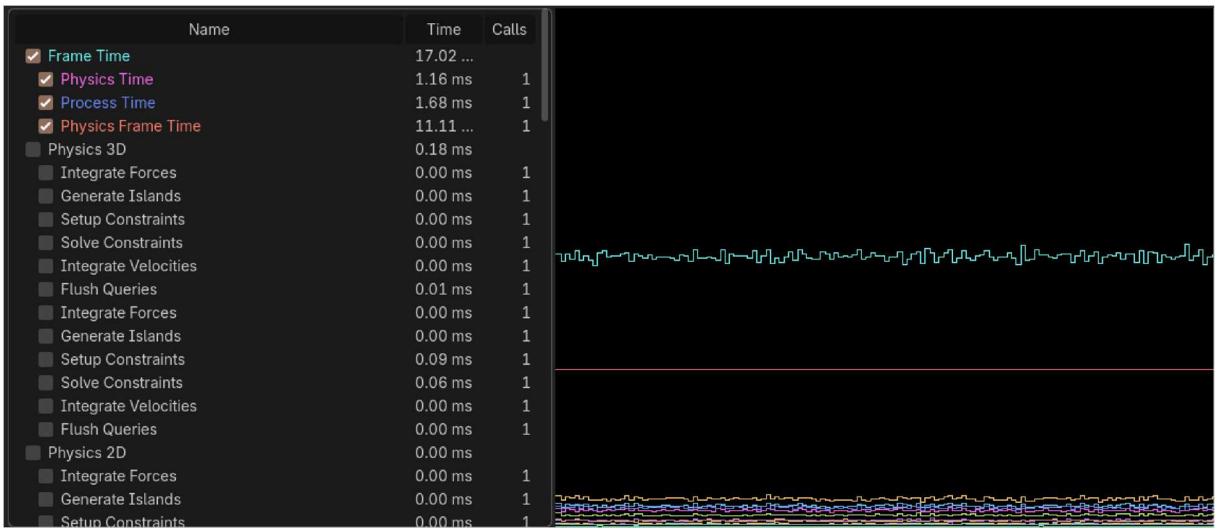


Figure II-17 Screenshot of Profiler monitoring RaveSpin in action.

From what we can see in Figure II-17, we can see how many milliseconds each part is taking to render. The ideal target is 60 frames per second, which is 16.66 milliseconds per frame. Keep in mind, that Process and Physics are run on two separate threads. From this profiler we can see what functions are taking up the most amount of time in each class.

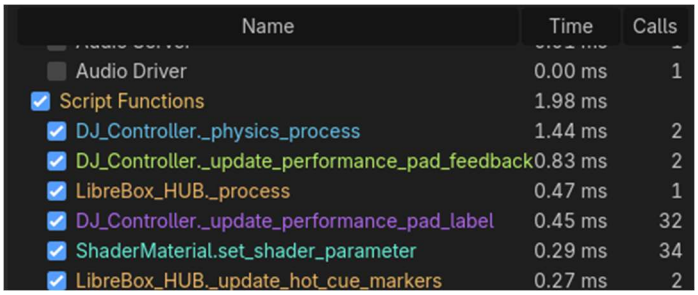


Figure II-18 Screenshot of Individual components and scripts taken to render each frame. DJ Controller is taking the longest time.

It is thanks to this profiler that I was able to determine the greatest causes of performance loss and was able to make Rave-Spin XR much more performant. More information can be found regarding this in Section IV.A.5 “Performance issues with DJ Controller” *(page 114)*.

3. AUDIO BUS LAYOUT

To make an accurate emulation of the Pioneer DDJ-FLX4, or any other DJ equipment, it's important to know how audio is routed through it. The main flow of audio will usually follow the path seen in Figure II-19 Diagram of a typical Audio signal path for most electronic music [41], [42], [43].

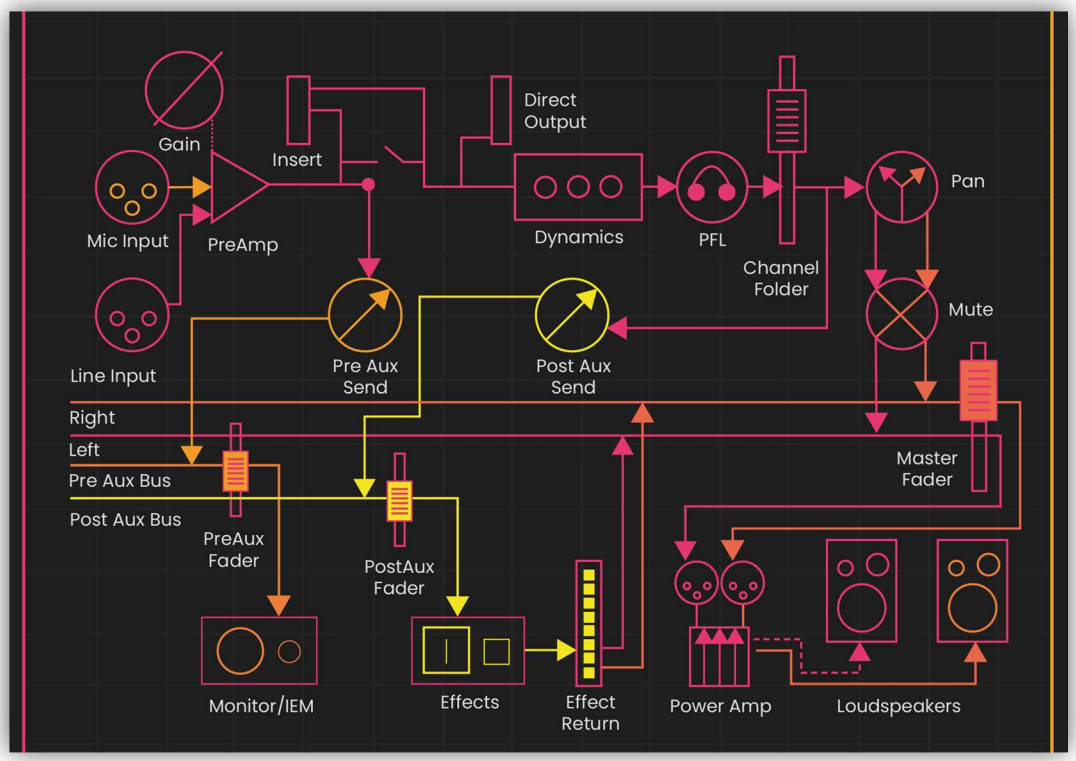


Figure II-19 Diagram of a typical Audio signal path for most electronic music handlers.

From this we can see that there's 2 important input sections:

- Channel X Input (Imagine this is our "Line input")
- Channel X Effects

Whatever audio track is put into Channel X *(The pioneer FLX4 has 2 decks, so 2 channels)* will be mixed in with the master output [44]. This means there's an audio bus for our Channel 1 input *(to visualise the audio waveform before any effects are applied)*, Channel 1 FX, do the same 2 for Channel 2, and then a master output. This meant that a total of about 5-6 audio busses would be needed.

Here's a sample diagram of the audio bus layout to be used in early version of Rave-Spin XR *(Figure II-20)*:

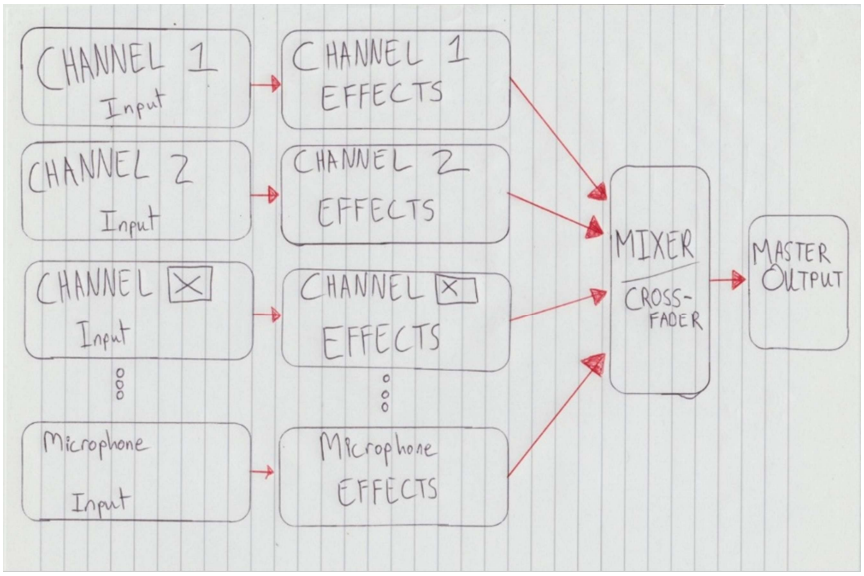


Figure II-20 Initial Sketch Diagram of Audio Bus layout.

Once implemented into Godot it looked like the following:

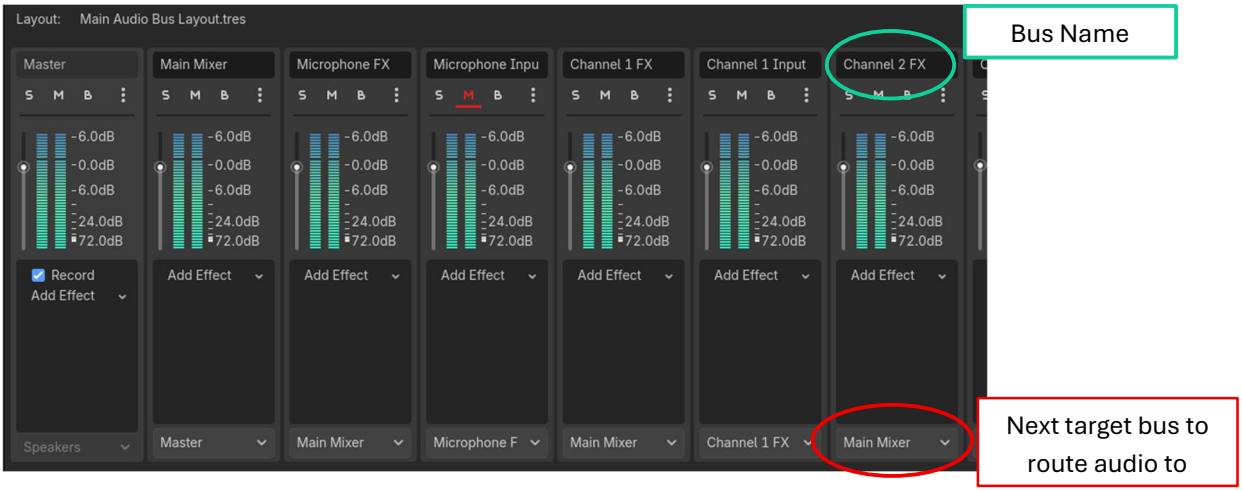


Figure II-21 Screenshot of first half of the old Audio Bus Layout used in RaveSpin.

Figure II-22 Screenshot of second half of the old Audio Bus Layout used in RaveSpin.

From what we can see in figures Figure II-21 Screenshot of first half of the old Audio Bus Layout used in RaveSpin" and Figure II-22 Screenshot of second half of the old Audio Bus Layout used in RaveSpin", going from a right-to-left direction, we can see that each song will initially go into an "input" bus.

That input bus could then be passed into another bus for applying various audio effects.

I ran into an issue, however. Godot can route audio from one bus to another, but any adjustments made in one bus do not carry over. So, if I have a song in "Channel 1 Input" and I increase its volume and apply an amplify effect to further increase the gain of the audio, these changes do not carry over to "Channel 1 FX".

The only time changes and effects persisted from bus to bus, is when they were immediately routed to the Master Output bus. To fix this I refactored my design and simplified it to use as few busses as possible so that anytime we need effects and adjustments to carry over, they can. We can see the redesigned audio bus layout in Figure II-23 Screenshot of current Audio Bus Layout in RaveSpin.

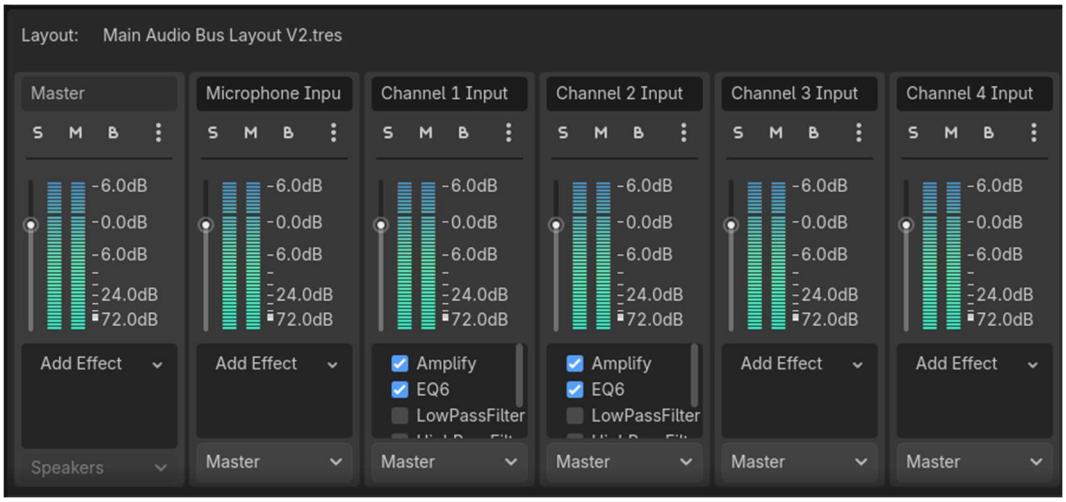


Figure II-23 Screenshot of current Audio Bus Layout in RaveSpin

The order of filters does also have an impact, to address this I developed my own custom class called "Bus Manager" *(see section III.C.3.b) "Bus Manager", page 77)* which manages all effects and makes sure the correct order is retained. We will always want the following effects order:

1) **Amplify** filter first, this will amplify or subdue any song we pass in.
2) We then want to pass it through an **EQ filter** to specify what parts of the audio spectrum we want. This allows us to boost or reduce the bass, midtones, and high tones of a song.
3) We then want to apply a **low pass filter**. This will cut off any frequencies above a certain threshold.
4) After that we then want to apply a **high pass filter**, this will act as an opposite to the low pass filter, as this will only allow frequencies above a certain threshold to pass through. Low pass filter and High pass filter will never both be activated at the same time. Thanks to our aforementioned "Bus manager," only one will be activated at any one time.

4. WAVEFORMS

Another factor we also need to be aware of is the Waveform of the audio we will be using. All audio inputs will be taken in in their original form first. This will let us analyse their waveforms before any effects are applied. We can check easily if any given audio wave form is normal, or if it requires any additional amplification or reduction first based off what the waveform looks like.

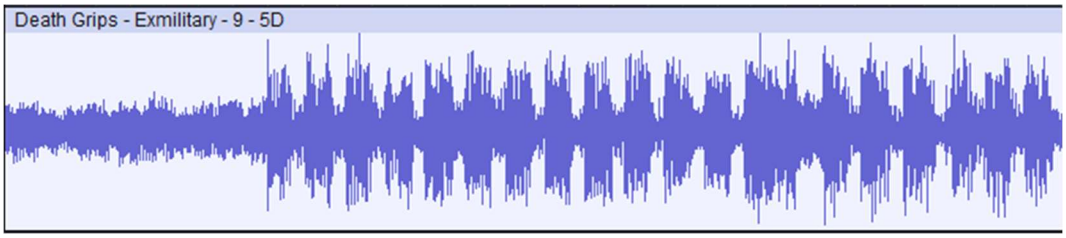


Figure II-24 Normal Audio Waveform

This figure shows a normal waveform, it is not too overblown or too quiet, therefore there's no amplification or reduction required. It should be ok to apply effects to.

Figure II-25 Undersaturated Audio Waveform

This audio is going to be too quiet. We need to apply more "Gain" *(basically its volume)* before we apply effects otherwise, we might not even hear them [41].

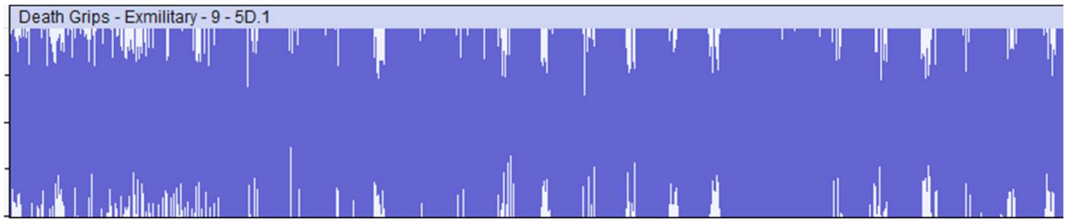


Figure II-26 Overblown Audio Waveform

There is a chance that this audio will be too loud. This audio needs to reduce its Gain otherwise the sound, effects included, will have very little definition and will sound muddled.

5. DJ EQUIPMENT

A) CDJS

One of the pieces of equipment that DJs can include in their setup is a CDJ. A CDJ typical consists of large spinnable wheel that lets users “scratch” the music.

This is where the music jumps backwards or forwards in during playback, depending on the scale of the spin taken by the user.

We can see an example of one with Figure 0-6 Picture of the Pioneer CDJ-3000 *(page 6)*.

B) PERFORMANCE PADS

Performance pads are another set of equipment that DJs may have. It allows them to perform multiple functions, such as firing off “Fire-&-Forget” audio samples. It also allows them to map different audio effects to different pads, allowing for extensive control at runtime. They can also set up “Hot Cues” which are set points on a song that the DJ can immediately jump to at runtime.

We can see an example of one with Figure 0-5 Picture of a Pioneer DDJ-XP2 *(page 6)*.

C) DJ CONTROLLERS

The Pioneer DDJ-FLX4 is one of the more common and popular DJ controller board out there. It is capable of achieving the majority of the requirements of a DJ and is also recommended for beginners too [8], [45]. The workflow and usage of this Controller is similar to other Controllers on the market, reducing any required onboarding. It comes with Jog wheels for scratching of songs. It comes with performance pads which we can use for setting up “Hot cues” *(points that we can jump to in our song)* and play other audio samples, and more. Using those same pads, we can also temporarily apply various audio effects for as long as we have the buttons activated for. More on this in section III.B.3.a) “Buttons & Pads” *(page 61)*. Thankfully there exists numerous resources out there about how this board works exactly. [46], [47]

6. FMOD

FMOD is a sound processing library that is widely used in the industry for its powerful suite of tools regarding audio processing. [48], [49], [50]

Many games engines use FMOD for their audio. Unfortunately, there is no native FMOD extension for Godot so if I wanted to use FMOD for this project, I would have had to compile FMOD into an Android-compatible GD-Extension *(an extension that Godot can recognise and use)*. Using FMOD I would have had more audio capabilities and effects available to me but for the purposes of this project, the native Godot audio capabilities were more than sufficient.

7. MUSICBRAINZ

MusicBrainz, an open music encyclopedia that collects music metadata and makes it available to the public.

MusicBrainz is an open-source collaborative effort to record metadata for all music. If we have the ID of a song, artist, or album, we can query the database and receive almost all of the information we could want, such as artist names, guest stars, song BPM, and more. [51]

Although this project will not be making any queries to MusicBrainz, the infrastructure is in place to make sure that the system could reliably gather the missing metadata from a song by querying MusicBrainz instead. All song, artists, and album resources, all contain MusicBrainz IDs for later querying.

D. EXISTING FINAL YEAR PROJECTS

1. AUGMENTED REALITY BILLIARDS ASSISTANT – LEE COX, 2025

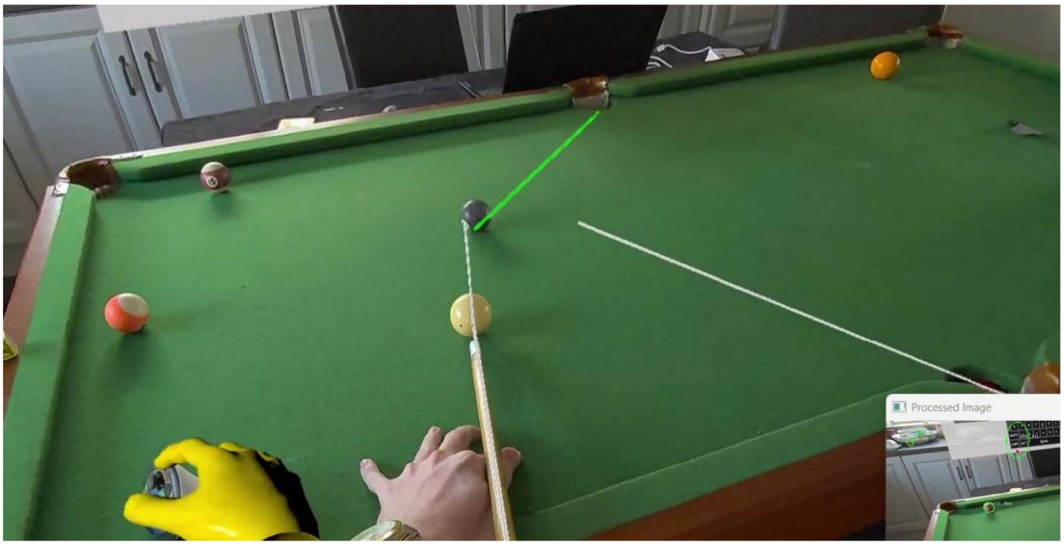


Figure II-27 Screenshot of AR Billiards Assistant by Lee Cox

A) DESCRIPTION

This is a XR application is a visualiser tool for English pool. Through wearing an XR headset, such as the Meta Quest 3S, it enables the user to see line trajectories of shots they line up with their cue. It is created with the same engine as Rave-Spin XR, the Godot Engine, and was also tested with the Meta Quest Headset platform.

B) COMPLEXITY

The complexity of this project comes from multiple factors. When the project was being developed, developers were unable to send the video feed from the headset to an external library for image processing, for understandable privacy concerns, this however is no longer an issue as Meta has since enabled this functionality.

This project circumvented this issue at the time by sending the video feed to a local server over **WLAN** *(Wireless Local Area Network)* for image processing. After the image processing was completed and the most optimised ball paths were calculated, the results were fed back to the headset over WLAN.

C) TECHNICAL ARCHITECTURE

The front end of the project is made with Godot. The same engine that is used for Rave-Spin XR. Godot is also responsible for many of the physics calculations at runtime. When the video feed is sent to the local server via WLAN for image processing, there are a number of python libraries used such as web sockets, NumPy, Mss, and Async-IO.

D) STRENGTHS

The project achieved all of its deliverables. With a reported accuracy rate of 80% of the projected path trajectory vs the actual trajectories. It showed that accurate hand tracking is feasible and that Godot does indeed have good networking capabilities.

E) WEAKNESSES

As mentioned, the constraint of not having access to the video feed of the headset during runtime to be used by external python processing libraries has now been addressed by Meta. If this project were to be done again, I believe an attempt could be made to have all processes running on the headset alone.

E. CONCLUSIONS

1. HARDWARE

So, for this project I decided to do all my testing on the Meta Quest 3s. It is one of the more affordable XR headsets on the market and runs on non-proprietary software such as Android and OpenXR. We also had a few of them available to borrow from Technological University Dublin, so this simplified matters.

2. SOFTWARE

The main piece of software responsible to build and assemble this project is the Godot Game engine. I used Godot to combine all data, such as 3D models, audio processing logic, scripts, and the required GD-Extensions *(Godot Engine Extensions)*, such as FFMPEG for audio static analysis, Godot-XR-Tools for XR functionality, and Rhythm Notifier for downbeat detection of songs during playback. These extensions were easily imported into the engine.

To communicate between Godot and the Headset, Godot has built in functionality for this use-case. Godot can make calls to **ADB** *(Android Debug Bridge)* to simplify deployment on the headset. I can also use ADB in some of the pre-included dev tools like "Extract Recordings from Headset.py" which will use ADB to enter the headset and extract all recordings made by the application. *(see section IV.C.2, "Extract Recordings from Headset (Python Script)", page 125)*.

I also created a custom GD-Extension which will have the FFMPEG library included. Unfortunately, due to multiple runtime permission issues with the Meta Quest 3S and Android, its functionality is limited for now *(see section IV.B.2 "Using External Compiled Binaries with Android/Meta", page 117)*.

3. LANGUAGES USED

This project is written primarily in GD-Script. This is because it offers the best debugging capabilities for our project which proved invaluable as the project progressed. Many of the modules inside Godot work best and natively with GD-Script. It also has automated documentation generation which proved quite helpful too. C# is an option but wasn't chosen because I am not as comfortable with it and it is not as natively supported and integrated into Godot as GD-Script.

For making our custom FFMPEG GD-Extension, we used Java so that we could include the FFMPEG toolkit suite in android studio *(see section XIII.A "FFMPEG GD-Extension" page 184)*.

C++ was also used in the debugging processes as many error messages thrown by Godot would give either the relevant error in GD-Script, or the C++ code segment in the relevant file that was throwing the error.

III. SYSTEM DESIGN

In this chapter we will go over the design of the proposed system, detailing the methodology, system architecture, and the key components necessary for implementation. It provides a structured blueprint for the development process of Rave-Spin XR.

A. SOFTWARE METHODOLOGY

When developing a project, there's multiple different methodologies and approaches we can take. All with their own pros and cons. For the sake of Rave-Spin XR, it was just me working on it so that was taken into consideration.

1. AGILE

Some methodologies exist such as Agile, which is an iterative, cycling approach to software design, with daily reviews on the teams progress and "sprints" which includes planning, task allocation, execution, review, and adjustment phases. [52]

This all allows for quick feedback and allows the team to address issues as they arise and enhance project quality progressively.

Agile was inappropriate for this project as it was just a solo developer *(myself)* working on this.

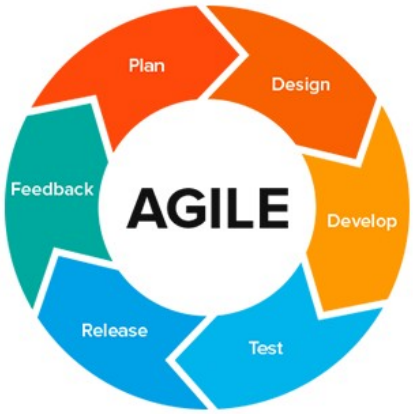


Figure III-1 Agile Diagram

2. WATERFALL

Another popular methodology is the Waterfall methodology which is a linear approach to software development breaking the whole process into 7 distinct steps.

Changes to the initial requirements can be slower but the path of development is straight forward from requirements gathering to deployment and then to maintenance.

This was unsuitable for this project due to the ever-changing requirements and the non-static nature of this project.

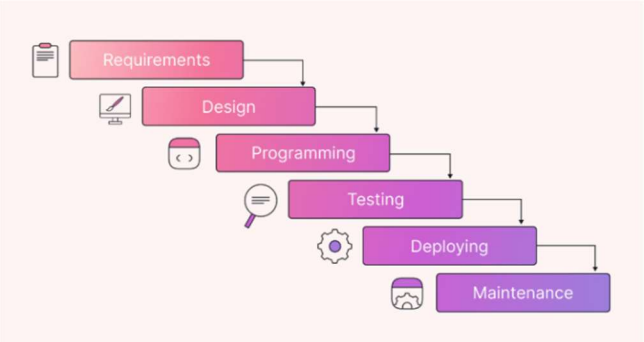


Figure III-2 Waterfall method Diagram.

3. R.A.D – RAPID APPLICATION DEVELOPMENT

Rapid Application Development was the methodology selected as it provides the best of both worlds. It's perfect for a solo developer and can handle changes to requirements quite well. It mostly focuses on making prototypes, testing them, getting feedback, and iterate on the prototype some more. [53], [54], [55]

You keep going until the prototype seems solid and you then move to the construction phase which is where you refine it and release it then afterwards.

This focus on prototype development and feedback was useful because it allowed rapid feedback on what works and what doesn't. I could do this quickly and without delving too deep on something which might not have been worth it and focus more on what does work. Because I was also using the end product myself, I could also give immediate feedback.

The requirements weren't solid and unchanging, so waterfall wasn't ideal for the project timeline and agile is aimed towards a team so as a solo developer that didn't work either. RAD was the best alternative.

FDD *(Feature driven development)* was considered too, but due to my inexperience with Godot, I was unable to give an exact time estimate of certain features. I did not want to get into a cycle of planning a feature and having it take much more time than expected [56], [57]. RAD works because I was constantly iterating on a prototype and having it tested and evaluated every week via informal testing.

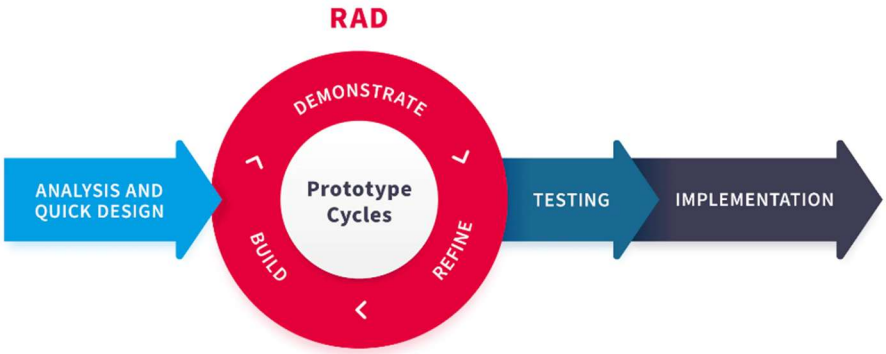


Figure III-3 R.A.D Diagram

B. OVERVIEW OF SYSTEM

The frontend of Rave-Spin XR is split into 4 main parts, they are The Arena, Librebox, DJ Controller, and Player. The backend, such as the Godot audio server, Audio Bus and Language managers, are all elements that run in the background that the user will never see but can receive updates from and manage from the frontend.

For the database / data storage, items such as saved songs and metadata, config files, are all stored on disk. Any songs that we import, will have their information saved into resource files that we can interact query efficiently. All together we can gather a rough idea of how Rave-Spin XR is structured with the rough system diagram in Figure III-4. The requirements of each use case can be found in Section III.D *(page 82)*.

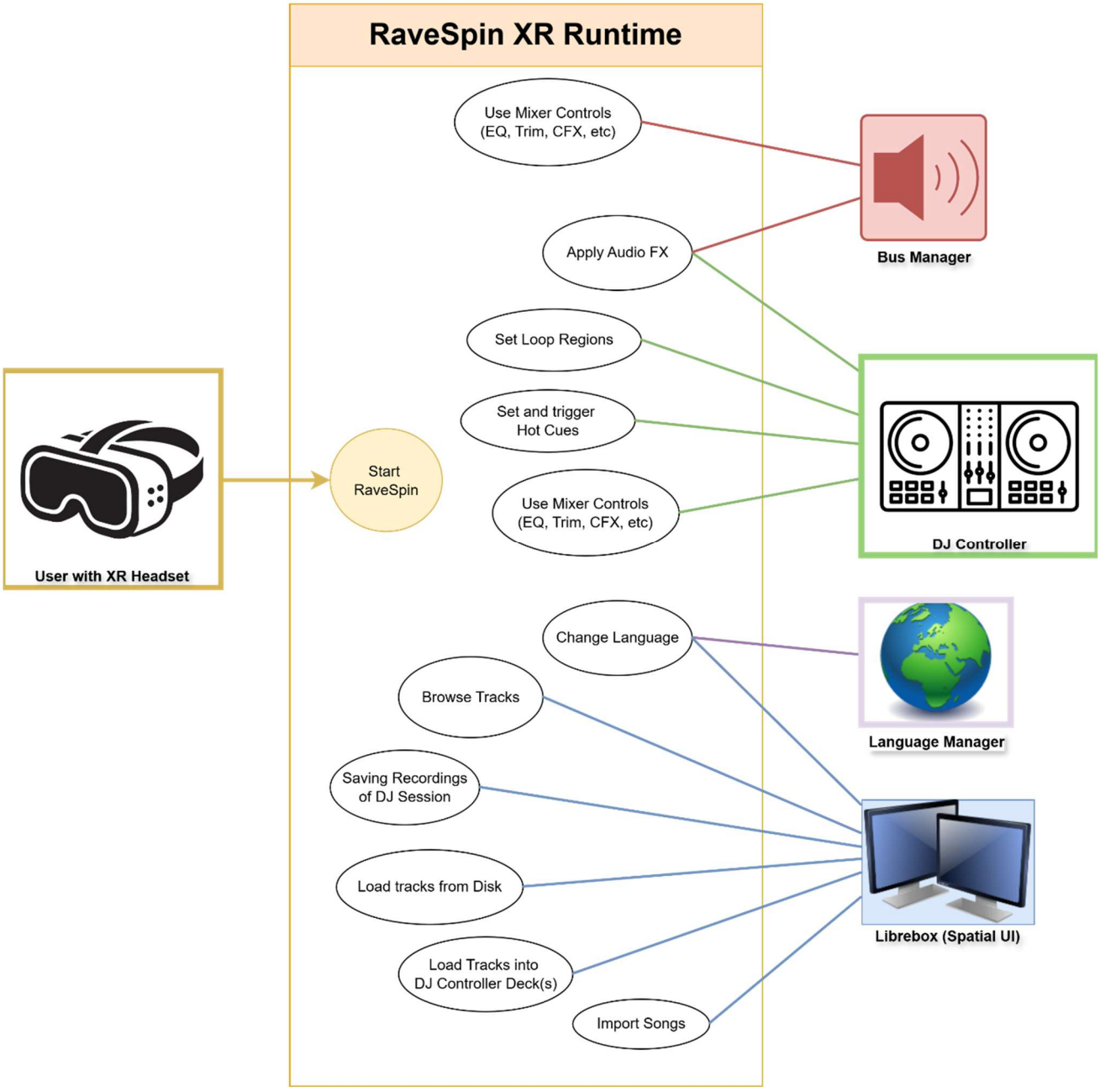


Figure III-4 Rough overview Diagram of System

1. ARENA

When launching Rave-Spin XR, Godot will require a “Starting scene.” This is the scene that the engine will always try to load first. In regular video games this would be a Main Menu that the player spawns into. In Rave-Spin XR, it’s where all the components that the user could want to interact with are loaded in and spawned.

This initial scene, which houses our DJ Controller, custom UI solutions *(both interactable and non-interactable)*, and various other elements, is called **the Arena**. The Arena is also responsible for making sure that OpenXR has launched successfully and that our headset can interact with the Player node inside the Arena scene.

Figure III-5 Screenshot of Arena Scene

2. LIBREBOX

All UI elements that the user can see, such as track selection, audio effects selection, language selection, need some way to be shown to the user. This is what Librebox is. It's a custom node that spawns into the Arena scene. All UI elements can be shown in 3D space, floating in front of the user.

The user can click on these UI elements using just their hands, the same way they would click on a floating touchscreen.

All UI elements are a child node of Librebox. Librebox handles all song CRUD operations, handles the audio effects policies, Language managers, Audio Bus Managers, all are children of the Librebox scene.

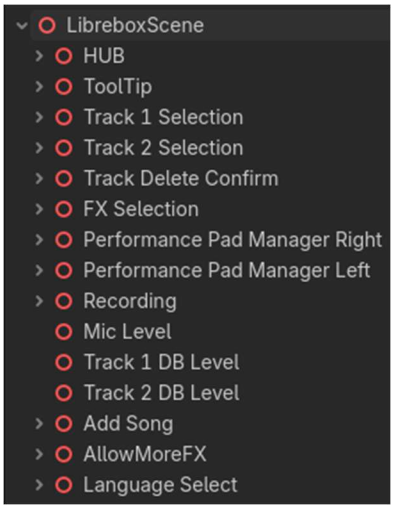


Figure III-6 Scene tree of the Librebox Node

A) XR TOOLS VIEWPORT 2D IN 3D

One of the challenges of XR development is how to interact with menus. Many options exist, such as having simple shape collisions that the Player could touch to activate different controls. What I instead opted to do for Rave-Spin XR was to take advantage of the Godot XR tools Viewport 2D in 3D. This works by taking a 2D image, a flat plane in 3D space, and then transplanting that 2D image onto that plane. This is the main idea behind 3D rendering with textures, but what makes this useful in this instance, is that any changes in the original 2D UI, reflect onto the 2D UI that has been transplanted onto a 3D plane.

Figure III-7 Example of a 2D UI scene

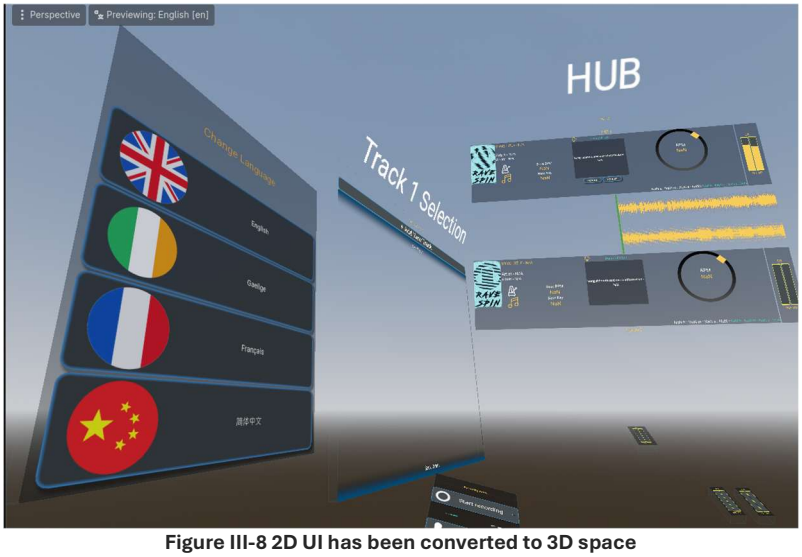


Figure III-8 2D UI has been converted to 3D space

This easy translation from 2D UI to 3D space, is the main backbone of Librebox UI. This allows us to use traditional 2D UI workflows and have them seamlessly integrate into 3D space.

Going forward, any 2D UI elements you may see, is converted to be used in 3D space.

B) XR HAND TOUCH INTERACTION

In previous versions iterations of Rave-Spin XR, UI interactions were done by doing a fist pose to activate a laser pointer that can be used to hover over UI elements. A Thumbs-Up pose would then be done to simulate a click on the UI element the laser was colliding with.

After some user testing, this proved very unpopular, so an alternative approach was needed. I saw users kept trying to extend out their arms to try to touch the buttons on the UI before I explained the Fist-Thumbs up combination that was required instead, so I thought it would be better if this was natively supported instead.

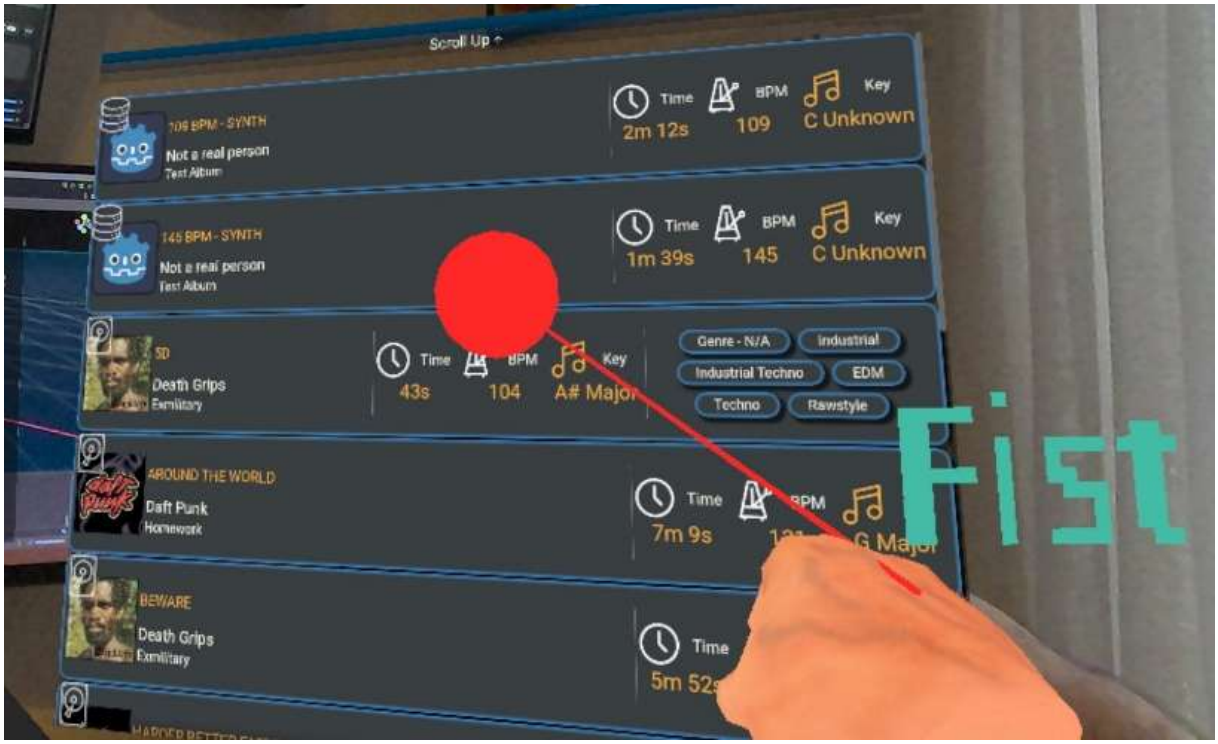


Figure III-9 How UI interaction used to work in RaveSpin (Laser pointer method)

After some testing, I was able to figure out how to make the XR 2D-in-3D viewports detect hand collisions. Using a custom node called “Viewport 2D in 3D Hand-Touch” that is attached to these viewports, I could then translate the position of this collision from global world 3D space to 2D local space. Once this translation step was done, we could have our custom node check what pose we were doing, this is so we could allow scrolling in any direction if we were doing a fist pose or an index pinch pose, and simulate the same “on click()” logic if we did another pose instead.

This custom Hand-Touch node is one of the core pillars of Librebox UI. Without it, UI interaction is much less intuitive and usable.

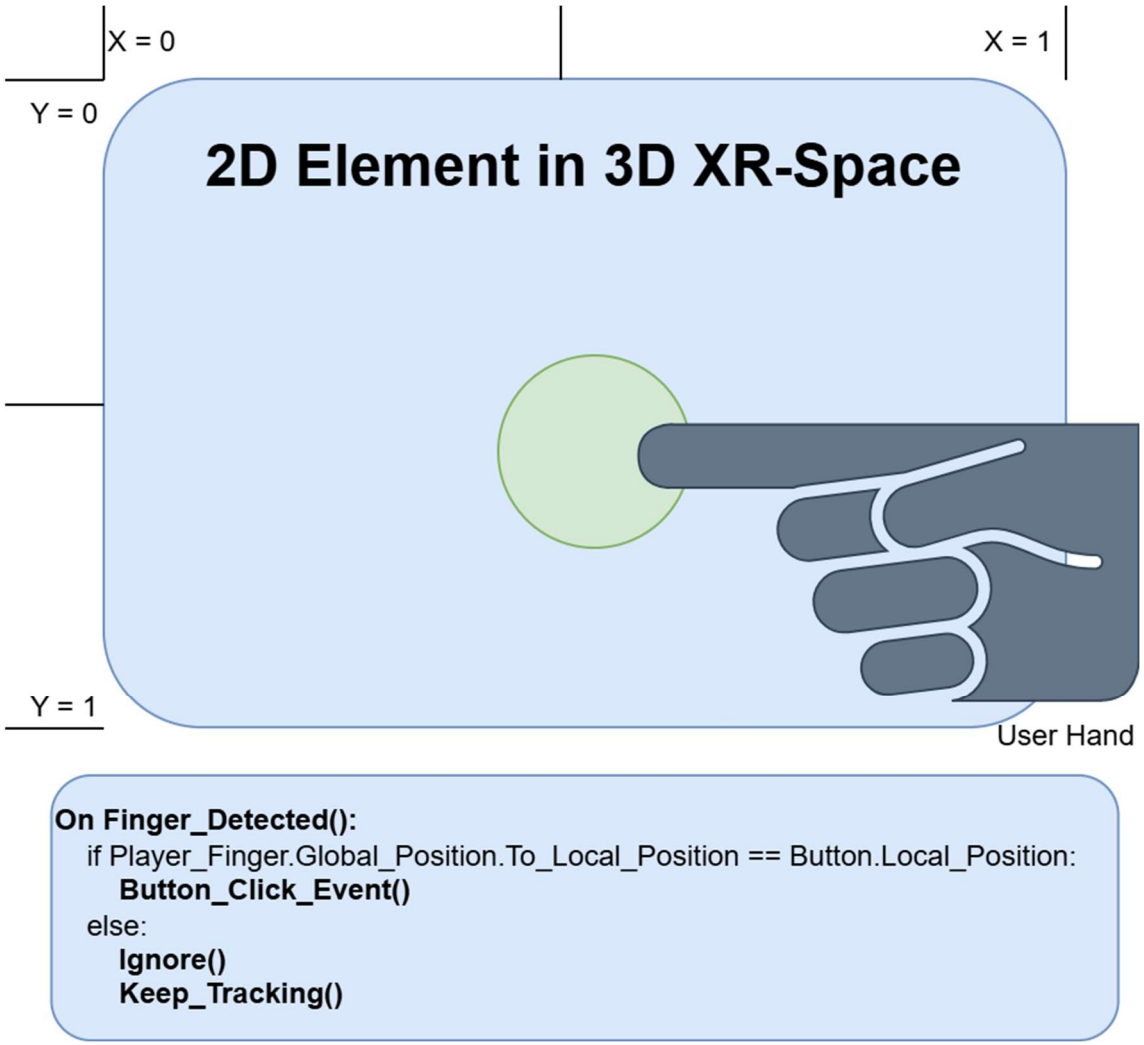


Figure III-10 Diagram of how the Viewport 2D-in-3D Hand Touch Node works.

I wanted to use the same naming scheme as the one used in the Godot-XR-Tools plugins as there was no functionality that does this already within the plugin and I was hoping to contribute this code to the Godot-XR-Tools repository as an open-source contribution. It wouldn't be a hard task as it's modular by design and has almost no hard dependencies on any specific classes or functionality from Rave-Spin XR.

C) HUB

The Hub is the main monitoring interface for Librebox, providing a real-time overview of both controller decks. Each deck is represented by a “Track Card”, which is also paired with a decibel meter *(see Figure III-13)* and a view of the song’s waveform *(see Figure III-14)* which pans with the song’s playback position.

Figure III-11 Screenshot of portion of Track Card

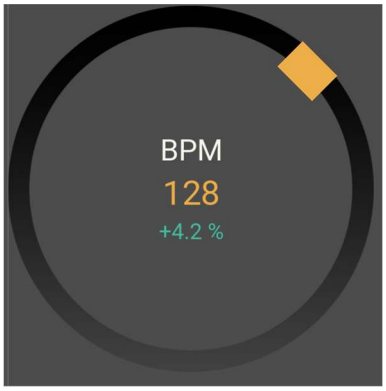


Figure III-12 Screenshot of BPM Spinner Gauge

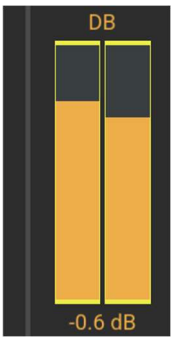


Figure III-13 Screenshot of DB Meter

Figure III-14 Screenshot of Song Waveform from Librebox Hub

Figure III-15 Dynamic list of all active audio effects

Each Track Card represents the currently loaded track's data, such as song title, artist name, album name, and album artwork *(see Figure III-11)*. The card displays the track's base BPM *(the song’s original BPM)* and its current BPM *(see Figure III-12)*.

A BPM spinner gauge *(see Figure III-12)* rotates in sync with the beat, activating a visual pulse on each downbeat and showing the percentage offset between the current BPM and the base BPM. The text then goes red for negative offsets, and teal for positive offsets.

The track's key *(e.g. "C Major", "G sharp Minor")* is also shown, along with an optional sidenote from the user for any personal notes about the song. At the bottom of each card, the current playback position and total track duration are displayed in a **MM : SS : MS** format. A song which is 65.4 seconds in will have the text change to “01:05:400”. There’s also a dynamically changing list for any audio effects currently applied to the channel by querying the Bus Manager.

Any effects that aren’t active are automatically removed *(see Figure III-15)*.

Between the two deck cards sit two waveform previews rendered through a custom Horizontal Pan shader. The waveform texture scrolls horizontally as playback progresses, with a play-head line in the middle that pulses from white to red on each beat. The waveform can also have up to 8 colour-coded hot cue markers, loop start and end markers *(visible only when armed)*, and a regular cue marker in cyan.

All marker positions are recalculated every frame based on the current playback alpha and passed to the shader. Each deck also has a stereo dB meter, which is a pair of vertical bars driven by a shader material that samples the Godot audio server to get the peak volume levels for a particular channel every frame. The meter displays the peak dB *(ranging from −48.0 dB to +6.0 dB, or "−INF dB" for silence)* and transitions to a red warning colour when the peak volume exceeds 0.1 DB.

Figure III-16 Screenshot of Librebox HUB – In English mode.

This is how the Hub looks when all components are joined together *(no songs have been selected. This is just the Godot editor preview)*.

D) TOOLTIP

When a player's hand hovers over any control on the DJ Controller, a tooltip spawns in 27cm above the component after a short delay. It then displays the control's name and a short, localised description of its function. This is to allow any users who are unfamiliar with the different buttons and components to learn their functions.

Each tooltip is displayed as a small billboard that continuously rotates to face the player's camera.

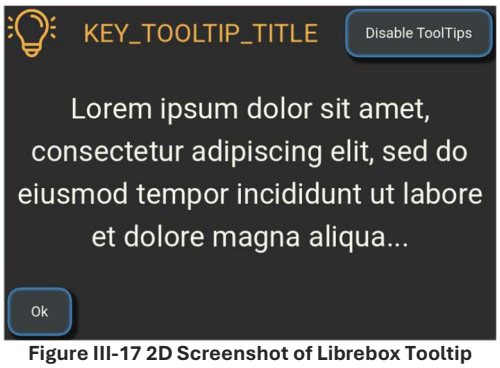


Figure III-17 2D Screenshot of Librebox Tooltip

Each component title and description has a key, which is then found in the translation CSV *(comma-separated values)* file, so the tooltip content updates automatically when the user changes language.

Tooltips animate in with a cubic-eased tween over 0.5 seconds and gently bobs vertically while open. If the user moves their hand to a different control while a tooltip is already visible, it smoothly transitions to the new position and updates its text rather than closing and reopening. A ticket-based system prevents race conditions during rapid hover changes. The user can also dismiss the current tooltip or disable all tooltips for the session entirely.

E) TRACK SELECTION

Track Selection presents a scrollable list of every song in the user's library, each shown as a horizontal card with the track's metadata.

Each track is also shown with a logo of the origin of the song *(Spotify, YouTube Music, Local Disk Storage, etc,)* that gets swapped out to the correct logo texture *(see Figure VI-6 "Screenshot of code to get the logo for a given music platform." page 149)* at runtime *(where the top-left white circle is located in Figure III-18)*.

The user can scroll via pinching and dragging the list or hover their hand inside the top or bottom hover zones labelled with "Scroll up/down." Tapping a card loads that track onto the respective deck and immediately updates both the Hub and the DJ Controller to reflect the change.

Each card also has a remove button, which opens a confirmation dialog rather than deleting straight away. The list rebuilds itself from disk after any CRUD operations, so it always reflects the current state of the library.

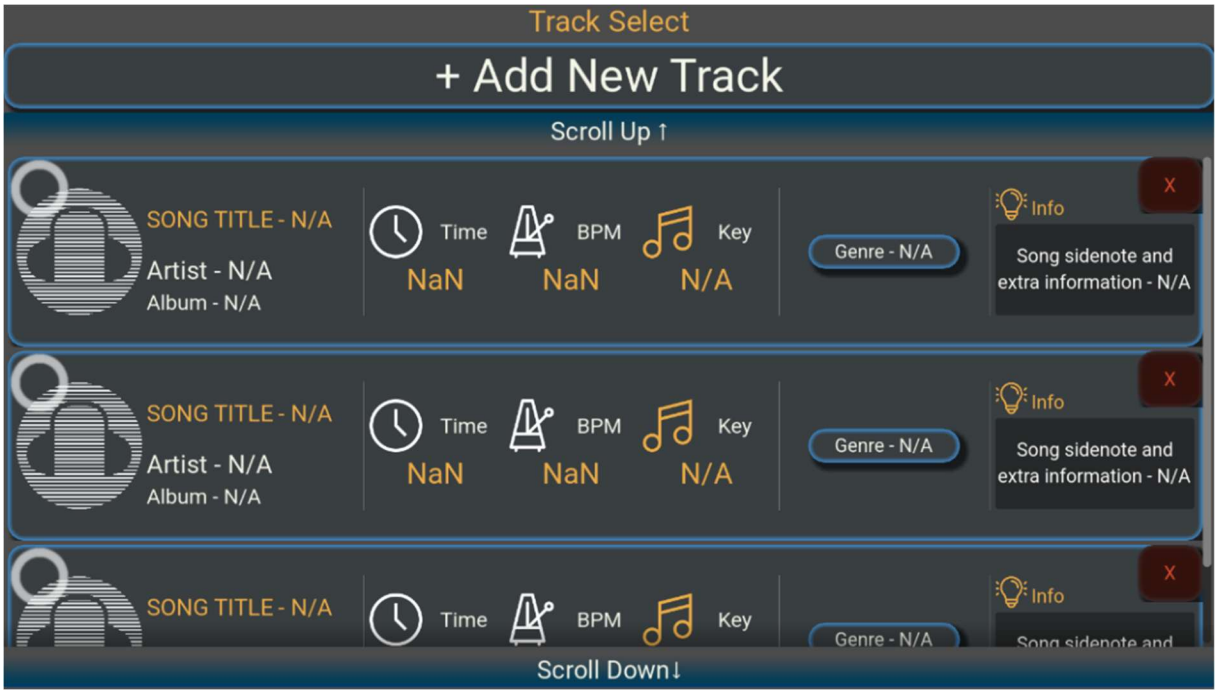


Figure III-18 Screenshot of Librebox Track Selection UI

F) TRACK DELETION CONFIRMATION

The Librebox UI Track deletion confirmation is a simple modal dialog that appears. It shows the name of the track about to be deleted, and if the user presses "Yes" then the track is deleted and the Track Selection list gets updated. Pressing "No" simply hides the dialog and cancels the operation. The dialog itself does not perform any deletion it only acts as a confirmation gate, leaving the actual logic to Librebox UI.

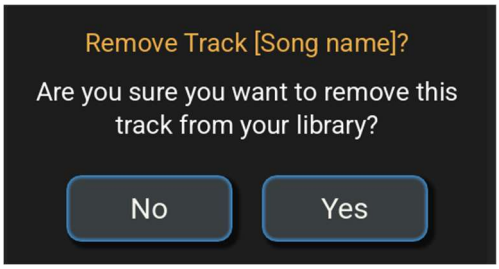
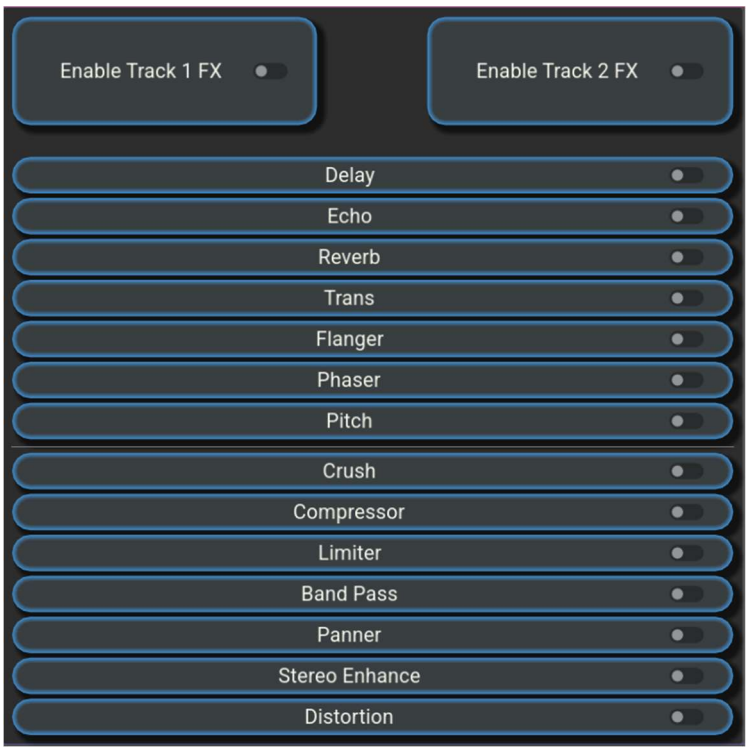


Figure III-19 Screenshot of Track Deletion Confirmation dialog.

G) FX SELECTION

FX Selection displays a grid of 14 toggle buttons, one for each available audio effect, such as Delay, Echo, Reverb, Trans, Flanger, Phaser, Pitch, Crush, Compressor, Limiter, Panner, Stereo Enhance, and Distortion. Two checkboxes at the top control what the FX policy is. The FX policy is what determines which deck is allowed to receive effects, so the user can restrict FX to just one side if they want, or have effects apply to both. When the Poly FX is disabled, enabling a new effect automatically removes whichever one was previously on. All labels are localised and refresh when the language changes.



H) PERFORMANCE PAD MANAGER

This menu lets the user choose what mode the performance pads operate in. The available modes are Hot Cue, FX Set 1, FX Set 2, Beat Jump, Sampler, and Key Shift. Some modes have sub modes, such as while in Hot Cue mode an additional toggle appears letting the user switch between placing cues or deleting them.

There are 2 panels, one for each deck.

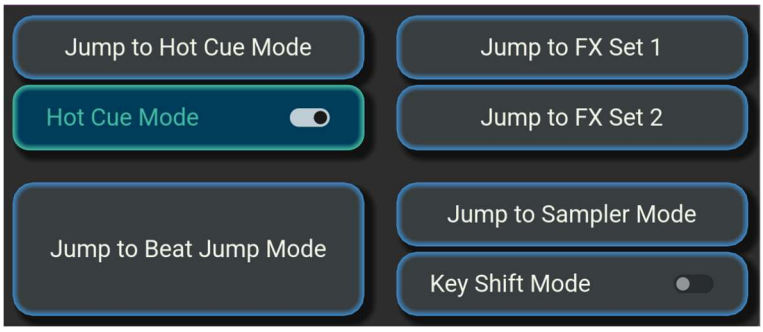


Figure III-20 Screenshot of Performance Pad Management UI

I) RECORDING

The Librebox UI Recording panel lets the user capture their mix session as a WAV file. A single toggle starts and stops the recording, and while active the panel visually pulses to make it obvious that recording is in progress.

On stop, the audio is saved in the background with a timestamped filename. A separate checkbox enables microphone input but on Android this first requests the necessary audio permission. Microphone gain is read from the DJ Controller's Mic Level each frame, so the user can adjust it in real time just as they would on real hardware.

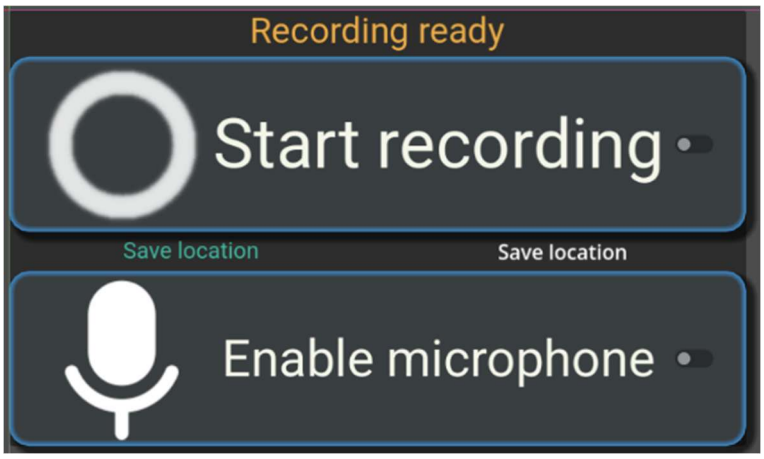


Figure III-21 Screenshot of Track Recording UI

J) ADD SONG

When the user presses "Add New Track" in Track Selection, this import form appears. Tapping "Local Disk" opens a file picker *(see the next section III.B.2.k) "File Dialog")*, which is filtered to MP3, WAV, and OGG audio files. Once a file is chosen, its embedded metadata, such as title, artist, BPM, musical key, album art, etc, is automatically extracted and used to pre-populate the form fields.

A waveform image is also generated asynchronously in the background using either FFMPEG or a fallback solution. Should both methods fail then the waveform will remain blank and the user will need to use the included dev tools to generate the waveform if they want it. The form provides dropdowns for selecting existing artists and albums already in the library, as to prevent data duplication, or text fields for creating new ones, along with a musical key picker and a genre tag system based off of the ID3 standard genre tags. [58]

On save, everything is validated and persisted to disk as Song resource files, and the Track Selection lists refresh to include the new entry. Buttons for Spotify, SoundCloud, and YouTube Music are present in the UI but currently disabled as placeholders for future development.

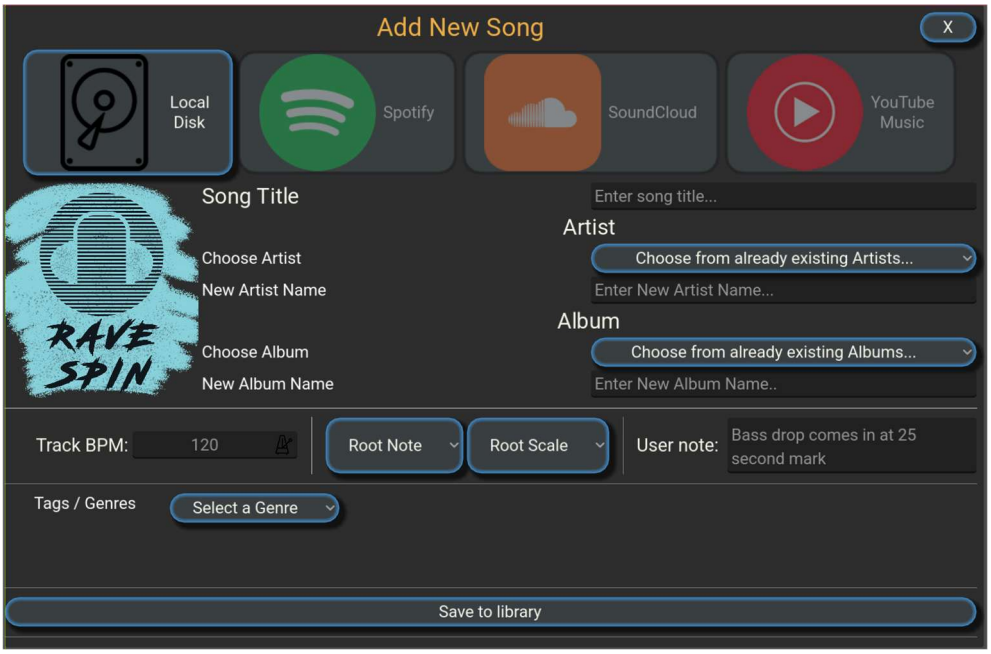


Figure III-22 Screenshot of Menu for importing new songs... Fields are auto populated when selecting a new song.

K) FILE DIALOG

The Meta Quest 3S runs on Android, and as such, has its own functionality and menus for selecting files and is optimised for that device. If for whatever reason this native functionality doesn't work, and the desired menus don't get created, a backup solution is needed.

When adding a song from local disk, Rave-Spin XR will call Godot to ask Android to create a native File Dialog menu *(see Figure III-23)*.

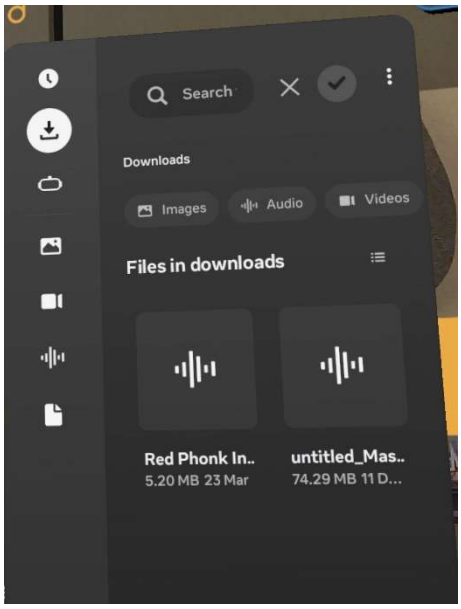


Figure III-23 Screenshot of the Meta File Picker Dialog

Should the native file picker fail to spawn, a backup file picker dialog will be created.

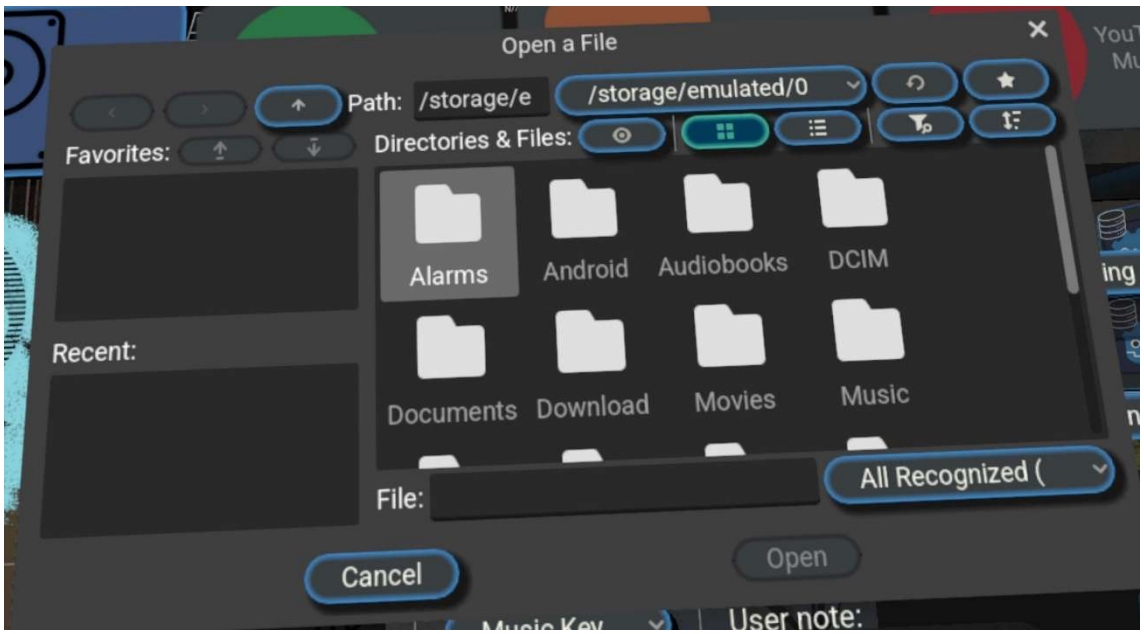


Figure III-24 Screenshot of Backup File Picker Dialog

L) FX STACKING POLICY (ADD MORE FX)

A single checkbox that toggles whether the user can stack multiple audio effects at the same time or is limited to one at a time. When Poly FX is off, activating a new effect on a channel automatically removes whatever was already there.

Some software has this Poly FX behaviour enabled by default but due to the limited hardware power, for Rave-Spin XR Poly FX is disabled by default to give the user a safety net against accidentally layering too many effects and muddying the mix.

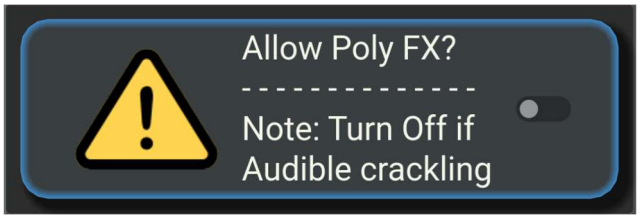


Figure III-25 Screenshot of button to allow Audio-Effect Stacking

M) LANGUAGE SELECT

Four buttons for English, Irish, French, and Simplified Mandarin Chinese.

Each button has a corresponding flag to represent the locale. Pressing one calls the Language-manager which switches the active locale to the newly selected one.

Every Librebox UI component that has any text will be signalled to refresh its text in the new language by checking its key in the translation CSV file and checking to see if a translation exists, if not, then English is used as a fallback solution.

The Language-manager then saves this into the user's config file, so it persists across sessions.

Adding another language in the future wouldn't be an issue as the infrastructure is present and any language can be easily added in the future.

Figure III-26 Screenshot of Language Selection UI

3. DJ CONTROLLER

The DJ controller is what is spawned in alongside Librebox. When both nodes are ready, the Librebox and the DJ controller sync up their tracks, songs, audio effect policies. There is a clear separation of duties as previously mentioned. To ensure that the controller can be switched out later in the future, it is never referred to as Pioneer DDJ-FLX4.

In Rave-Spin XR, the DJ controller is a class, of which DDJ-FLX4 is just an object of that class. If the user wanted to implement the Pioneer DDJ-FLX10, or another DJ Controller, they could do so easily. The design of the Pioneer DDJ-FLX4 in Rave-Spin XR is meant to replicate the original as closely as possible. Admittedly this was done in previous versions of Rave-Spin XR but certain elements were removed as they were replaced by alternative solutions.

Figure III-27 Top-down view of DDJ-FLX4

Figure III-28 Top-down view of modified FLX4 used in RaveSpin

A) BUTTONS & PADS

All controls on the DJ Controller are instances of the same **Base_Control** class. Each control has 2 distinct collision areas, a "**Highlight** area" and a "**Activation** area".

When the player's index finger enters the **Highlight** collision area with a valid hand pose *(to help prevent accidental mis-presses)*, the button highlights itself white to indicate it can be pressed, and if the player has an invalid pose like a fist or index-pinch pose, we assume the player might be scratching the jog wheel and highlight the button red instead, to indicate no activation will happen unless the pose changes.

When the player does a valid pose and enters the **Activation** area, it triggers a small animation of the button going downward to give some tactile visual feedback, and then the corresponding "activated" signal is fired.

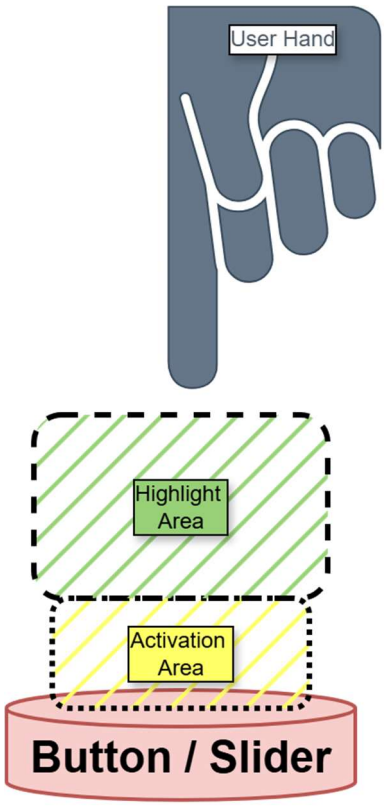


Figure III-29 Diagram of how controls are structured with Highlight area and Activation area.

Figure III-30 Illegal Pose... Disallow activation

Figure III-31 Valid pose and in the 'highlight' zone, but haven't activated control just yet

Figure III-32 Valid pose, and close enough to 'activate' the control

In the following two figures, we can see a behind-the-scenes from Godot at how the controls in Rave-Spin XR are set up. The Highlight collision area *(blue)* and the Activation collision area *(red)* all listening for any collision events from the Players hand model in-game.

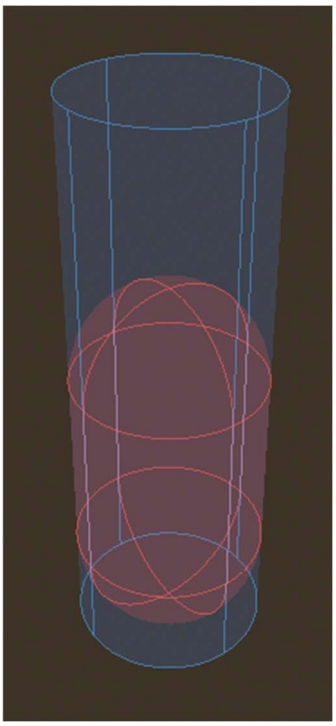


Figure III-33 Behind-the-scenes How a button looks with Debug Collision Colours enabled

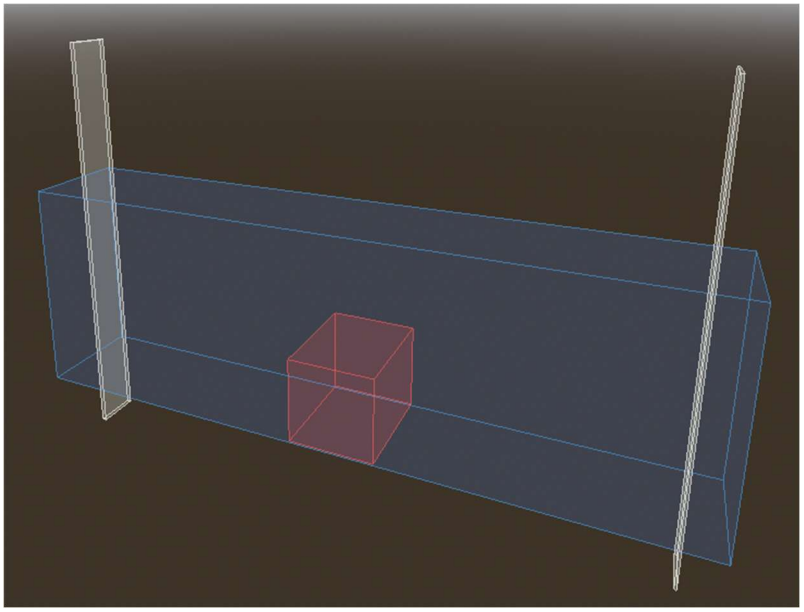


Figure III-34 Behind-the-scenes How a Slider looks with Debug Collision Colours enabled

(1) PERFORMANCE PADS – HOT CUE MODE

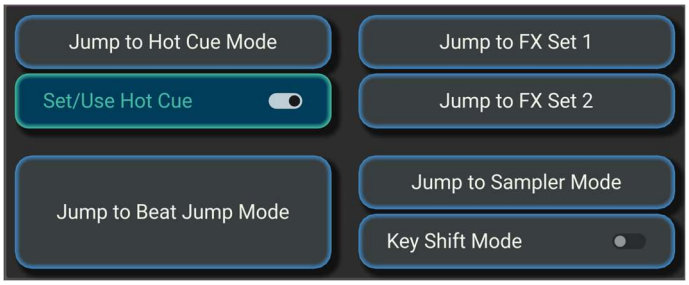
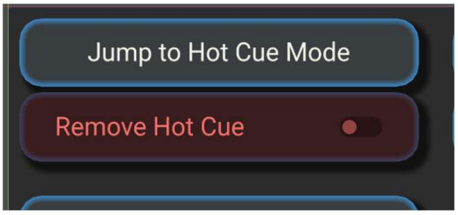


Figure III-35 UI for performance pad selection mode.

Hot Cue is the default mode for the performance pads. In the normal "use or set" mode, pressing a pad that has no cue stored saves the track's current playback position. Each pad has a unique colour so when set, it will map its playback position onto the waveform texture of the song that's playing for the deck in its colour, and then the button model itself will pulse in the same colour. Pressing a pad that already has a cue stored jumps the track to that saved position instantly.

If the user switches to "delete" mode via the Librebox Performance Pad Manager, pressing a pad clears whatever cue was stored on it.



Cues are stored per deck, so the left and right decks each have their own independent set of eight hot cue slots each.

As mentioned, pads with stored cues pulse with a unique colour coded glow when not being touched, so the user can see immediately which cues are set.

Figure III-36 Screenshot of Hot Cues in action. Hot Cues with a saved position are pulsing their own unique colour.

(2) PERFORMANCE PADS – FX

In FX Set 1 and FX Set 2 modes, the pads become triggers for different unique audio effects. Each pad maps to a specific audio effect.

FX Set 1 covers Delay, Echo, Reverb, Trans, Flanger, Phaser, Pitch, and Crush.

FX Set 2 covers Compressor, Limiter, Band Pass, Panner, Stereo Enhance, Distortion, Delay, and Reverb.

Pressing and holding a pad activates its assigned effect on the deck for as long as it is held. Releasing it deactivates the effect immediately. This allows the user to punch effects in and out rhythmically during a mix. At runtime, the pads are labelled to the effect they relate to.

Figure III-37 Screenshot of Performance pads in FX mode (FX Set 1)

(3) PERFORMANCE PADS – BEAT JUMP

In Beat Jump mode, the pads are mapped to fixed jump distances. The user can instantly jump back up to 8 beats or jump forward up to 8 beats.

Pressing a pad instantly seeks the track forward or backward by that number of beats, calculated from the track's BPM.

The seek is clamped to the track's bounds, so it can't jump to before the start or past the end of the track.

At runtime, the pads are labelled to how far back or how far forward they'll jump.

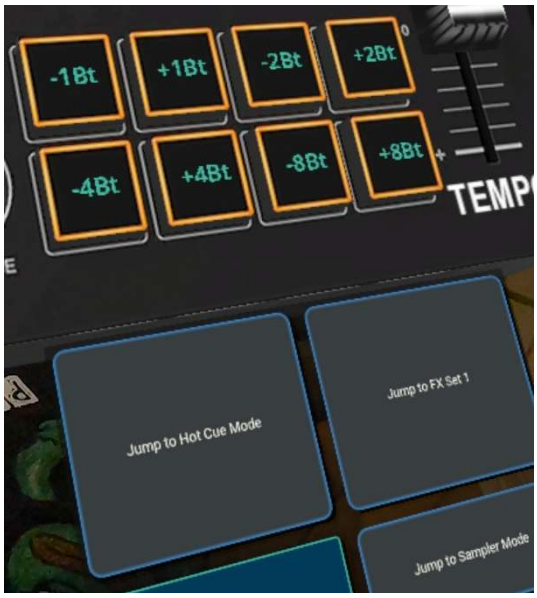


Figure III-38 Screenshot of Performance pads in Beat Jump mode.

(4) PERFORMANCE PADS – SAMPLER

In Sampler mode, each pad triggers a pre-defined audio sample when pressed.

Each pad can be configured with a unique sound file and a display name in the Godot editor. Samples play immediately on press and can overlap, so the user can layer multiple samples at once.

Each sample will have a short nickname which is what the pad will have its labelled changed to.

Figure III-39 Screenshot of Performance pads in Sampler mode.

(5) PERFORMANCE PADS – KEY SHIFT

In Key Shift mode, the pads apply a pitch shift to the deck in fixed semitone increments.

The pads can cause a pitch-shift all the way up to +7 semitones.

Pressing a pad sets the pitch shift for that deck and automatically activates the Pitch effect through Bus Manager.

(6) PLAY / PAUSE

If a track is loaded into the memory of a deck, it'll start playing... If it's pressed again, the song will pause... that's about it.

(7) CUE

Each deck has a "Cue" button that works on a press-and-hold basis. Holding it jumps the track to the stored cue point and plays from there for as long as it is held. Releasing it returns the track to the position it was at before the button was pressed, provided the track was paused beforehand.

If no cue point has been set, pressing it seeks to the very start of the track. Cue points can be set by pausing the track at the desired position and tapping the Cue button, which saves that position as the new cue point.

The stored cue point is also shown as a marker on the Hub's waveform display for that song.

B) SLIDERS

Sliders on the DJ Controller track the position of the player's index finger along their length. Two invisible nodes mark the minimum and maximum ends of each slider, and the finger's position between them is mapped to a normalised 0 to 1 value. A small tolerance zone around the centre allows sliders to snap to 0.5, so you'll never have a slider that's at 0.4999...

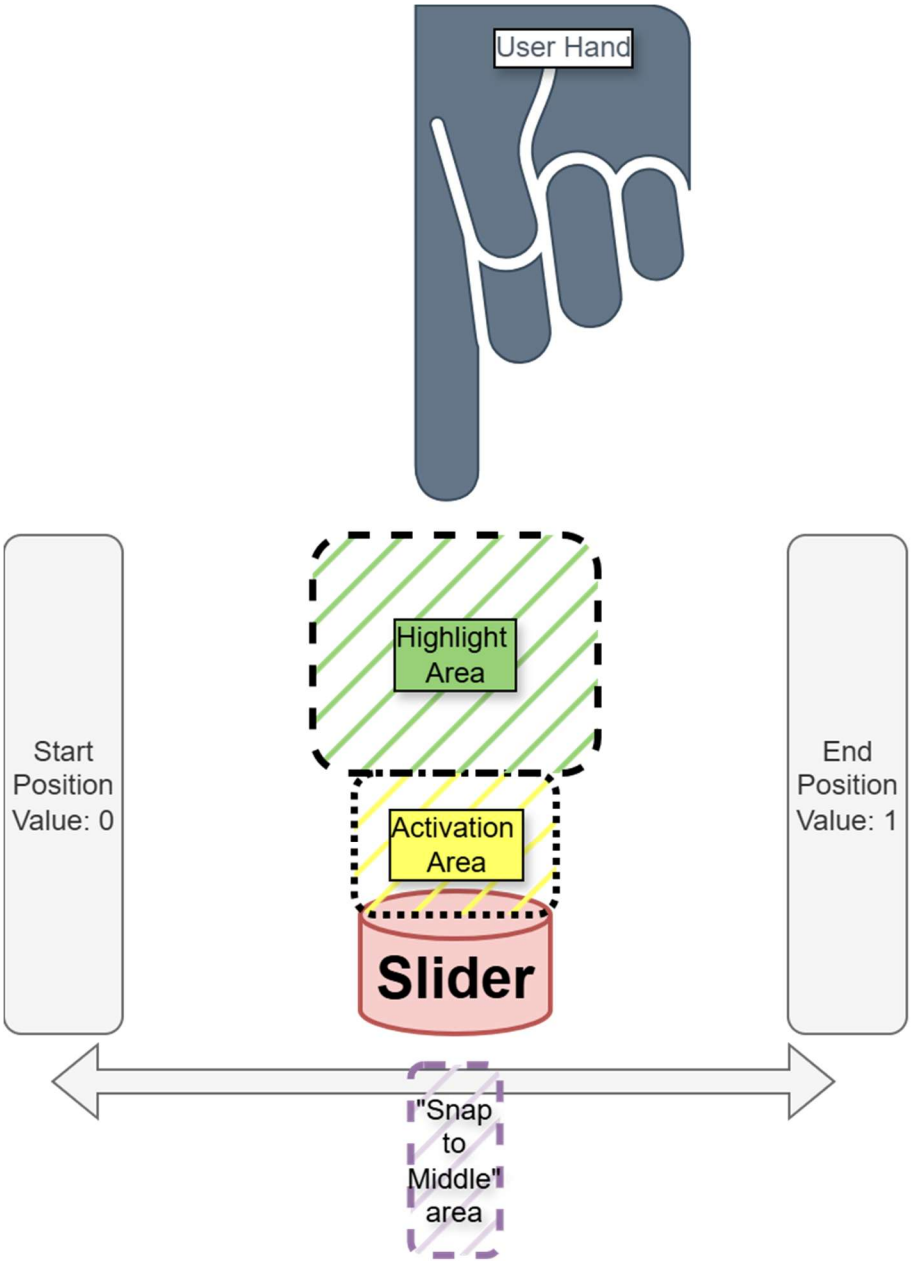


Figure III-40 Diagram of how the Slider logic works to determine value.

(1) CROSSFADER

The crossfader blends audio between the left and right decks.

At 0.0 (fully left), only the left deck is heard.

At 1.0 (fully right), only the right deck is heard.

In the centre, both decks play at equal volume. The blending isn't linear. It uses preloaded curves so that the transition between decks feels nice and smooth instead of sounding abrupt.

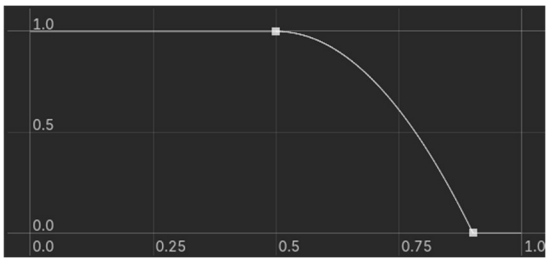


Figure III-41 Crossfade Volume Curve for Left Channel

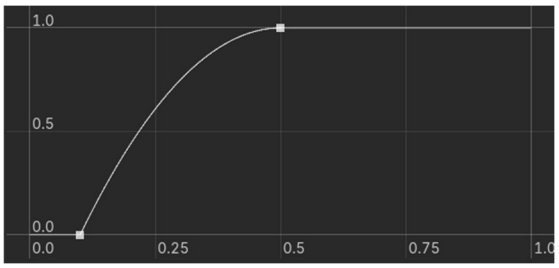


Figure III-42 Crossfade Volume Curve for Right Channel

The individual Channel faders for both left and right channels are combined with the crossfade.

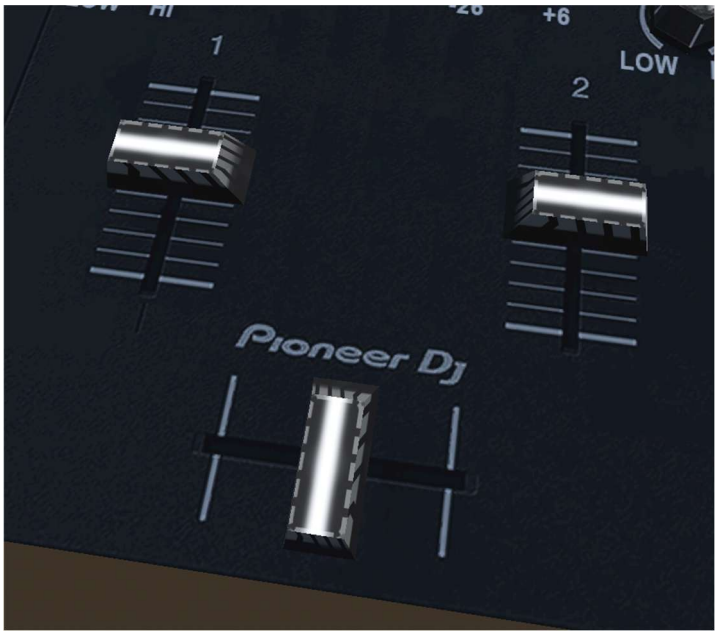


Figure III-43 Screenshot of Left and Right Channel faders (vertical sliders) and Crossfade (middle slider)

C) KNOBS

Due to the issues mentioned in Section IV.B.1.a) “Inaccurate Pose detection at certain angles” *(page 116)* and Section IV.A.3 “Pioneer DDJ-FLX4 Controls” *(page 112)*, Knobs work slightly differently than expected.

Activating a knob causes a vertical slider to spawn which is easier to use than twisting your hand (anti)clockwise.

Figure III-44 Screenshot of the Colour FX slider that spawns to control the Colour FX knob when the knob is pressed.

D) JOG WHEELS

Each deck has a jog wheel that the user can grab and rotate with their hand. The DJ Controller calculates the angle of the player's finger relative to the centre of the wheel each frame and tracks how that angle changes between frames to determine rotation speed and direction.

To interact with the jog wheel, the player's hand must be in either a fist or an index pinch pose. Like a Vinyl record player, when the user touches the wheel as it’s spinning, the track pauses immediately if it was playing, and the system remembers whether it was playing so it can resume on release.

While holding the wheel, rotating it clockwise moves the track forward in time and rotating it anticlockwise moves it backward, at a rate of roughly 2 seconds of audio per full rotation.

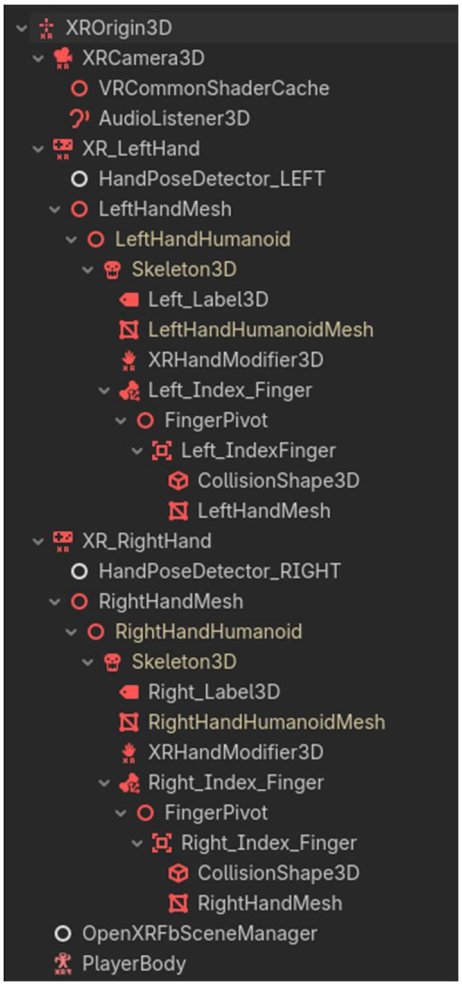
This produces a slight scratching effect as the playback position updates in real time with each frame of rotation, but it is not as clear or as “Swish” as the scratch effect, you’d get on a real physical DJ Controller or CDJ with dedicated jog wheels. When the user releases the wheel, playback resumes from the new position if the track was playing before they grabbed it.

When a track is playing and the jog wheel is not being touched, it spins at a speed proportional to the track's current playback rate, giving a visual indication that the deck is live and playing.

This is mentioned more in Section V.A.3.c) “Functional Testing” *(page 135)*.

4. PLAYER

For any XR application, it's important that the user's head and hand movements are tracked. The Player root in Rave-Spin XR acts as the spatial anchor for all hand and head tracking. The Player comes with a reference to an "XR Camera 3D" which is the user's headset but in game-space. The Player class follows the headset's position and orientation, translating all movements from global world space to local game space.



On startup, the Arena scene tries to place the player camera roughly in front of the Librebox UI and Controller. If the location of the user is ever wrong and the other items in the Arena scene are too far away, the Player class uses the Meta Relocation API to respawn the player in front of the Librebox UI and Controller.

Each hand of the Player node is bound to the location and orientation of the users' hands; with a humanoid mesh whose skeleton deforms to match the orientation of the users' hands and finger movements.

The Player class also evaluates what pose the user is performing with their hand. The recognised poses include Point, Thumbs Up, Fist, Peace Sign, Index Pinch, Spock, Metal, and Extended Middle Finger.

Vulgar poses were programmed in, such as the extension of the Middle Finger. This was meant to be a fun easter egg that would surprise users who performed the vulgar pose, but user testing showed that some users performed pointing and interactions with their middle finger, so it was no longer an easter egg and became a valid pose for interactions.

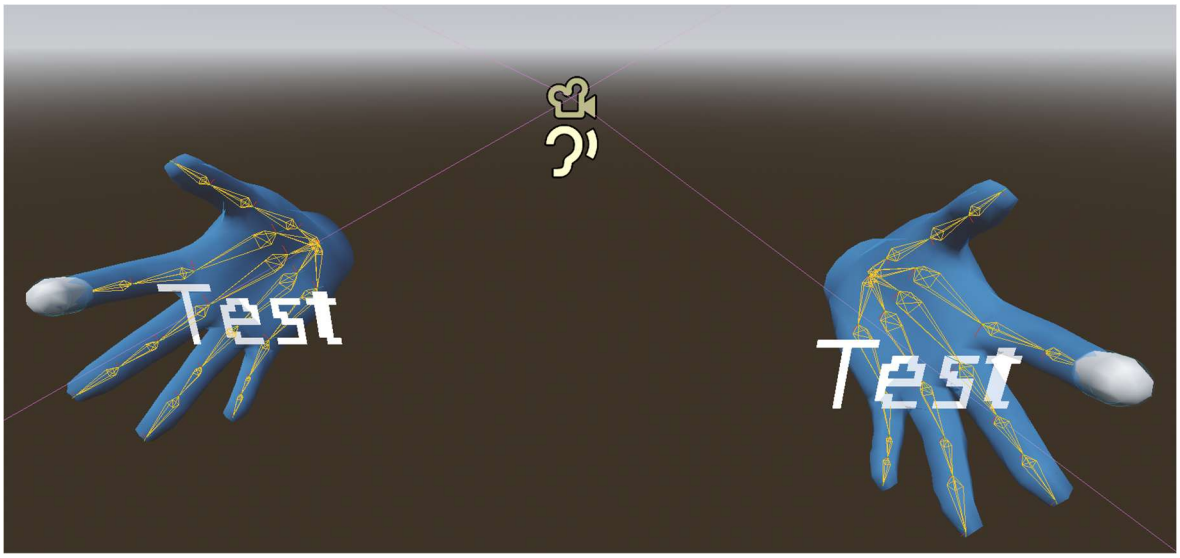


Figure III-45 Screenshot of Player scene.

C. SYSTEM ARCHITECTURE

Rave-Spin XR follows an OOP *(Object Oriented Design)* Architecture, with static and singleton classes such as the DJ Controller, Librebox, Bus Manager, and Player. There can only ever be one instance of each, and this one instance is responsible for their own select set of roles.

An MVC *(Model-View-Controller)* pattern is then used for song data and influences how it is stored, processed, and presented to the user.

1. MVC (MODEL-VIEW-CONTROLLER)

A) MODEL

The Model is built on Godot's Resource system. Song, Artist, and Album are all custom Resource types saved to disk as .TRES files. A Song resource saves a title, audio stream *(or a file path to an audio file if it's a user imported song)*, BPM, musical key, genre tags, waveform texture, and references to its Artist and Album resources. All resources have either a MusicBrainz ID or an alternative ID.

This means that if we are missing any metadata in the song itself, we could in theory get the required information from MusicBrainz, which is a globally curated and open-source collection of song metadata. If the song was from an alternative source, such as Spotify, the alternative ID could point to a Spotify URL where we could then fetch the details from. The infrastructure is there to complete this, but its implementation was outside the scope for this project.

Artists and Albums are also stored as their own resource files, also with MusicBrainz IDs. When loaded, their unique names and IDs are statically saved to globally accessible registers when the app initialises so they can be looked up across the application at any time, thus preventing data duplication. This means the entire music library is file-based.

There is no database, just resource files on the local filesystem that are loaded into memory at runtime. This can make data management easier from the users' perspective as they can simply delete the song resource file they don't want if they would prefer to do so.

Later versions of this project may convert Albums and Artists to be a part of a database instead.

B) CONTROLLER

The Controller is the GD-Script layer that reads and manipulates these resources. Song.gd, Artist.gd, and Album.gd each contain static methods for fetching, querying, and creating their respective resources, retrieving all song paths from disk, looking up an artist by name, or generating a new album resource when importing a song.

When the user adds or deletes a track, it is this layer that handles the CRUD operations on the underlying files.

C) VIEW

The View is the Librebox UI that presents this data to the user. Song resources are converted into Horizontal Track Cards in the Track Selection menu, and their metadata is displayed on the Librebox Hub and Librebox Track selection. The user never interacts with the resource files directly, they only see and interact with the UI representations.

2. ERD OF RESOURCE FILES

Here we can see an ERD *(Entity Relationship Diagram)* of how our Songs, Albums, Artists, and Genres all relate to each other.

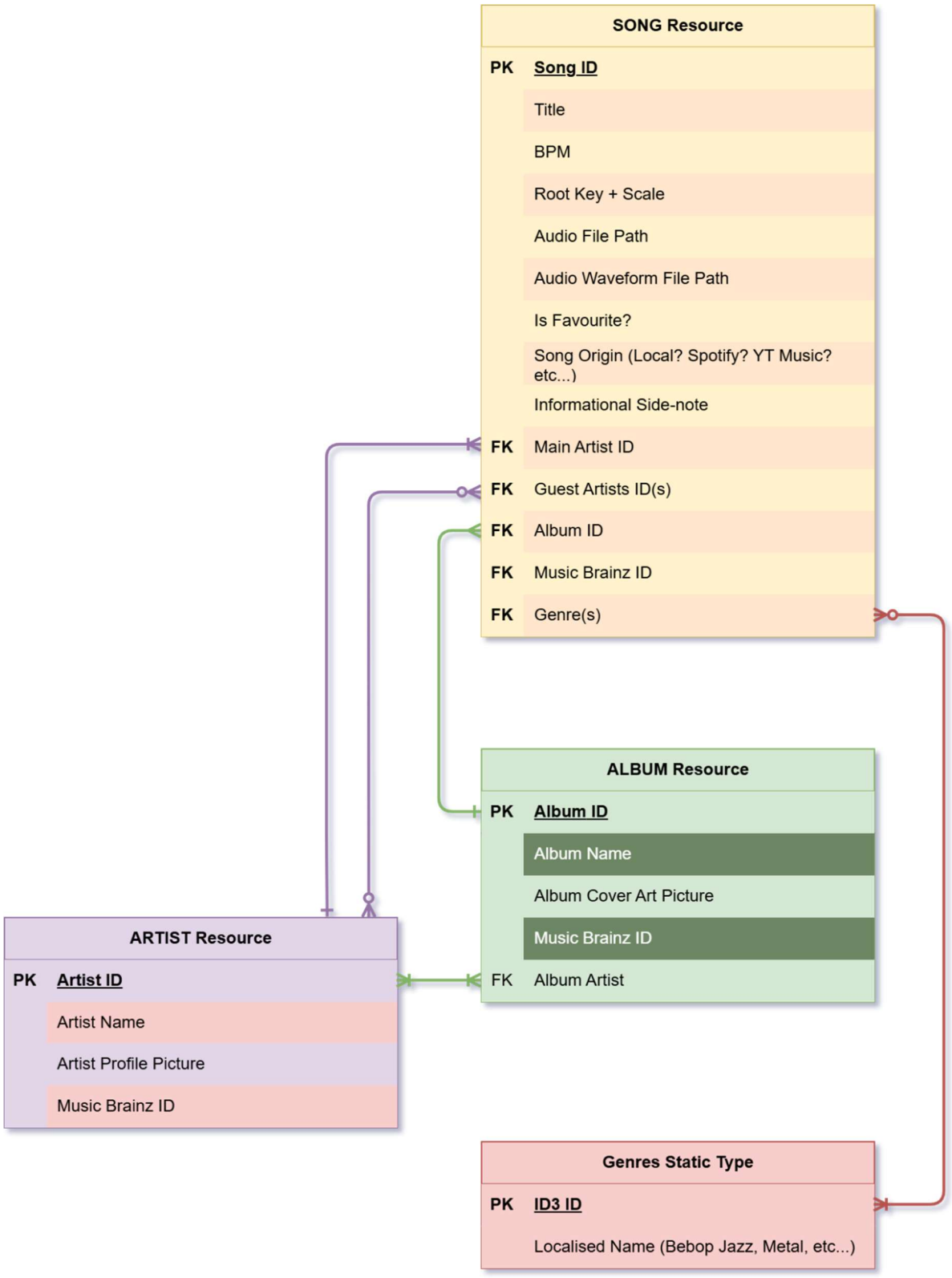


Figure III-46 ERD of Custom Resource files.

3. SINGLETON 'MANAGERS'

The main components which drive most of the logic and systems of Rave-Spin XR are the singleton managers.

A) DJ CONTROLLER

DJ Controller is the central state machine, owning all playback state, audio stream players, loop points, hot cues, crossfader and channel fader positions, performance pad modes, BPM sync state, and jog wheel tracking.

(1) START

On startup, the DJ Controller first sets itself as the singleton instance so the rest of the application can access just the one instance, instead of creating more instances. This is enforced with the function "**DJ_Controller.Get_Instance()**." This static method only returns a reference to one single instance ever.

```
## Returns the active singleton instance.
static func Get_Instance() -> DJ_Controller:
    return Controller_Instance
```

It then initialises the performance pads in distinct steps.

1) Firstly, it sets up default state for both performance pads found in the decks. All pads start in Hot Cue mode, and all hot cue points, active audio effects, are all reset and set to either 0 or null. This is to make sure that all pads start off disabled and we never run into undefined behaviour.
2) Secondly, it goes through the scene tree of the DDJ-FLX4 to find every physical pad node, sort them by their name *(All pads are labelled 'A' through to 'H')*. For each pad we go through, the DJ Controller will start listening to when they're pressed, released, and activated, and assign those events to the correct functions with the correct parameters.

All four audio stream players are paused. Some DJ Controllers have 4 decks as compared to the 2 on the DDJ-FLX4, there will always be four audio stream players but the last two will be ignored as we're using a two-deck solution. To address any bugs with the state of controls being in the wrong state, DJ Controller goes through every control in the scene and their state is reset. Every fader, knob, and jog wheel is set to their default positions.

The jog wheel setup phase locates the left and right jog wheel models in the scene, positions their collision areas to match, and collects starts listening to the Player Finger nodes for hand tracking.

Finally, all knob popup labels are configured to show the correct localised text by using their localisation keys and display formats *(dB for trim and EQ's, normalised 0 to 1 value for the FX-level knob)*.

Originally, I had tried to make multithreading a togglable option so that there would be dedicated high-priority threads to be spawned for managing loops and jog wheel seeking so these operations would not block the main thread. Multithreading is usually a cause of major bugs for most software projects. Either fortunately, or unfortunately, DJ Controller will only allow the Arena scene to finish initialising if multithreading is turned on. Unassigned threads in Godot appear to cause the program to stall.

Turning multithreading off causes the program to stall. More information can be found regarding this in Section IV.A.5 "Performance issues with DJ Controller" *(page 114)*.

(2) EACH FRAME

On every couple of frames, the DJ Controller caches the current playback position and stream length for both decks, throttled to every third frame to reduce overhead. Every physics frame *(ticks slightly less frequently)*, a larger set of updates runs:

- Loop points are checked to see if playback has passed the loop end, and if so, the tracks playback position is set back to the loop start point.
- The tempo is then read from both tempo sliders and recalculates the pitch scale for each deck. If beat sync is turned on for the deck in question, then the tempo slider is ignored and is set to match the other deck's BPM exactly.
- Channel volumes are recalculated, using the crossfader slider position and the decks' channel volume slider, this is expanded upon in Section III.B.3.b)(1) "Crossfader" *(page 68)*.
- Trim, EQ, and Colour FX are then checked. The EQ for the channel is updated based on the Trim and EQ knobs, and the High pass/Low pass filter is based on the Colour FX knobs value.
- Audio effect intensity is then updated for all active effects on both channels based on the FX-Amount knob value.
- The jog wheels then run for both decks. If they get activated by the user they'll try track angular rotation speed on a frame-to-frame basis, seeking the audio to match, and spinning the model.

Properties

Array[AudioStreamPlayer]	AudioPlayerList
Variant	BPM_Adjust_Range
E_BPM_Lock_Status	BeatSyncState
Knob_Control	Beat_FX_Knob
AudioBusLayout	Bus_Layout
Variant	Channel_1_Left_Bus_Index
Variant	Channel_2_Right_Bus_Index
Variant	Channel_3_LeftALT_Bus_Index
Variant	Channel_4_RightALT_Bus_Index
Variant	Channel_Faders
DJ_Controller	Controller_Instance
Knob_Control	Core_FX_Knobs_EQ_HIGH_L
Knob_Control	Core_FX_Knobs_EQ_HIGH_R
Knob_Control	Core_FX_Knobs_EQ_LOW_L
Knob_Control	Core_FX_Knobs_EQ_LOW_R
Knob_Control	Core_FX_Knobs_EQ_MID_L
Knob_Control	Core_FX_Knobs_EQ_MID_R
Knob_Control	Core_FX_Knobs_SOUND_COLOR_FX_L
Knob_Control	Core_FX_Knobs_SOUND_COLOR_FX_R
Knob_Control	Core_FX_Knobs_TRIM_L
Knob_Control	Core_FX_Knobs_TRIM_R
Variant	Crossfade_Alpha
Curve	Crossfader_Curve_Left
Curve	Crossfader_Curve_Right
Array[bool]	Hot_Cue_Add_Mode_By_Track
Array	Hot_Cue_Sec_By_Track
float	Jogwheel_Audio_Warp_Seconds_Per_Full_Rotation
float	Jogwheel_Auto_Spin_Rotations_Per_Second_At_Normal_Speed
bool	Jogwheel_Enable_Audible_Scratching
float	Jogwheel_Grab_Radius_Meters
Array[int]	Key_Shift_Semitones_By_Track
Area3D	Left_Jogwheel_Activation
CollisionShape3D	Left_Jogwheel_Collision_Shape
Player_Finger	Left_Jogwheel_Finger
MeshInstance3D	Left_Jogwheel_Mesh
Node3D	Left_Jogwheel_Node

Figure III-47 The Properties of DJ Controller (part 1)

Array[bool]	Loop_Enabled
Array[bool]	Loop_End_Armed
Array[float]	Loop_End_Sec
Array[bool]	Loop_Point_Snapping_Enabled
Array[bool]	Loop_Start_Armed
Array[float]	Loop_Start_Sec
Array[int]	Performance_Pad_Mode_By_Track
Array[float]	Regular_Cue_Sec_By_Track
bool	Restrict_Jogwheel_To_Matching_Hand
Area3D	Right_Jogwheel_Activation
CollisionShape3D	Right_Jogwheel_Collision_Shape
Player_Finger	Right_Jogwheel_Finger
MeshInstance3D	Right_Jogwheel_Mesh
Node3D	Right_Jogwheel_Node
Array[int]	Selected_FX_Set_By_Track
float	Track_1_playback_length
float	Track_1_playback_pos
float	Track_2_playback_length
float	Track_2_playback_pos
Variant	Use_2_Track_Bus_Layout
bool	Use_Multi_Threaded_Jog_Warp
bool	Use_Snap_Style_Jogwheel_Grab
Variant	debug_visable
float	track_1_new_bpm
float	track_2_new_bpm

Figure III-48 The Properties of DJ Controller (part 2)

Methods

```
                void  Clear_All_Hot_Cues(which_track: int) static
                void  Clear_Loop(which_track: int) static

                bool  Get_Hot_Cue_Add_Mode(which_track: int) static
               float  Get_Hot_Cue_Sec(which_track: int, cue_index: int) static
      DJ_Controller  Get_Instance() static
      DJ_Controller  Get_Instance_await() static
               float  Get_Loop_End_Sec(which_track: int) static
               float  Get_Loop_Start_Sec(which_track: int) static
               Color  Get_Performance_Pad_Color(pad_index: int) static
                 int  Get_Performance_Pad_Mode(which_track: int) static
               float  Get_Regular_Cue_Sec(which_track: int) static
                 int  Get_Selected_FX_Set(which_track: int) static
               float  Get_Track_Current_BPM(which_track: int) static
               float  Get_Track_Playback_Alpha(which_track: int) static
  AudioStreamPlayer  Get_Track_Playback_Player(which_track: int) static
               float  Get_Track_Playback_Position(which_track: int) static
               float  Get_Track_Speed_Mult(which_track: int) static

                bool  Has_Hot_Cue(which_track: int, cue_index: int) static
                bool  Has_Regular_Cue(which_track: int) static

                bool  Is_Loop_Enabled(which_track: int) static
                bool  Is_Loop_End_Armed(which_track: int) static
                bool  Is_Loop_Point_Snapping_Enabled(which_track: int) static
                bool  Is_Loop_Start_Armed(which_track: int) static

                void  LoadTrackIntoMemory(which_track: int, which_song: Song)

                void  Loop_Extend(which_track: int)
                void  Loop_Make_4_Beat_Toggle(which_track: int)
                void  Loop_Set_End_Point(which_track: int)
                void  Loop_Set_Start_Point(which_track: int)
                void  Loop_Shorten(which_track: int)

                void  Play_Pause(p_which_track: int)

                void  Set_Hot_Cue_Add_Mode(which_track: int, add_mode_enabled: bool) static
                void  Set_Loop_Point_Snapping_Enabled(which_track: int, enabled: bool) static
                void  Set_Performance_Pad_Mode(which_track: int, mode: int) static

                void  Update_Channel_DBs()
                void  Update_Channel_Tempo_Adjusts()
                void  Update_Channel_Trim_EQ_CFX()

                void  _seek_track_phase_to_match(track_to_move: int, reference_track: int)
```

Figure III-49 The Methods used by DJ Controller

B) BUS MANAGER

Bus Manager is a purely static utility class, meaning it doesn't exist as a node in the Arena scene tree and is only called through globally accessible method calls. It's responsible for the audio bus hierarchy, such as the various available audio effects, custom channel EQs, High Pass filters, and Low Pass filters, trim EQ levels, and the FX stacking policy. The audio for each channel follows a fixed chain.

The input feeds into a Trim stage that adjusts the volume or the "gain" of the incoming audio.

It goes through to an EQ filter. The different bands of the audio spectrum have their gains and volumes set by the EQ-Low, EQ-Mid, and EQ-High knobs. The EQ knobs will affect different parts of the EQ differently, like EQ-Low should affect the lower frequencies more than EQ-High for example. *(see Figure III-51)*

After that comes the Colour FX stage, which is a pair of low pass and high pass filters which are both set by the one Colour FX knob.

- If the Colour FX Knob is < 0.5, the high pass filter is deactivated and the low pass filter is activated instead and the further down the knob goes the lower the low pass filter threshold is set.
- If the Colour FX Knob is > 0.5, the low pass filter is deactivated and the high pass filter is activated instead and the further up the knob goes the higher the high pass filter threshold is set.

The final stage in the chain is the Audio FX slots, which will hold whichever audio FX the user has activated. Each effect type has its own parameter scaling so the shared FX intensity knob controls it in a way that makes sense for that effect, such as reverb which scales its wet/dry mix, the compressor scales its ratio, and the limiter scales its threshold. If stacking of FX is allowed, then the effects are just added to the chain, if not, then the currently active effect just gets swapped out and replaced.

```
const EQ6_LOW_WEIGHTS : Dictionary[E_BAND_HZ, float] = {
    E_BAND_HZ.HZ_32 : 1.0,
    E_BAND_HZ.HZ_100 : 0.8,
    E_BAND_HZ.HZ_320 : 0.3,
    E_BAND_HZ.HZ_1000 : 0.0,
    E_BAND_HZ.HZ_3200 : 0.0,
    E_BAND_HZ.HZ_10000 : 0.0
    }

const EQ6_MID_WEIGHTS : Dictionary[E_BAND_HZ, float] = {
    E_BAND_HZ.HZ_32 : 0.0,
    E_BAND_HZ.HZ_100 : 0.2,
    E_BAND_HZ.HZ_320 : 0.7,
    E_BAND_HZ.HZ_1000 : 1.0,
    E_BAND_HZ.HZ_3200 : 0.4,
    E_BAND_HZ.HZ_10000 : 0.0
}

const EQ6_HIGH_WEIGHTS : Dictionary[E_BAND_HZ, float] = {
    E_BAND_HZ.HZ_32 : 0.0,
    E_BAND_HZ.HZ_100 : 0.0,
    E_BAND_HZ.HZ_320 : 0.0,
    E_BAND_HZ.HZ_1000 : 0.0,
    E_BAND_HZ.HZ_3200 : 0.6,
    E_BAND_HZ.HZ_10000 : 1.0
}
```

Figure III-51 Weight distribution of knobs and how they influence the EQ.

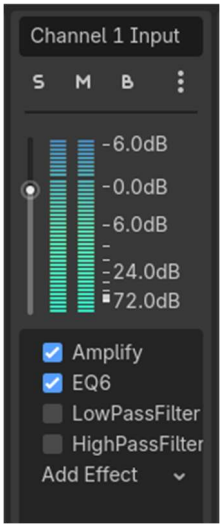


Figure III-50 Screenshot of the Channel #1 audio bus. The order of the effects matters and effects can be added at runtime.

Properties

```
E_Track_FX_Policy  CURRENT_TRACK_FX_POLICY   [default: 0]
             bool  ONE_FX_AT_A_TIME          [default: true]
   E_BEAT_FX_TYPE  active_effects_Channel_1  [default: {}]
   E_BEAT_FX_TYPE  active_effects_Channel_2  [default: {}]
```

Methods

```
      bool  Allow_Multiple_FX_at_same_time() static
      void  Apply_Beat_FX_Level(channel: int, alpha: float) static
  E_BAND_HZ  Calculate_Weights_DB(low_alpha: float = 0.5, mid_alpha: float = 0.5, high_alpha: float = 0.5) static
  E_BAND_HZ  Calculate_Weights_Normal(low_alpha: float = 0.5, mid_alpha: float = 0.5, high_alpha: float = 0.5)
             static
      bool  Can_Track_1_Take_FX() static
      bool  Can_Track_2_Take_FX() static
      void  Clear_Pitch_Shift_Semitone_Override(channel: int) static
       int  Get_Channel_Index_e(Which_Bus: E_AUDIO_BUSSES) static
       int  Get_Channel_Index_i(Which_Channel: int) static
      void  Set_Allow_Multiple_FX_at_same_time(enabled: bool) static
      void  Set_Pitch_Shift_Semitone_Override(channel: int, semitones: int) static
      void  Set_Track_1_can_take_FX(enabled: bool) static
      void  Set_Track_2_can_take_FX(enabled: bool) static
      bool  Toggle_Allow_Multiple_FX_at_same_time() static
       int  _get_next_free_slot(channel: int) static
      void  _set_beat_fx_effect_level(effect: AudioEffect, fx_type: E_BEAT_FX_TYPE, alpha: float,
            channel: int = 0) static
      void  add_beat_fx(fx_type: E_BEAT_FX_TYPE, channel: int) static
    String  beat_fx_translation_key(fx: E_BEAT_FX_TYPE) static
   Variant  get_beat_fx_class(fx_type: E_BEAT_FX_TYPE) static
      bool  is_beat_fx_active(fx_type: E_BEAT_FX_TYPE, channel: int) static
      void  remove_beat_fx(fx_type: E_BEAT_FX_TYPE, channel: int) static
      void  toggle_beat_fx(fx_type: E_BEAT_FX_TYPE, channel: int) static
```

Enumerations

```
enum E_Track_FX_Policy:
```

- FX_Disabled = 0
- Only_Track_1 = 1
- Only_Track_2 = 2
- Both_Tracks = 3

Figure III-52 The Properties and methods of Bus Manager

C) LIBREBOX MANAGER

Librebox Manager acts as the bridge between the UI and the DJ Controller. When a user selects a track, Librebox receives the *"track selected"* signal, loads the Song resource, hands it to the DJ Controller for playback, and tells the Librebox Hub to refresh.

The Librebox will wait for the DJ Controller to refresh and then will update all text components to the correct language once all aspects of the Arena scene are fully booted up.

During a session, Librebox Manager handles all logic for the UI and storing and fetching of music and its data. Whenever the user selects a track, the Librebox Manager receives the selected Song resource from the Librebox Track selector that the user just pressed, stores it locally, passes it to the DJ Controller to load into memory for playback, and refreshes the Hub display to reflect the change.

It also exposes playback queries such as current position, progress alpha *(0: the song just started, 1: the song just ended)*, audio stream player references, and so much more. This is so all UI components, like BPM Spinners, Waveform textures of the song that's being played, etc, can all get the info they need without needing to talk to the DJ Controller directly.

If a song is deleted from the library, Librebox checks whether it is currently loaded on either deck or clears it if it is. It can also handle BPM syncing between decks by calculating the pitch ratio needed to match one deck's tempo to the other, or "Beat-Sync" as it's called.

Methods

```
           LibreBox  Get_Instance_await() static
              float  Get_Track_BPM(which_track: int) static
              float  Get_Track_Playback_Alpha(which_track: int) static
AudioStreamPlayer    Get_Track_Playback_Player(which_track: int) static
              float  Get_Track_Playback_Position(which_track: int) static
               void  On_Song_Change_Track_1(New_Song: Song)
               void  On_Song_Change_Track_2(New_Song: Song)
               bool  Pause_Track(p_which_track: int) static
               void  Play_Pause(p_which_track: int)
               bool  Play_Track(p_which_track: int, reset_from_start: bool = false) static
               bool  Refresh()
               bool  Sync_Track_BPMs(Track_we_want_to_Match: int, Track_we_want_change: int, match_beat: bool)
               void  Update_Controller_and_Hub(Which_Track: int, New_Song: Song)
               void  _ready()
               void  apply_song_deleted_from_library(deleted_metadata_path: String) static
               void  refresh_both_track_selection_lists() static
```

Figure III-53 The methods and functions of Librebox.

Properties

```
                LibreBox_HUB  HUB_Menu_ref
                     LibreBox  LibreBox_instance
LibreBox_TrackSelection  Track_1_Selection_ref
                        Song  Track_1_Song
LibreBox_TrackSelection  Track_2_Selection_ref
                        Song  Track_2_Song
                        Song  Track_3_Song
                        Song  Track_4_Song
```

Figure III-54 The properties of Librebox.

D) LANGUAGE MANAGER

Language Manager handles all localisation changes, loading the user's preferred locale from a config file on startup and allowing runtime language switching that causes every UI component to re-render its labels by notifying and emitting a “Locale Changed” signal to the underlying Godot Translation Sever, which Librebox and DJ Controller listen to also.

Because all components can listen to this signal, it allows all elements to be refreshed to the correct localised version.

Properties

```
String  current_locale  [default: "en"]
```

Methods

```
String  _load_saved_locale()
String  _locale_from_os()
  void  _ready()
  void  _save_locale(locale: String)
  void  apply_locale(locale: String)
String  get_locale_display_name(locale: String)
```

Constants

- `CONFIG_PATH = "user://language.cfg"`
  Runtime language manager. Loads locale preference, applies it through `TranslationServer`, and persists selection.
- `CONFIG_SECTION = "i18n"`
- `CONFIG_KEY_LOCALE = "locale"`
- `SUPPORTED_LOCALES = Array[String](["en", "fr", "zh_CN", "ga"])`
  Locales registered in Project Settings -> Localization (en = default / source strings).
- `LOCALE_DISPLAY_NAMES = {"en": "English", "fr": "Français", "ga": "Gaeilge", "zh_CN": "简体中文"}`

Figure III-55 The properties and methods of Language Manager

E) TRANSLATION CSV FILE

The way Godot will typically do translations, is that it will look for “Keys” in any UI element that may contain text. The same can be done for GD-Script by wrapping everything in **tr(...)**. This will cause whatever text is inside to be checked to see if it contains text that can be localised.

All translations are held in **LANG_Translation.csv**. The header is in the format.

Key	En (English)	Fr (French)	Zh_CN (Mandarin)	Ga (Irish)
KEY_RAVESPIN	“RaveSpin”	“DiscoSpin”	“炫盘”	“Casadh Rave”
KEY_LEFT	“LEFT”	“GAUCHE”	“左”	“CLÉ”

I created the Keys with the English version and asked generative ai to generate the missing translations. Later, any incorrect translations could be fixed by editing the .CSV file.

4. CONCLUSION

The overall data flow is straightforward. XR hand input is detected by the Player class, which activates the Players finger collision shapes, which collide with interactable control nodes on the DJ Controller or Librebox UI, which emit signals to their respective managers, which update audio state and refresh the interface.

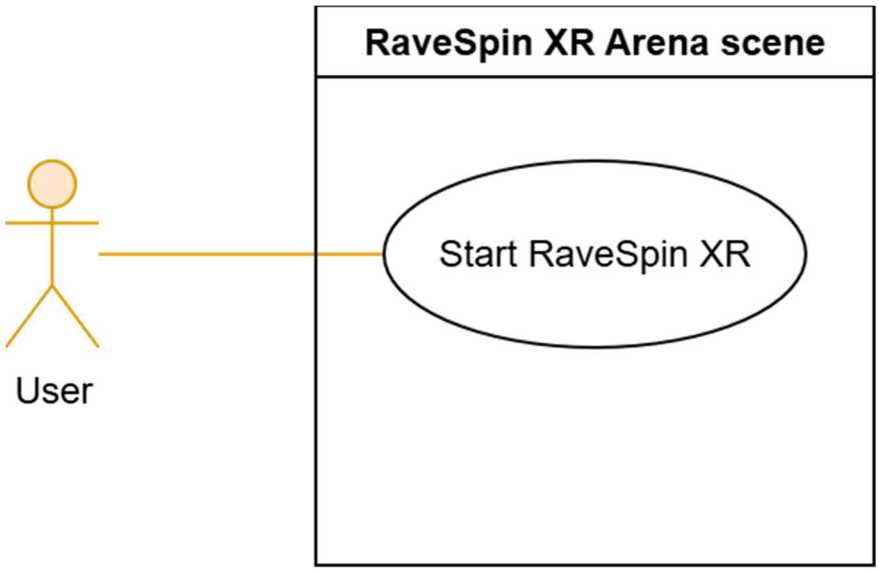
Everything runs on the main thread except loop management, recording saves, and rendering, which use separate threads to avoid blocking. Extensive efforts were made to make the program as multithreaded as possible, but Godot does struggle with this as any changes to items that are in the Arena scene cause significant stutters.

Due to rendering being detached from the main thread, this gives Rave-Spin XR a smoother feel as any hitches or stutters don't stop head tracking, leading to less motion sickness. Any crashes will however stop all threads, rendering thread included.

D. REQUIREMENTS ANALYSIS

To figure out what the functional and non-functional requirements of the system would be, the following use cases were defined. Each use case specified what the system needed to do and provided a verifiable end-goal that could be verified during user evaluation. The use cases requirements and their accompanying non-functional requirements helped to form the project's requirements traceability framework. Every requirement can be traced back to the use case that motivated it, and to the test that verified it.

1. USE CASE 1 - STARTING RAVESPIN SESSION



Goal	User starts the application and can start a session environment
Preconditions	App is installed and just opened
Postconditions (Success)	Arena scene is loaded and user can interact with controls/UI
Postconditions (Failed)	Session does not start. User remains outside interactive scene
Actors	User, Arena Scene
Trigger	User opens Rave-Spin XR from headset/app launcher
Description	The app initialises the OpenXR instance and loads the Arena scene which is where our DJ Controller, Interface, Player, and core systems for interaction will be spawned
Priority	High

Main Flow (MF)		
Step	Action	**Alternative**
1.1	User launches Rave-Spin XR	
1.2	System creates and initialises OpenXR, setting what headset camera to use, and other required variables.	**EF 1.2**
1.3	System enables XR passthrough so user can see outside environment too.	**AF 1.3**
1.4	System loads the Arena scene	
1.5	System initialises core subsystems such as DJ Controller, Librebox UI, Audio Bus layout	
1.6	System applies initial camera recentre	
1.7	User can interact with User interfaces and DJ Controllers	**AF 1.7a, 1.7b**

Alternative Flow (AF) 1.3: Passthrough not allowed		
Step	Action	**Alternative**
AF 1.3.1	System detects passthrough is unavailable on the device.	
AF 1.3.2	System falls back to showing the outside world as just the default Godot skybox environment.	**MF 1.4**

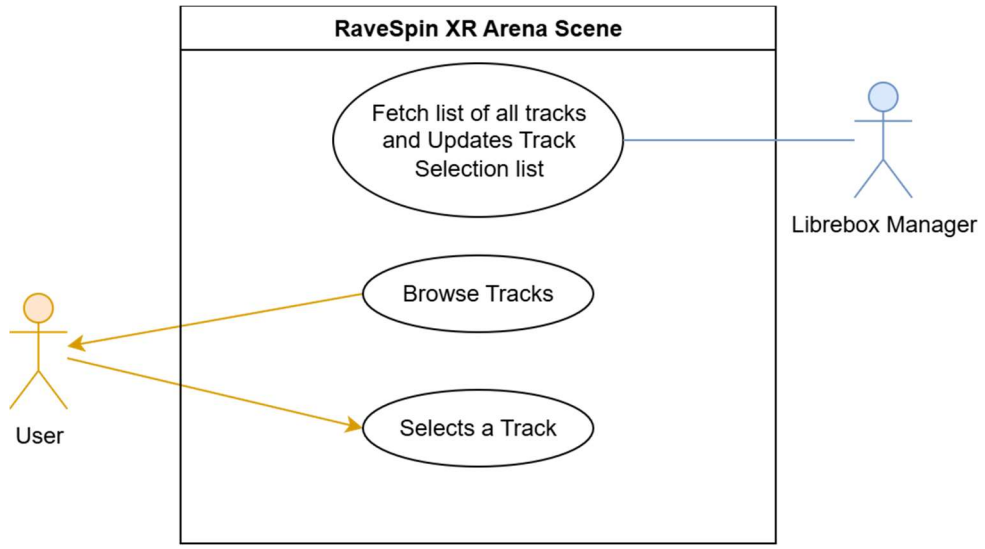
Alternative Flow (AF) 1.7a: Player is too close or too far away		
Step	Action	**Alternative**
AF 1.7a.1	User manually triggers another recentre again	**MF 1.7**

Alternative Flow (AF) 1.7b: Player is unsure how to use the system		
Step	Action	**Alternative**
AF 1.7b.1	In-app tooltips and “Getting Started” video hosted publicly online are all present to try assist and guide the user.	**MF 1.7**

Exception Flow (EF) 1.2: OpenXR not supported on headset		
Step	Action	**Alternative**
EF 1.2.1	XR interface fails to initialise, Rave-Spin XR cannot start	

Non-Functional Requirements	
Responsiveness	Startup time needs to be under 10~15 seconds on the Quest 3S.
Stability	The initialisation sequence must not leave the system in a partially loaded state. All race conditions must be addressed and be consistent.

2. USE CASE 2 - BROWSE AND SELECT TRACK FROM LIBRARY



Goal	Allow the user to browse and select a song from the track library
Preconditions	Session is running; Track Selection panel is accessible
Postconditions (Success)	A song is selected and loaded into the DJ Controller for manipulation
Postconditions (Failed)	No track is loaded. The DJ Controller will still have its previous song loaded.
Actors	User, Librebox Track Selection Menu, Song Resource Layer
Trigger	User selects a song for a deck from the Track Selection Menu
Description	The system populates a scrollable list of "track cards" from available songs metadata. The user browses and selects a song from this list by clicking a card from the list *(for this use case, let's assume it is for the left deck. The same shall apply for right deck).*
Priority	High

Main Flow (MF)		
Step	Action	**Alternative**
2.1	System enumerates all available song metadata paths *(both built-in demo songs and user-imported)*	**EF 2.1**
2.2	System instantiates a track card for each valid song entry so the user has something they can click	**EF.2.2**
2.3	User scrolls through the track card list	
2.4	User taps a track card to select a song	**AF 2.4**
2.5	System updates the DJ Controller to load the song into the appropriate deck. Librebox UI is also updated to show new Song details	

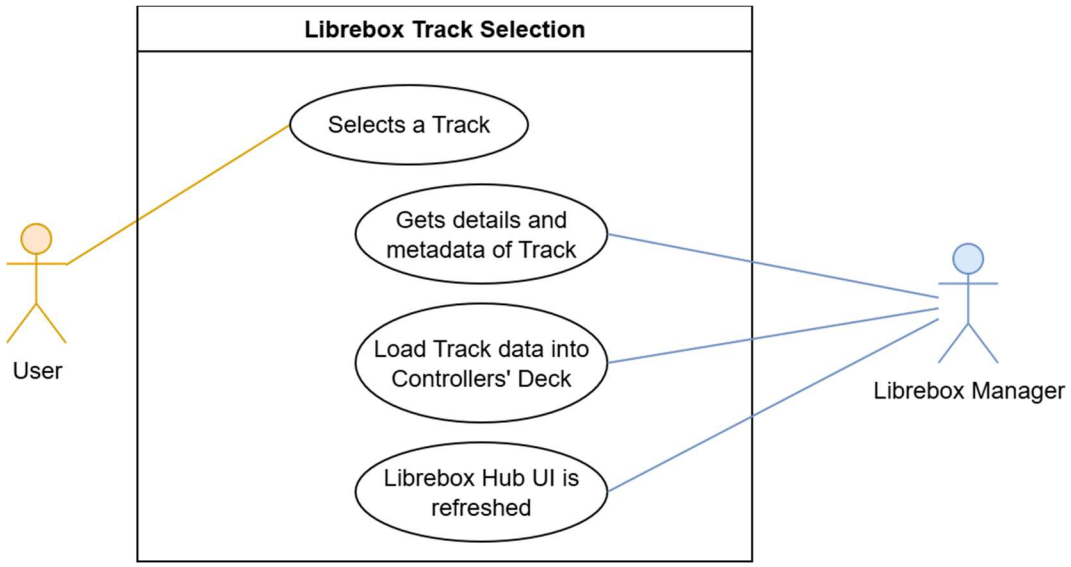
Exception Flow (EF) 2.1: No songs are available		
Step	Action	**Alternative**
EF 2.1.1	There are no songs at all detected. Either the user deleted them all or storage access has been denied	
EF 2.1.2	The "Add track" button is now bigger. The user can import a new Song and populate the list.	**MF 2.1**

Exception Flow (EF) 2.2: Song metadata entry is corrupt or unreadable		
Step	Action	**Alternative**
EF 2.2.1	Rave-Spin XR will skip this track if there is no valid path to any audio file.	
EF 2.2.2	If there is a valid path, then all missing data entries will be set to default values or "NAN" or "N/A"	**MF 2.2**
EF 2.2.3	If there is no valid audio path, then this will be skipped	**MF 2.2**

Alternative Flow (AF) 2.4: User pressed the delete button		
Step	Action	**Alternative**
AF 2.4.1	Ask user if they're sure that they want to delete the track	
AF 2.4.2	User pressed "No." Cancel operation	**MF 2.4**
AF 2.4.3	User pressed "Yes." The Song is deleted	**MF 2.2**

Non-Functional Requirements	
Usability	Track cards must display legible metadata *(title, artist, BPM)* at a comfortable distance
Data Validity	Only valid songs with loadable audio should be shown as selectable

3. USE CASE 3 - LOAD SELECTED TRACK ONTO DECK



Goal	Assign a selected song to a specific deck and prepare it for playback
Preconditions	A valid song has been selected from the Librebox Track Selection
Postconditions (Success)	The deck has a loaded audio stream, and the Librebox HUB displays the song's metadata and waveform
Postconditions (Failed)	All items remain empty or retains their previous state
Actors	Librebox UI, DJ Controller
Trigger	User just selected a valid song for a deck of the DJ Controller
Description	Librebox UI forwards the selected song to the DJ Controller, which loads the audio stream and resets local state. The Librebox HUB then refreshes to display updated metadata and waveform.
Priority	Medium

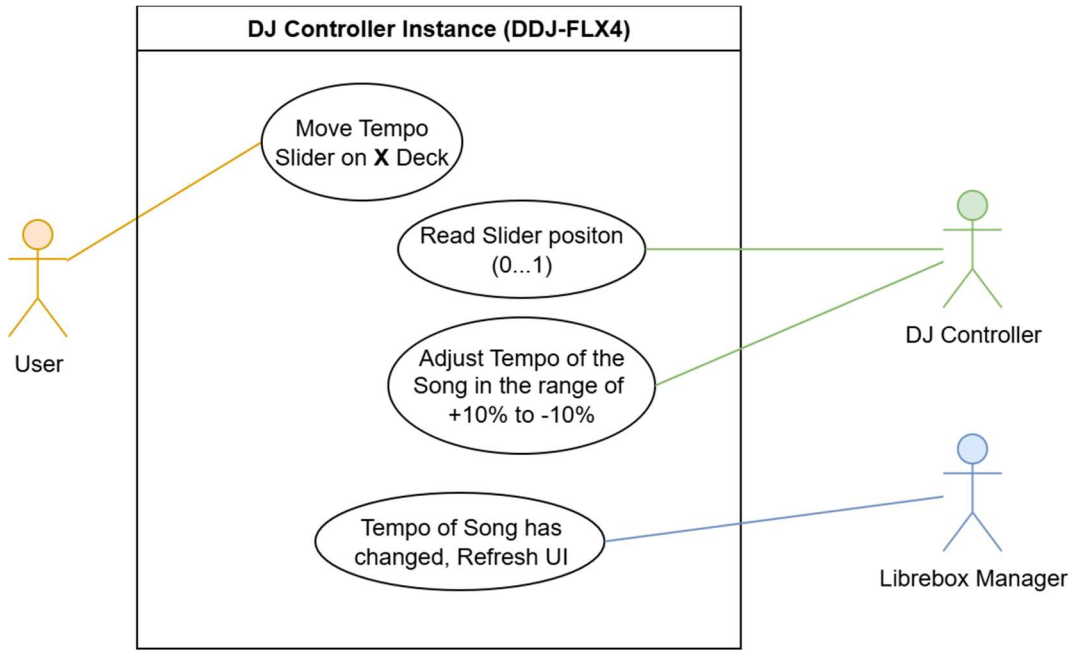
Main Flow (MF)		
Step	Action	**Alternative**
3.1	Librebox Track-selection receives the song selection signal for a specific deck of the DJ Controller	
3.2	Librebox forwards the load request to DJ Controller	**EF 3.2**
3.3	DJ Controller resets local state such as cue points, loop regions, and playback position.	
3.4	Unload previous song. Load the new song from disk	
3.5	HUB refreshes the active track card and waveform	**EF 3.5**
3.6	Deck on DJ Controller is now ready for playback	

Exception Flow (EF) 3.2: DJ Controller isn't available just yet		
Step	Action	**Alternative**
EF 3.2.1	DJ Controller is still being initialised and is not ready.	
EF 3.2.2	Defer all requests until after DJ Controller is ready.	**MF 3.3**

Exception Flow (EF) 3.5: The metadata has no valid waveform image supplied.		
Step	Action	**Alternative**
EF 3.5.1	The system detects that no waveform texture exists.	
EF 3.5.2	System attempts waveform discovery from alternative paths such as "user://waveforms/."	**MF 3.5**
EF 3.5.3	If waveform is still not found, the Librebox HUB displays a blank waveform image. Everything else proceeds as normal	**MF 3.5**

Non-Functional Requirements	
Consistency	Deck state and HUB visuals must always reflect
Runtime Stability	The load operation must not freeze or destabilise Librebox or the DJ Controller.

4. USE CASE 4 - CONTROL DECK PLAYBACK AND TEMPO



Goal	Allow the user to control playback state and tempo of a loaded song in a deck in the DJ controller.
Preconditions	Deck has a valid loaded song
Postconditions (Success)	Playback and tempo state are updated. Changes are immediately audible and visually apparent with Librebox UI changes
Postconditions (Failed)	The control input is ignored. Deck retains its previous state
Actors	User, DJ Controller
Trigger	User interacts with play/pause button or tempo slider on the 3D controller
Description	The DJ Controller controls pause/play state and applies pitch-scale / tempo changes to the active deck's audio stream
Priority	High

Main Flow (MF)		
Step	Action	**Alternative**
4.1	User presses play or pause on the target deck	
4.2	DJ Controller toggles the deck's audio stream paused state	**AF 4.2**
4.3	User adjusts the tempo slider on the DJ controller	**AF 4.3**
4.4	Audio output reflects the updated tempo	**AF 4.4**

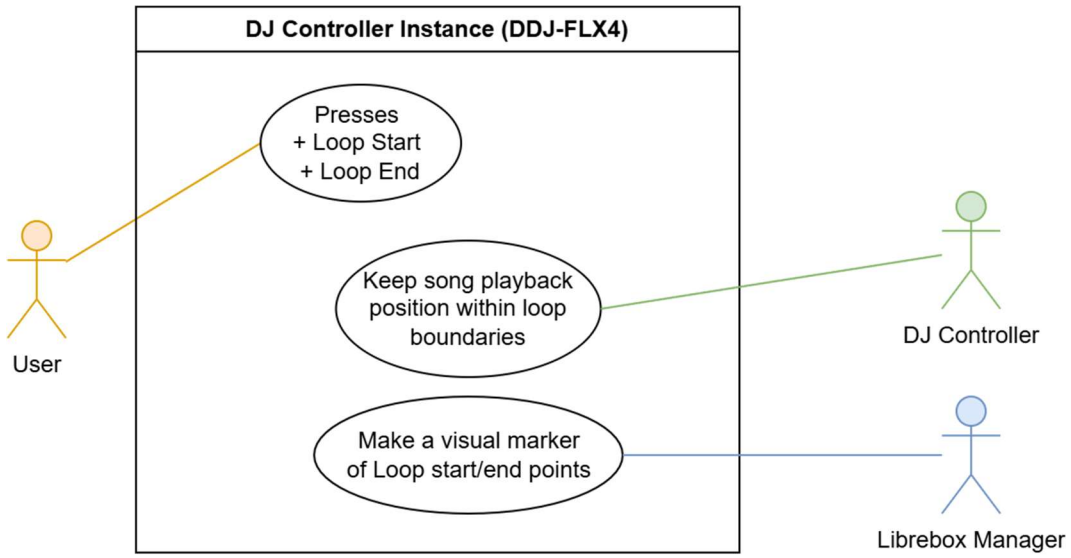
Alternative Flow (AF) 4.2: There is no song selected		
Step	Action	**Alternative**
AF 4.2.1	No song has been selected from the Librebox Track selection menu, so there is no song loaded in the deck.	
AF 4.2.2	Nothing happens	**MF 4.1**

Alternative Flow (AF) 4.3: Beat sync is turned on		
Step	Action	**Alternative**
AF 4.3.1	The BPM/Tempo of this track is already set to match the BPM/Tempo of the other deck because the user has Beat Sync turned on.	
AF 4.3.2	Nothing happens until Beat sync is turned off or if the tempo of the other deck is changed	**MF 4.1**

Alternative Flow (AF) 4.4: The Volume is set to 0		
Step	Action	**Alternative**
AF 4.4.1	The deck has its volume set to 0. Nothing will be heard	**MF 4.1**

Non-Functional Requirements	
Real-Time Responsiveness	Tempo changes must be perceived as immediate.
Audio Quality	Tempo adjustments should avoid any audible crackling or artifacts
Informative UI / Clarity	If Beat sync is enabled, the user should be made aware that they can't simply adjust the tempo with just the slider. Visual aids should be used to illustrate this.

5. USE CASE 5 - SET AND MANAGE LOOP REGIONS



Goal	Allow the user to define, activate, and deactivate loop regions on a deck for the current song
Preconditions	The target deck has a loaded song with a valid playback timeline
Postconditions (Success)	A loop region is active with valid boundaries. Playback loops within the region.
Postconditions (Failed)	Loop is not set or is disabled. Normal playback continues
Actors	User, Loop Management UI, DJ Controller
Trigger	User presses “Loop Start” and “Loop End”
Description	The user wants to loop a part of the current song. They mark loop start and end points, which line up with the beat of the song *(if enabled)*. The DJ Controller enforces cyclic playback within the defined region.
Priority	High

Main Flow (MF)		
Step	Action	**Alternative**
5.1	User presses the "loop start" button on the DJ Controller	
5.2	System makes remembers the position of the song where the user made this point. System then tries to nudge this position so that it starts on a beat, accounting for any expected latency too. Updates the Waveform texture too so the user can see where the loop starts.	**AF 5.2**
5.3	Later in the song, the user presses the "loop end" button on the Controller.	**EF 5.3**
5.4	System immediately loops back to the loop start position.	**EF 5.4**
5.5	Playback will loop indefinitely.	
5.6	User Halves/Doubles the loop region	**EF 5.5**
5.7	User disables the loop. Playback continues.	**AF 5.7**
5.8	User re-enables the loop. Playback snaps back and loops between the loop region again	**EF 5.8**

Alternative Flow (AF) 5.2: Beat Snapping is turned off		
Step	Action	**Alternative**
AF 5.2.1	The User disabled beat snapping.	
AF 5.2.2	The system will not nudge the loop start point to line up with the start of a beat.	

Alternative Flow (AF) 5.7: User deleted the loop instead		
Step	Action	**Alternative**
AF 5.7.1	The user chose to delete the loop region altogether	
AF 5.7.2	Playback will continue as normal now, but a new loop region must be set	**MF 5.7**

Exception Flow (EF) 5.3: User set the loop end point to an invalid location		
Step	Action	**Alternative**
EF 5.3.1	End point is either before start point or at the same location as the start point	
EF 5.3.2	Do not use this point as the end point, the user must select later in the song	**MF 5.2**

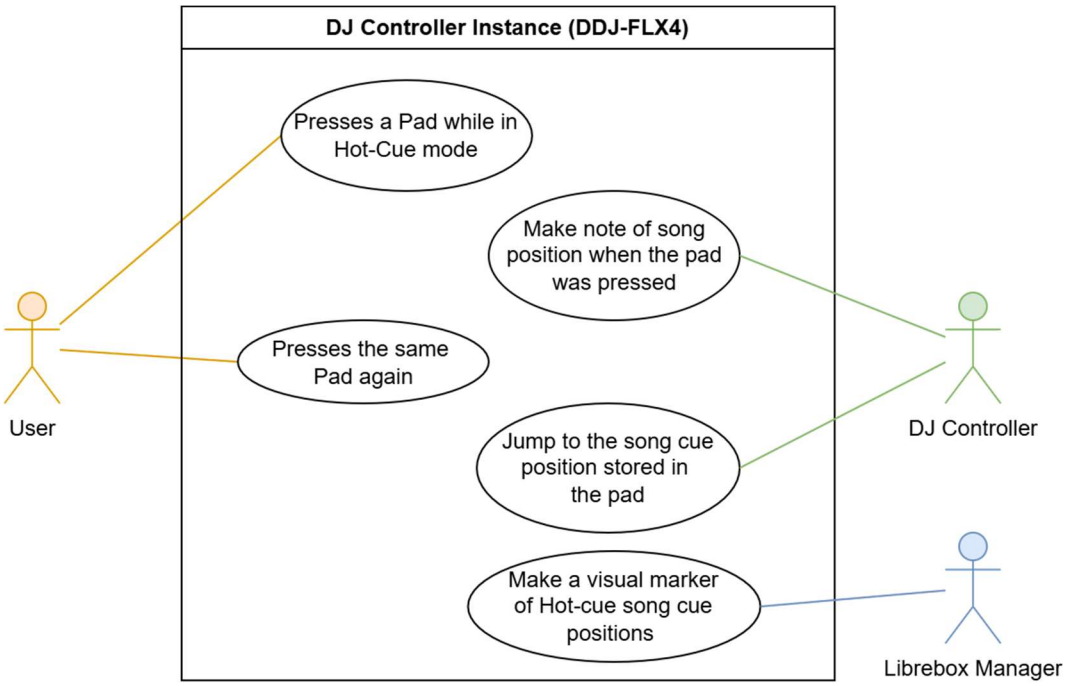
Exception Flow (EF) 5.4: Invalid loop start point		
Step	Action	**Alternative**
EF 5.4.1	Loop point was either never set or is AFTER the loop end point. It's not a valid loop region.	
EF 5.4.2	Do not attempt to loop	**MF 5.2**

Exception Flow (EF) 5.5: User tries to push loop length past limits		
Step	Action	**Alternative**
EF 5.5.1	Loop region is already too short, or too large.	
EF 5.5.2	Do not attempt to shorten or extend the loop region	**MF 5.6**

Exception Flow (EF) 5.8: Loop region is invalid		
Step	Action	**Alternative**
EF 5.8.1	Loop region has been deleted.	
EF 5.8.2	Playback will not change	**MF 5.2**

Non-Functional Requirements	
Beat Alignment	Loop boundaries should snap to beat positions to avoid audible mismatches, especially when Beat sync is active for any deck.
Seamlessness	Loop transitions can't produce audible clicks or gaps or other audio artefacts.

6. USE CASE 6 - CREATE AND TRIGGER HOT CUES (PERFORMANCE PADS)



Goal	Allow the user to save and recall cue points using performance pads for quick navigation during a performance
Preconditions	The target deck has a loaded song, and the Performance Pad is in Hot Cue mode.
Postconditions (Success)	Cue points are stored and playback seeks to the cued position when pressed
Postconditions (Failed)	Cue point is ignored, normal playback continues
Actors	User, DJ Controller, Librebox UI HUB
Trigger	User enters Hot Cue mode and presses a pad(s)
Description	When an empty pad is pressed, the DJ Controller stores the current playback position. When the same pad is pressed again playback will seek to the stored position.
Priority	Medium

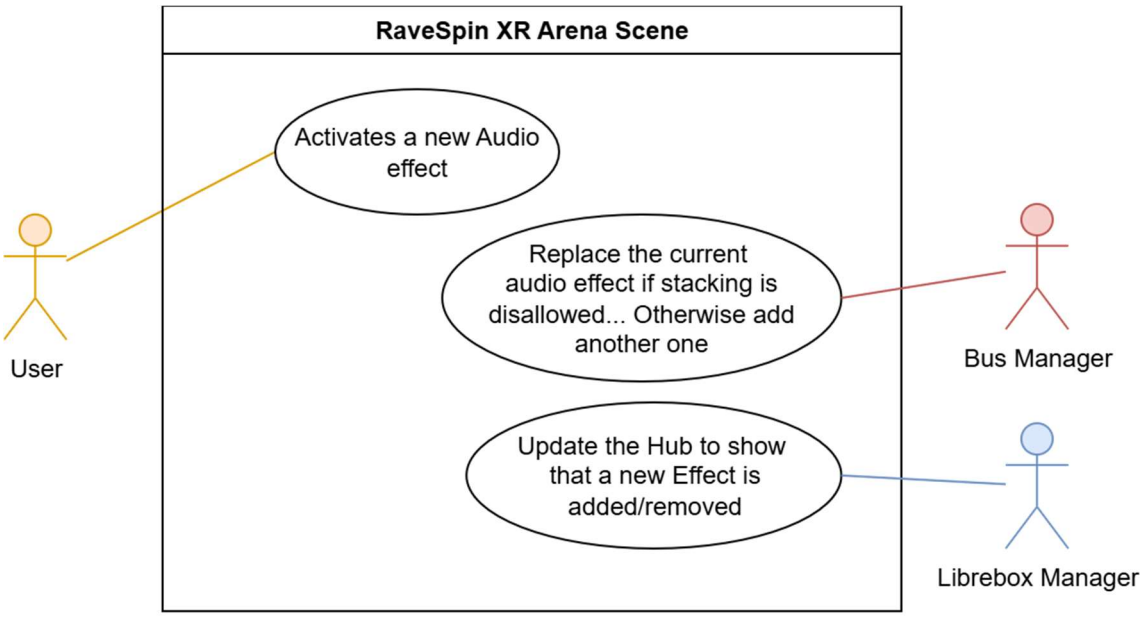
Main Flow (MF)		
Step	Action	**Alternative**
6.1	User sets the performance pad to Hot Cue mode	
6.2	User presses an unassigned pad	**AF 6.2**
6.3	DJ Controller stores the current playback position to the pad that was just pressed.	
6.4	Pad will start pulsing a unique colour to show that it has a Cue stored.	**AF 6.4**
6.5	Librebox UI HUB will update the waveform texture to show where the Hot Cue is in the song.	
6.6	User presses the Pad. The song will immediately seek to the stored position.	

Alternative Flow (AF) 6.2: The performance pad is set to Hot Cue – Delete mode		
Step	Action	**Alternative**
AF 6.2.1	The user presses a pad with a stored cue	
AF 6.2.2	The cue will be deleted	
AF 6.2.3	The pad will stop pulsing. The Librebox UI HUB will no longer show the cue point on the Waveform texture	**MF 6.1**

Alternative Flow (AF) 6.4: User selected a different mode		
Step	Action	**Alternative**
AF 6.4.1	Performance pad is no longer in Hot cue mode.	
AF 6.4.2	All pads with a stored hot cue will stop pulsing until we go back to hot cue mode	**MF 6.1**

Non-Functional Requirements	
Latency	Pad triggers must be instant.
Mode clarity	The active pad mode must be visually indicated so the user doesn't attempt to use the pads incorrectly.
Clarity of pad status	User must be made aware of what pads have a hot cue assigned and which don't. Where these hot cues lead to must also be demonstrated.

7. USE CASE 7 - APPLY BEAT FX VIA BUS POLICY



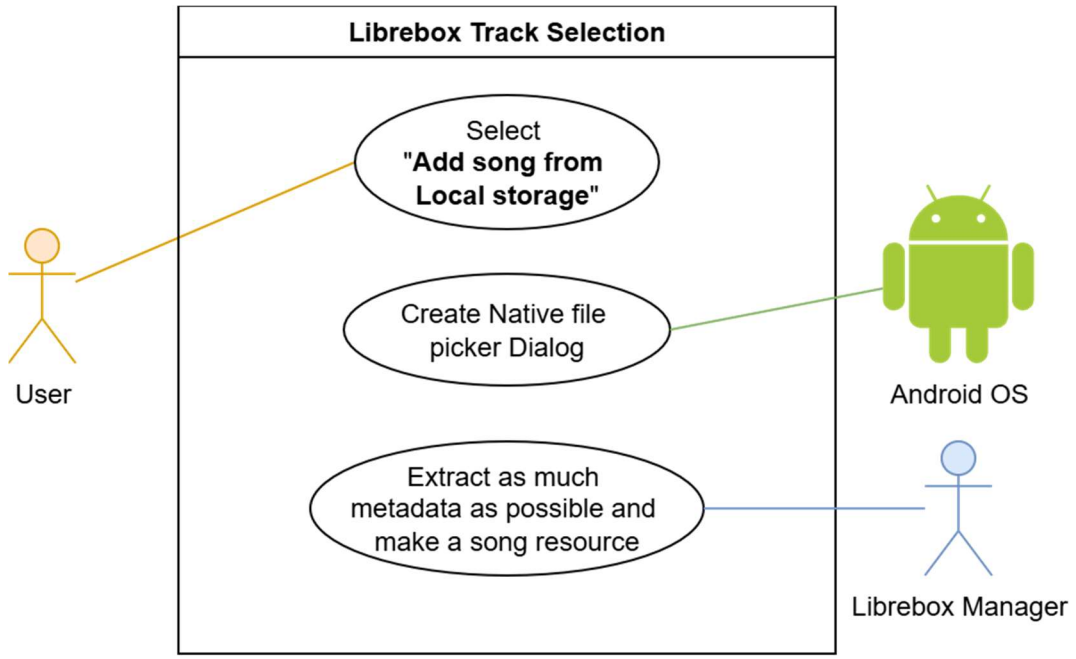
Goal	Allow the user to select and apply real-time audio effects to a deck's audio output
Preconditions	At least one deck has a loaded and playing song so the user can head the FX.
Postconditions (Success)	The selected effect is audibly applied to the target deck at the user-defined intensity
Postconditions (Failed)	The selected effect is not allowed and is rejected; the output remains unchanged.
Actors	User, DJ Controller, Bus Manager
Trigger	User changes FX
Description	User selects an audio effect from the Librebox UI FX Selection menu. Bus Manager evaluates how the effect will be applied depending on what FX policy it has. If it's allowed to, it will apply the request effect to the desired deck.
Priority	High

Main Flow (MF)		
Step	Action	**Alternative**
7.1	User selects an audio effect from the Librebox UI FX Selection menu. *(e.g. Delay, Reverb, Phaser, etc…)*	
7.2	User adjusts the FX intensity knob on the DJ Controller so that it is above zero. *(An intensity of zero means all FX are disabled)*	
7.3	User selects what channels are allowed to receive effects.	
7.4	User selects whether multiple effects are allowed to stack on top of each other, or if only one effect is allowed at a time.	
7.5	Bus Manager is responsible for managing the Godot audio server, which processes the modified bus chain in real time.	
7.6	User hears effect(s)	**AF 7.6**

Alternative Flow (AF) 7.6: User has disallowed any effects to be applied		
Step	Action	Alternative
AF 7.6.1	User has either set neither to be allowed to receive any effects, or the FX intensity has been set to zero	
AF 7.6.2	Bus Manager disables all effects. User will hear nothing	MF 7.3

Non-Functional Requirements	
Real-Time Responsiveness	Any FX parameter changes must be immediate.
Audio Quality	If audio cracking occurs *(especially when you have multiple stacked effects)* then the user must be allowed to mitigate this.

8. USE CASE 8 - IMPORT SONG



Goal	Import a new song from local disk into the user's music library so it can be loaded onto a deck.
Preconditions	The source audio file exists on the device and is in an accessible folder.
Postconditions (Success)	A Song metadata resource is created in the user library (user://Music/) and appears in the Librebox UI Track Selection list. Persists between runs.
Postconditions (Failed)	Song is not imported. User receives an error message
Actors	User, Librebox UI Add Track Menu, Android File System
Trigger	User wants to add a local song to their Rave-Spin XR music library.
Description	The user opens the Librebox UI Add Track menu and selects a local audio file to add. The system validates the file is valid and tries to extract its metadata. The system creates a custom song metadata resource that can be used by the other system components. The library list is refreshed.
Priority	High

Main Flow (MF)		
Step	Action	**Alternative**
8.1	User opens Add Track panel.	
8.2	User selects a local audio file (MP3, WAV, or OGG) to import.	**EF 8.2**
8.3	System generates a Waveform image of the song.	**EF 8.3**
8.4	Add track menu shows all fields about the song, such as title, BPM, artist, album art.	**EF 8.4**
8.5	User changes any fields that are incorrect	
8.6	User adds song to library	

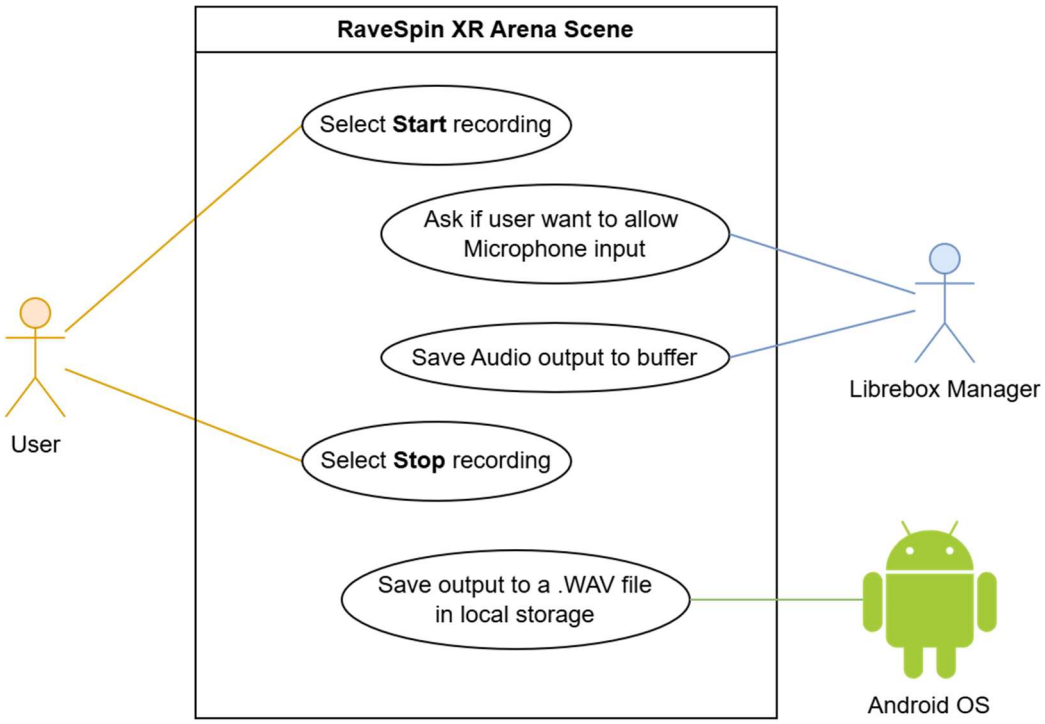
Exception Flow (EF) 8.2: File is inaccessible		
Step	Action	Alternative
EF 8.2.1	No File is stored in a location that Rave-Spin XR has no permission to access	

Exception Flow (EF) 8.3: A waveform cannot be generated		
Step	Action	**Alternative**
EF 8.3.1	System could not use FFMPEG to generate a waveform of the song	
EF 8.3.2	System will try use a different addon to generate a waveform image	**MF 8.4**
EF 8.3.3	If backup waveform generation addon doesn't work, show a blank image instead. The user will have to use "generate_waveforms.py" to generate the waveform image.	

Exception Flow (EF) 8.4: Details were missing from original audio file		
Step	Action	**Alternative**
EF 8.4.1	The original chosen audio file had missing metadata	
EF 8.4.2	System will try guess what this metadata is, or use "N/A" or blank values instead.	**MF 8.4**

Non-Functional Requirements	
Data Integrity	Imported metadata must be persistently stored and readable across future sessions
UI Feedback	Import success or failure must be communicated to the user clearly and immediately

9. USE CASE 9 – SAVE RECORDING



Goal	Record and save a session's audio output for later playback
Preconditions	Arena scene has been initialised and recording is ready
Postconditions (Success)	A recording is saved to disk and is accessible to the user later.
Postconditions (Failed)	No valid recording file is saved; the user is informed of the failure.
Actors	User, Librebox UI Recording Menu, DJ Controller, File System
Trigger	User starts and stops a recording.
Description	User starts recording, performs their DJ session, and stops recording. The system captures the master audio output and saves the buffer to a file on the device.
Priority	High

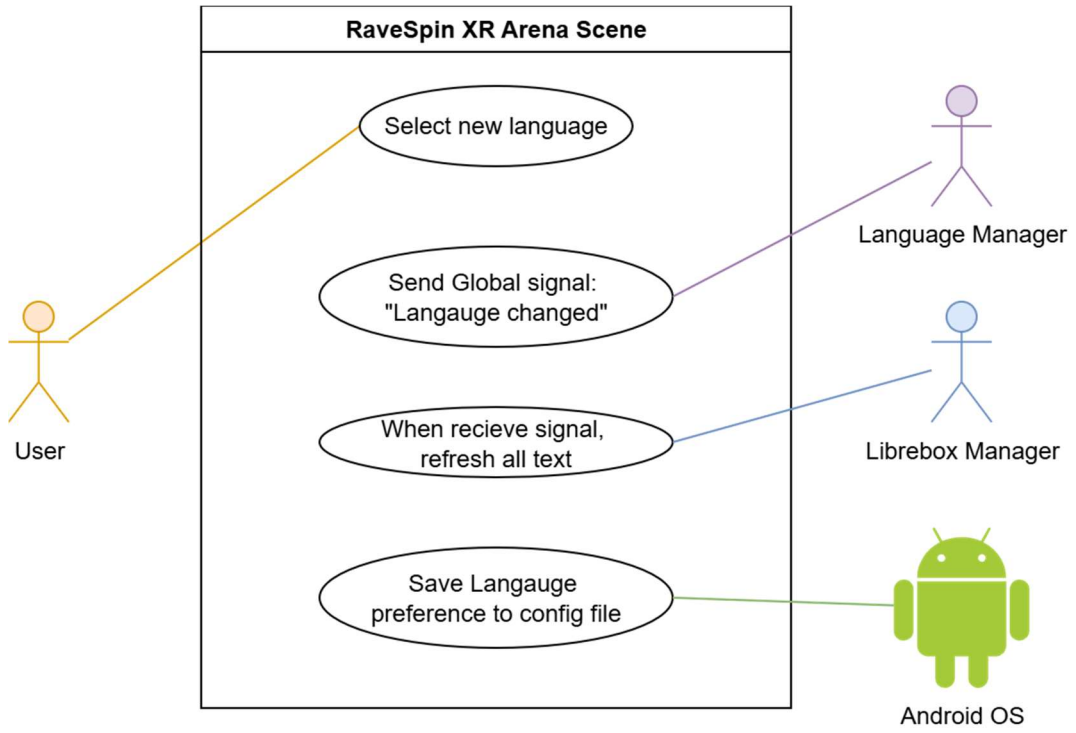
Main Flow (MF)		
Step	Action	**Alternative**
9.1	User presses “starts recording”	**AF 9.1**
9.2	System begins capturing the active master audio output.	
9.3	User presses "Stop Recording"	
9.4	System finalises recording file and writes to storage	
9.5	System confirms the save and displays the recording file path	**EF 9.5**

Alternative Flow (AF) 9.1: User also selects “Enable Microphone”		
Step	Action	**Alternative**
AF 9.1.1	System asks user if they would like to allow Rave-Spin XR	
AF 9.1.2	If the user agrees, show the microphone input button as enabled. Start recording microphone too.	**MF 9.2**

Exception Flow (EF) 9.5: The system failed to save the recording		
Step	Action	**Alternative**
EF 9.5.1	Recording could not be saved. Due to Insufficient storage, permission errors, etc...	
EF 9.5.2	Notify user that saving the recording failed and tell them why.	
EF 9.5.3	Apologise and prompt user to try record another session.	**MF 9.1**

Non-Functional Requirements	
Performance	Recording must not introduce perceptible playback stutter or latency
Storage Reliability	The save operation must avoid producing partial or corrupt output files

10. USE CASE 10 - CHANGE LANGUAGE



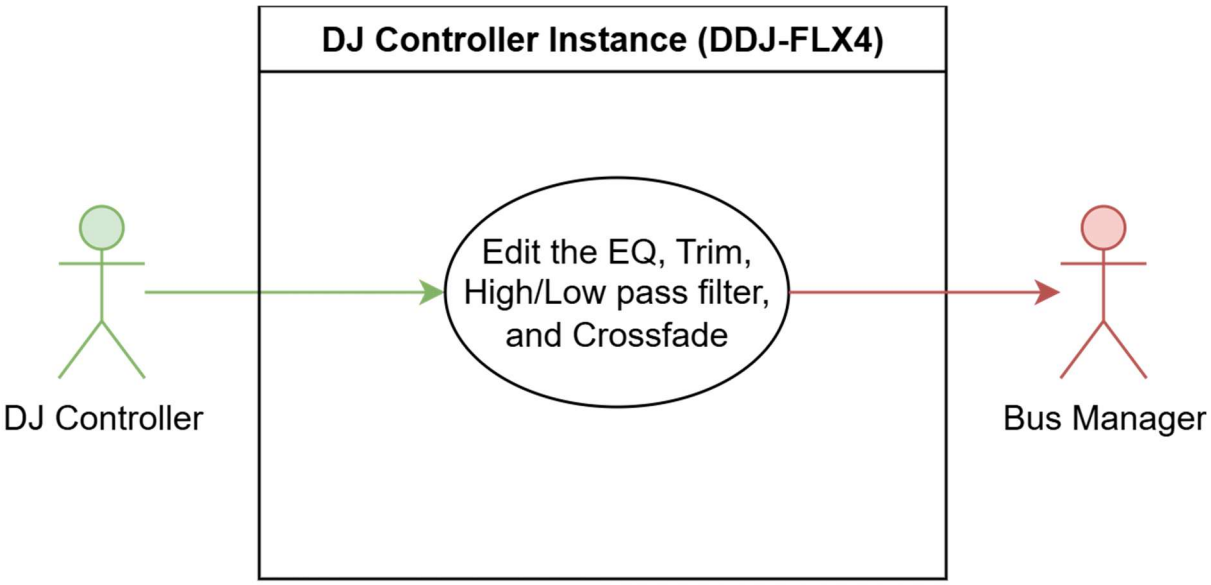
Goal	Allow the user to switch the UI language at runtime
Preconditions	The Arena scene is initialised, and the desired locale exists *(Only English, French, Mandarin, and Irish are supported)*
Postconditions (Success)	Selected locale is applied and persisted for next launch. All text elements refresh to reflect the change.
Postconditions (Failed)	Locale remains unchanged or fallback to English
Actors	User, Librebox, Language Manager
Trigger	User chooses a new language option from the Librebox UI Language Selection menu
Description	The user selects a new locale for the system to use. The language manager saves the change to a configuration file so that language changes persist through sessions. All text elements refresh to reflect the new change
Priority	Medium

Main Flow (MF)		
Step	Action	**Alternative**
10.1	User selects the new locale option from Librebox UI Language selection menu	
10.2	Language manager applies and saves the new language selection.	
10.3	System refreshes all text elements and labels to use the correct translation.	**EF 10.3**
10.4	System saves locale preference for future sessions	

Exception Flow (EF) 10.3: Missing translation for a certain element		
Step	Action	**Alternative**
EF 10.3.1	The element in question doesn't have a valid translation	
EF 10.3.2	Fallback to English version instead for just this element.	**MF 10.3**

Non-Functional Requirements	
Localisation Quality	Visible strings must update consistently
Persistence	Locale choice should survive restart and persist between sessions.

11. USE CASE 11 – MIX AUDIO (EQ / CROSSFADER / TRIM)

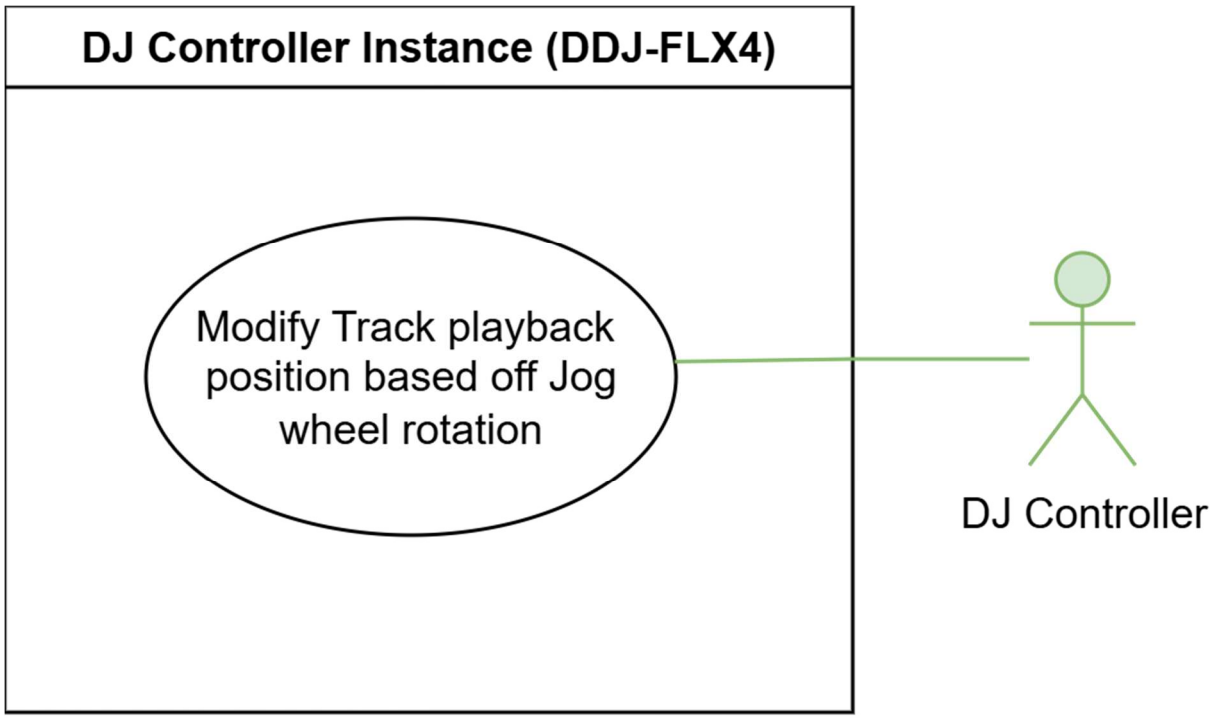


Goal	Allow the user to blend audio from two decks using per-channel EQ, trim faders, and the crossfader
Preconditions	At least one deck has a loaded song.
Postconditions (Success)	The audio mix reflects the knobs on the DJ Controller, such as the fader, crossfader, and EQ positions in real time
Postconditions (Failed)	Mixer adjustments produce no audible change
Actors	User, DJ Controller, Bus manager
Trigger	User interacts with mixer knobs, faders, and/or the crossfader on the DJ Controller
Description	The DJ Controller polls the knobs and slider positions each frame, maps them to audio parameters, and routes the values through the Bus Manager to accurately influence the bus chain.
Priority	High

Main Flow (MF)		
Step	Action	**Alternative**
11.1	User adjusts the trim gain on the DJ controller for a deck. This gives the deck a volume boost/dampen.	
11.2	User uses EQ knobs *(Low, Mid, High)* on the controller. This effects if we hear the lower portion, mid portion, or higher portion of the audio spectrum for that deck.	
11.3	User adjusts the fader for the channel on the Controller. This is effectively a volume slider for the channel.	
11.4	Controller notifies bus manager with any changes to the trim/gain, EQ, and channel fade for a specific bus. All of which update in real-time.	
11.5	User controls how much of Deck-1 (Left deck) or Deck-2 (right deck) by using the crossfade. Left means only listen to Deck-1, Right means listen to only Deck-2, and middle means listen to both tracks at once.	

Non-Functional Requirements	
Immediate feedback	Knob and fader changes must be reflected in the audio output immediately.
Precision	EQ band values must map smoothly across the full gain range.
Linear-to-DB	Knob values range from zero to one inclusive. EQ bands and gains are measured in decibels and can have different minimum and maximum decibel ranges.

12. USE CASE 12 – JOGWHEEL SCRATCHING



Goal	Allow the user to seek through a track using the Jog Wheel
Preconditions	The target deck has a loaded song.
Postconditions (Success)	The deck's playback position is updated according to the jog wheel rotation
Postconditions (Failed)	Jog wheel input is ignored. Playback position remains unchanged
Actors	User, DJ Controller
Trigger	User places hand on and rotates the jog wheel
Description	The DJ Controller detects the users' hand. If the hand is close enough to activate the jog wheel, and making a valid pose *(fist, pinch, or flat hand)*, the DJ Controller will calculate angular displacement, and map it to a seek offset based on the tracks' tempo, and updates the deck's playback position accordingly
Priority	medium

Main Flow (MF)		
Step	Action	**Alternative**
12.1	User places hand on the virtual jog wheel of a deck	
12.2	System detects the hand gesture and activates jog wheel to listens to hand position changes	**AF 12.2**
12.3	User rotates the jog wheel clockwise *(seek forward)* or counterclockwise *(seek backwards)*.	**AF 12.3**
12.4	DJ Controller maps the rotation to a seek offset based on the configured seek rate *(seconds per full rotation, currently set to 1)*	**AF 12.4**
12.5	Audio output and waveform display reflect updated position	

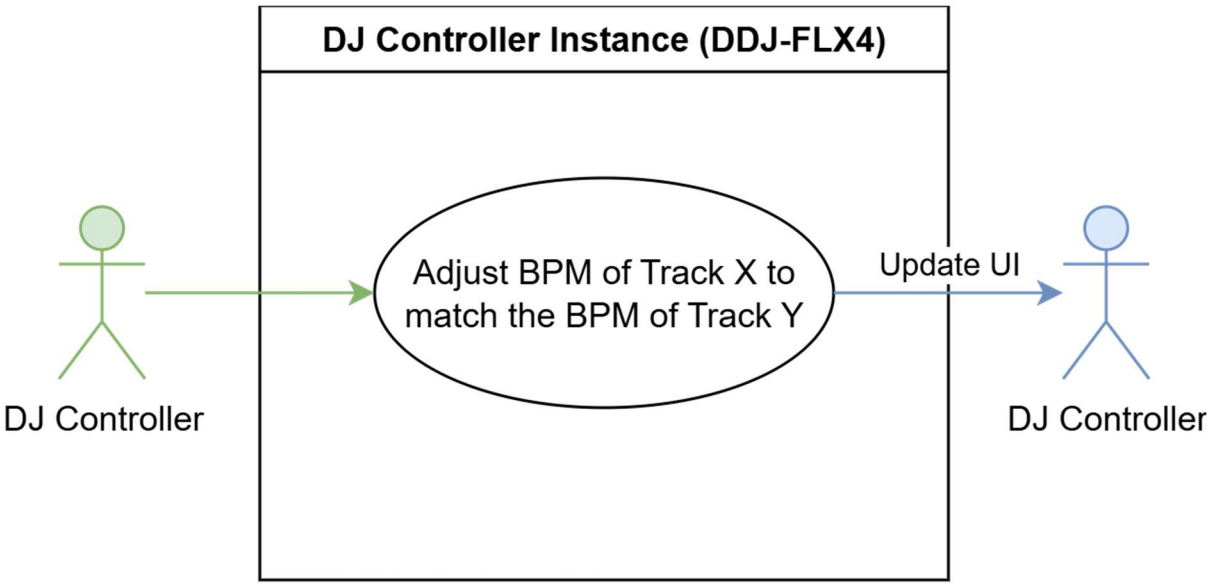
Alternative Flow (AF) 12.2: User isn't making a fist, pinch, or open-palm gesture		
Step	Action	**Alternative**
AF 12.2.1	System will highlight the Jog wheel red to show that it's not listening to hand movements	
AF 12.2.2	Only start listening once user starts doing the correct pose	**MF 12.2**

Alternative Flow (AF) 12.3: User has moved their hand too far away		
Step	Action	**Alternative**
AF 12.3.1	User was rotating the jog wheel, but has moved their hand too far away	
AF 12.3.2	Jog wheel will stop listening and continue as normal	**MF 12.1**

Alternative Flow (AF) 12.4: User is at the start/end of a song		
Step	Action	**Alternative**
AF 12.4.1	Deck has reached the start or the end of a song	
AF 12.4.2	Controller will block Jog wheel from seeking out of the song bounds	**MF 12.1**

Non-Functional Requirements	
Immediate feedback	Jog wheel rotation must produce an immediate audible response to simulate realistic vinyl interaction.
Gesture accuracy	The radial detection must be precise enough to distinguish intentional seeks from accidental hand movement.
Intuitive Jog wheel logic	Users shouldn't need to guess how jog wheel spinning will work. Should follow norms such as Clockwise == advance, anticlockwise == rewind, like real life vinyl players and CDJs.

13. USE CASE 13 – BEAT SYNC



Goal	Allow the user to synchronise the tempo of two decks so their beats align
Preconditions	Both decks have loaded songs.
Postconditions (Success)	Both decks play at the same tempo. Any tempo changes on the leader deck propagate to the follower track.
Postconditions (Failed)	Sync is not established. Both decks have independent tempos
Actors	User, DJ Controller
Trigger	User presses the sync button on a deck
Description	The DJ Controller matches the follower deck's tempo to the leader deck's BPM by adjusting the pitch-scale multiplier. Any tempo changes on the leader automatically propagate to the follower until sync is disabled.
Priority	medium

Main Flow (MF)		
Step	Action	**Alternative**
13.1	User presses the sync button on a deck	
13.2	DJ Controller reads the BPM metadata of both loaded tracks	**EF 13.2**
13.3	DJ Controller sets the beat sync state, which is "Left Synced to Right Track," "Right Synced to Left Track," or "Off" *(which is the default)*	**AF 13.3**
13.4	DJ Controller adjusts the follower deck's pitch-scale multiplier to match the leader deck's BPM. If the follower track had a BPM of 60, and the leader track has a BPM of 90, the follower track will have their pitch-scale multiplier set to 1.5x.	
13.5	Follower track waits a few milliseconds until the next beat of the leader track	
13.6	User adjusts the leader deck's tempo. Follower track automatically changes pitch scale to match.	

Alternative Flow (AF) 13.3: Beat sync was already on. Pressing it again disables it		
Step	Action	**Alternative**
AF 13.3.1	Beat sync policy is set to "off"	
AF 13.3.2	BPM of the follower track is set to whatever it was beforehand	**MF 13.1**

Exception Flow (EF) 13.2: One or more tracks weren't loaded with a song		
Step	Action	**Alternative**
EF 13.2.1	DJ Controller didn't have a song selected for one of the tracks.	
EF 13.2.2	Action is ignored as both BPMs are required to sync tempos	**MF 13.1**

Non-Functional Requirements	
Timing accuracy	The tempo match must be precise enough that beats do not audibly drift over extended playback
UI Feedback clarity	The user must be able to clearly identify which deck is the leader and which is the follower, and what the current beat sync policy is.

14. SUMMARY OF REQUIREMENTS

Here, in the table below, we can see a compilation of the requirements of the system.

Functional Requirements	Non-Functional Requirements
XR passthrough	Timing accuracy
Accurate hand tracking	Audio quality
Import audio files from disk	Immediate feedback
Audio metadata extraction	Gesture accuracy
Load music into DJ Controller deck(s)	Intuitive jog wheel logic
Control playback of a song on a deck	Precision
Adjust tempo of a song on a deck	Linear-to-DB translation
Adjust EQ and audio effects of a song on a deck	Localisation quality
Set and manage loop regions	Custom configuration persistence
DDJ-FLX4 performance pad functionality	Data integrity
Recording and saving of DJ sessions	Informative UI and clarity
Localisation	Real-Time Responsiveness
Jog wheel functionality	Low latency
Beat sync	Runtime stability
Downbeat alignment	Consistent beat alignment
	Seamlessness when looping

Table III-1 Compilation of requirements of the system

IV. PROJECT DESIGN

This section covers the development progress made to date, demonstrating the project's feasibility through the work completed across its key areas.

A. POTENTIAL ISSUES ADDRESSED IN PROTOTYPE

1. HAND CONTROLS

Hand movement in initial prototypes had no means of allowing the fingers or hands to bend or deform to match the orientation of the users' fingers. This originally led to unnatural and stiff movement, as the player node only tracked the location and base rotation of the user's hand. All fingers were ignored.

Now hand tracking extends to fingers and allows for fully accurate 1-to-1 matching with the user's real hand. Material used for the Hand model in Rave-Spin XR has also been changed to a semi-transparent blue holographic material. This is to both keep in with the cyber futuristic aesthetic but also to allow the user to see what controls lie past their hand. Due to the cramped nature of some of the components, it's more helpful to see these controls if your hand may be blocking it.

Some issues still remain however, as mentioned below in section IV.B "Inaccurate Hand-Tracking"

2. UI INTERACTION

Continuing on from Section III.B.2.b), (page 49): In previous prototypes, interactions with the Librebox UI could only be done with XR Controllers as they were what the included Godot XR Function Pointer natively worked with. The Hand Pose Detector was then later incorporated, and the Godot XR Function Pointer was attached to both Players Hands. This allowed the user to point at the Librebox XR UI Viewports and simulate a click from a fake XR Controller by making a Thumbs-Up pose, thus emulating XR Controllers input.

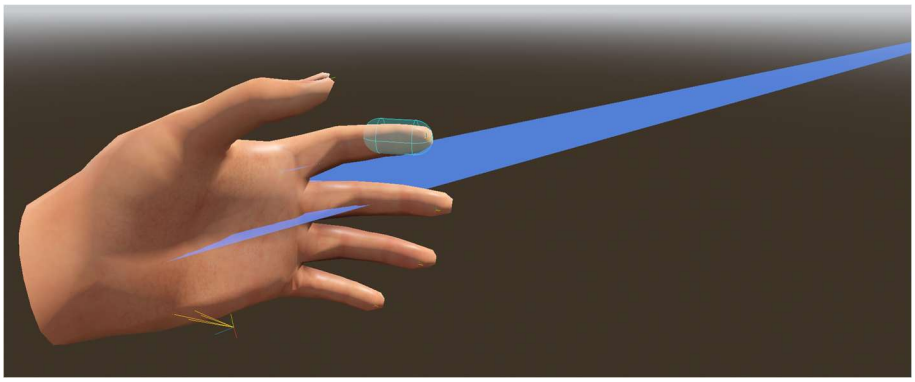


Figure IV-1 Old version of Hand from Player node. Laser + hand pose was used to interact with UI elements.

Figure IV-2 Old version of Hand from Player node. Laser + hand pose was used to interact with UI elements (in-action)

Because having a large laser pointer extruding from your hand was distracting, especially when not using the UI, it was made so that only a fist pose could activate the laser pointer and would turn red when colliding with a Librebox UI node. This played to the strength of the Hand Pose detector at the time as it was only a Fist pose and a Thumbs-up pose that could be consistently recognised.

After much early user testing and negative feedback regarding this, the Player class had to be reworked. Librebox XR UI Viewports were to allow the user to simply touch the viewports, as if they were giant touchscreens. This worked consistently and was much nicer to use than the Fist + Thumbs-Up Pose combo that was previously used.

After some reworking for both the XR Viewports and the Player Class, UI interaction is now much more responsive and the Code that handles this Hand-interaction is being refactored and will be submitted to the Godot XR Tools addon public repository. This is covered in Section III.B.2.b), “XR Hand Touch Interaction” *(page 49)*

3. PIONEER DDJ-FLX4 CONTROLS

The controls on the original versions of the DDJ-FLX4 were extremely finicky to say the least. Sliders jumped all over the place and did not consistently follow the users' hand and buttons kept "double pressing." Knobs were unusable.

These issues have since been addressed. Buttons now feel much more natural and have a slight cooldown between presses to prevent accidental double-click. Thanks to the insights of Bryan Duggan and his XR projects, sliders now work in a much more responsive and consistent manner. Due to inaccurate hand tracking when making the hand motion required to turn a knob *(as mentioned below in section IV.B "*

Inaccurate Hand-Tracking*"),* I instead opted to make a vertical slider spawn from the knob instead. This proved effective for two main reasons:

1. Slider logic was already implemented and extremely smooth and satisfactory.
2. It solved the issue of hand-tracking becoming extremely jittery when the hand was rotated to face away from the Meta Quest 3S sensors.

This proved to allow for much more precision and much better user feedback. Many users noted at first that they would prefer to have the ability to twist the knobs as you would with real-world DJ Controllers, but after getting used to it, they reported much better usability. Controlling knobs with sliders is here to stay.

4. PIONEER DDJ-FLX4 CLARITY

Another issue that was raised was, although the rendition of the Pioneer DDJ-FLX4 was very accurate, it was hard to tell components apart. As the project continued, I made sure to adapt the DDJ-FLX4 to XR. I did this by removing parts which didn't make sense in XR, such as the DB meter that was built-in to the original texture of the DDJ-FLX4 *(this was replaced with spatial one provided by Librebox UI)* and smaller controls were either expanded or moved elsewhere as to make them easier to access.

We can see the progress of the model used for the FLX4 in page as it goes from an accurate model to a more efficient model.

From Figure IV-3 Picture of FLX4 model in action (English language selected), we can see the DB meters that were originally on the FLX4, *(see Figure 0-2 Picture of a Pioneer DDJ-FLX4)* has been removed and changed to a DB meter from Librebox, and the names of the controls has been removed so that localised names can be used at runtime.

We can see the default English version in Figure IV-3. When we change the language, the labels are updated at runtime, see Figure 0-3 and Figure 0-4 on page 5 for screenshots.

Figure IV-3 Picture of FLX4 model in action (English language selected)

5. PERFORMANCE ISSUES WITH DJ CONTROLLER

When programming the logic behind how the DJ Controller would work, especially with Jog wheels, loops, Hot cues, I very quickly ran into the issue of terrible performance. I tried to alleviate the issue in several different ways.

A) MULTITHREADING

All logic in Godot, and most other programs, run a single line of instructions, or a single "thread" of instructions. Multithreading is the act of having multiple threads running in parallel. Godot is capable of this but with heavy restrictions. Any threads that impact the active scene tree *(in the case of Rave-Spin XR, it is the Arena scene)* will face several hitches as they must wait for the main thread to finish. Multithreading in Godot is ideal for any background processes and heavy calculations that can wait a few milliseconds. In the case of Rave-Spin XR this was done to manage several different components of the DJ Controller such as managing loops, cues, and Jog wheel "scratching" which works by seeking the song multiple times a second to a new position based upon by the angular offset that the jog wheel was moved. Although this greatly helped, performance issues kept persisting.

B) CHANNEL DISABLING

I noticed that after not having anything play for a few seconds, such as pausing a track for a few seconds, resuming playback cause the application to stall for several seconds. The profiler could not find the exact cause, instead just showing that the Physics took several hundreds of milliseconds during the frames where this pause occurred. I noticed that in the settings, there is a setting for "Channel Disable Time."

The purpose of this setting is:

Audio buses will disable automatically when sound goes below a given dB threshold for a given time. This saves CPU as effects assigned to that bus will no longer do any processing.

The default value for this was two. This meant that after 2 seconds had passed of no playback, the audio streams holding the music of a deck would discard itself. This meant that resuming a track after pausing it for more than 2 seconds caused a dramatic stall in the project.

This was addressed by changing the time to 300 seconds and have all pause/resume logic be delegated to separate threads. This helped even further but the performance issues still occurred when doing any jog wheel scratching or loops.

C) ON-DEMAND MP3 DECODING

This was the main culprit for all performance issues. MP3 music files are compressed, and Godot must decompress and decode them before any audio output is heard. These decoding steps are never shown in the Godot Profiler. The Profiler would say “This frame took 200ms to complete”, but when examining all the functions that occurred that frame and adding up all their individual times, it might add up to a far less alarming figure, such as 2-5ms. When selecting a song and performing the usual actions that caused such great performance drops, it never occurred in the first 6-8 seconds of a song. In that narrow period, Jog-wheel scratching, looping, and hot cues and effects caused no performance issues.

It was this moment, and an off-handed comment from a fellow student, Eduard Dravnieks, about how he only ever uses the .WAV music file format for any of his audio, that I realised what the issue could have been.

When downloading songs to test out the project, I used a site which grabbed the metadata from a YouTube link, such as the video thumbnails, proper ID3 tags, and downloaded the YouTube video in the form of a .MP3 music file with all relevant metadata attached.

As mentioned previously, MP3 is a compressed format, with a minor degree of quality being lost. Files must be decoded on the fly to hear anything correctly. WAV on the other hand is an uncompressed format. This compression step is why songs are typically shared as .MP3, but for producers, and audio purists, who need as much quality as possible, WAV is used instead.

I used FFMPEG to convert these .MP3 files into the .WAV format, changed where the Song resources would point to for the song’s files, and relaunched the application.

All performance issues were resolved.

I could set a cue at the start of a song, and another at the very end, and there would be an instant jump between the two cues with no lag. Jog wheel scratching had no delay and no performance drops. The decoding for .MP3 files in Godot is usually fine for normal playback, but for the purposes of Rave-Spin XR, it was simply inadequate and never showed on the Godot profiler. Because .WAV is inherently uncompressed, there is no heavy decoding required.

As mentioned previously, I mentioned that the first 6-8 seconds of a song caused no issues when preforming usually slow actions. This is because Godot initially decodes and caches this initial segment, then after the first segment, it decodes on-the-fly, leading to the performance issues.

In future iterations of the system, FFMPEG could be used to automatically convert any incoming compressed audio file into the .WAV format. This would allow for more flexibility for the user because if they decide to import their songs by using a music platform that allows downloading of songs but in a compressed format like MP3, they could still import it with no issues into Rave-Spin XR.

B. ADDITIONAL ISSUES

1. INACCURATE HAND-TRACKING

Although hand tracking for the most part is dependable and responsive, some issues remain.

A) INACCURATE POSE DETECTION AT CERTAIN ANGLES

When the users' hand is in front of the headset, the correct pose is identified. However, when the users' arm is extended and the hand is moved to face away from the headset, the pose is incorrectly identified either as a fist or a pointing pose.

From the provide pictures below, we can see "Figure IV-4 Screengrab of me doing a Thumbs-Up pose facing the Meta Quest 3S headset" and "Figure IV-5 Screengrab of me doing a Thumbs-Up pose pointing away the Meta Quest 3S headset".

Figure IV-4 Screengrab of me doing a Thumbs-Up pose facing the Meta Quest 3S headset

Figure IV-5 Screengrab of me doing a Thumbs-Up pose pointing away the Meta Quest 3S headset

In both pictures I am doing a Thumbs-Up, but when my hand rotates to face away from the sensors on the Meta Quest 3S, the reading loses its accuracy and either registers the wrong pose being performed, and/or jitters around the scene. In Figure IV-5, we can see it fails to register the pose at all. This occurs regardless of orientation.

Debugging this seemed to be out of the project scope as under normal circumstances, the pose detection works fine. I am presuming that due to the small number of sensors on the Meta Quest 3S *(the headset I have completed all testing on)* and perhaps an issue with the implementation of XR hand-tracking in Godot, that these inaccuracies occur.

B) HAND TRACKING ISSUES AT NIGHT

At nighttime, or in low light conditions, similar to how your mobile phones camera may lose picture clarity, the same downside exists for the Meta Quest 3S. When testing Rave-Spin XR in low-light conditions, fast hand movements which previously were not an issue, suddenly proved to be very temperamental. The same jitter that occurred in section IV.B.1.a) “Inaccurate Pose detection at certain angles” also occurred in low-light conditions.

The reason why digital cameras lose their quality in lowlight conditions is that they must increase their ISO and/or shutter speed. This is because there is less light to work with and the camera must allow lighter in at the cost of noise and potential blur due to movement [59]. I believe the Meta Quest 3S also suffers from this issue and uses regular digital cameras in tandem for User hand-tracking, thus leading to the hand-tracking issues.

Figure IV-6 Comparison of low ISO (left) vs high ISO (right)

2. USING EXTERNAL COMPILED BINARIES WITH ANDROID/META

As mentioned previously, I had wanted to use FFMPEG for generating the waveform textures for audio files at runtime. To get FFMPEG into Godot was a challenge.

Firstly, one of the strengths about Godot is the easy exploration and installation of modular engine plugins. I used this for basic Hand pose detection and the Godot XR tools. I had hoped that one existed for FFMPEG but one did not. I then sought to create my own Godot Engine extension *(or “GD-extension”)*, to use for just FFMPEG’s waveform generation ability.

This involved multiple steps, such as compiling the Godot engine from source, and then using Android Studio to generate a compiled binary that the Godot engine could use and would have FFMPEG functionality included. At first this seemed to work, everything compiled correctly, and Godot could recognise my custom GD-extension, and it threw no errors at runtime.

The issue came when testing. I explicitly made multiple checks in the FFMPEG extensions code, it would check itself every step of the way and throw an error if one was ever encountered, providing a unique error code so I could quickly diagnose the issue.

When importing a song, and adding it, everything passed all checks. No errors were reported, but the waveform texture never spawned, and after a 2nd check, Godot could not find the newly generated waveform texture. After much debugging, and ensuring file paths were correct, I came to the hypothesis that either Android, or Meta, have severe restrictions over what runtimes are allowed generate files [23].

After studying the issues that Lee Cox ran into in his project, "Augmented Reality Billiards Assistant (2025)", particularly with the permission issues posed by Meta, I believe this may be the cause. I attempted to use another publicly available plugin to generate the waveform as a fallback solution in case the original attempt failed.

This however caused more issues as the backup solution only generates waveform for .WAV audio files, and due to the 4 main formats that a WAV file may be stored *(8-bit PCM, 16-bit PCM, IMA ADPCM Compressed, and Quite-OK-Audio Compressed)*, the generated waveform had a ¼ chance of being accurate. I tried to implement my own custom Waveform generating solution within GD-Script but this falls into the realm of Audio Decoding which is well outside the scope for this project.

Other files created by the Godot Runtime have no issues being created, such as recordings, configurations files, etc... So were I to try this again, I would attempt to have FFMPEG return back a raw buffer, which Godot would then try to save as an image and save that image to disk itself.

My knowledge of Kotlin and Java is not good enough yet to attempt this in the short timespan unfortunately.

I could also possibly create my own fork of the Godot Engine source code and manually integrate FFMPEG into my own custom compiled version of the Godot Engine. When compiling the Engine and using the FFMPEG calls, the headset should see the calls as native engine calls, rather than external calls to another included binary [23]. This should work in theory, but this will require further testing to verify this.

I had enough issues and difficulty getting the GD-Extension to compile so I believe compiling the whole Godot Engine with custom FFMPEG code inserted could present even larger problems than before.

3. GITHUB DESKTOP

Godot stores its scene files (.tscn) as plain text rather than binary. This is useful because it makes them human-readable and means Git can differentiate changes within files easily and merge them. The downside is that a bug in GitHub Desktop *(version 3.5.4)* exposed a detrimental flaw to the project.

When multiple scenes were open in the Godot editor at the same time, GitHub Desktop ran the risk of simply mixing up which content belonged to which file.

What this meant in practice was that the contents of one scene file could be written into another. For example, the Arena scene file might end up containing the scene data from Librebox, with the original Arena data either partially or fully overwritten.

In some cases, sections of one file were inserted into another, leaving a hybrid of both. Because the resulting files were still valid text, and possibly because they passed internal linter checks, Godot didn't throw any immediate errors on project startup. The only indication something was wrong was when a scene file could throw a series of warnings about node references pointing to things that did not exist, and/or when opening the scene, I was greeted by another scene altogether.

This happened more than once. The issue was first noticed after committing changes to the Git repository using GitHub Desktop and seeing labels for nodes that should be in the DJ Controller .tscn scene file, were inside Librebox. These are two separate scenes that do not contain each other's scene data.

To reproduce it, I tried repeatedly opening Godot, and then GitHub Desktop, all without touching any files. GitHub Desktop would sometimes report changes that shouldn't have existed, as no editing of files took place, just closing and reopening of the same two programs.

Reviewing the changes made the problem obvious: the contents of **Add_Track_UI.tscn** were sitting inside **MainHub_UI.tscn**. There were now effectively two copies of **Add_Track_UI.tscn** and **MainHub_UI.tscn** had been overwritten. As mentioned, no intentional editing of files occurred. The behaviour only appeared when opening GitHub Desktop. Having other applications open seemed to have no effect.

I tried to uninstall GitHub Desktop entirely and use the Git terminal for all git commits going forward. After I had done this, no further corruption ever occurred after that.

This was a very bizarre and ironic bug. Although I am glad I was eventually able to catch it, I still had several hours of work corrupted and a git discard of all changes was required.

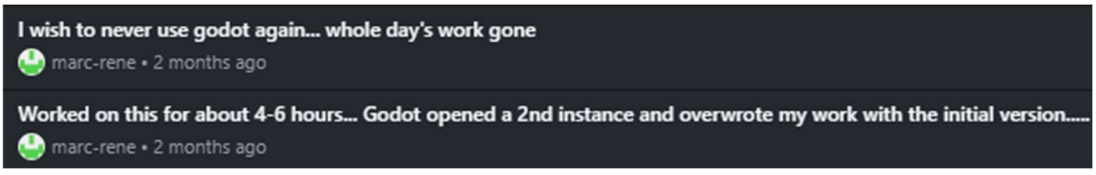


Figure IV-7 My commit messages when this issue first happened, after a full 2 days of work. Mistakenly believed it was Godots' fault.

4. RESTRICTIVE SPOTIFY WEB API

One of the main aims of this project was to allow the user to sync with their Spotify library. Primarily as a proof of concept to show that it could be done. At the project's inception, this was possible. The Original plan was to use the Spotify Web API to gather information on the users Spotify account and fetch their songs, playlists, and securely store their credentials so that they didn't need to log-in every session.

Due to the very nature of DJing, the user could easily find themselves in a location with poor Wi-Fi signal so it was imperative that the songs could be downloaded to disk. To do this I was going to use a tool like SpotDL, which is an open-source tool for downloading Spotify songs.

Later in the project, in February 2026, Spotify changed their policy and how their Web API could be used [60].

These changes effectively broke SpotDL and I could not find an alternative that seemed safe and reliable. This was incredibly frustrating as I had already done a minor amount of work on implementing Spotify integration but had to later scrap it due to the policy changes.

When importing a song using the Librebox UI, I left the buttons for Spotify, YouTube Music, and SoundCloud there but disabled them. Due to the open-source nature of Rave-Spin XR, should either myself, or another member of the open-source community find a solution, the infrastructure is already half-implemented.

C. CUSTOM DEVELOPMENT TOOLS

When making Rave-Spin XR, there was a few features that Godot did not have natively included, to remedy this, I made a series of my own tools to help with the development of the project.

1. GENERATE WAVEFORMS (PYTHON SCRIPT)

To make sure that we had the waveform images of the songs we wanted to test, there had to be an easy way to quickly generate the required images. I wanted to use FFMPEG, specifically the FFProbe that comes with it, as it is a very powerful tool and can be used for generating waveforms of songs. To avoid generating waveforms for songs one-by-one, I made the script “generate_waveforms.py” to go recursively go through the entire project folder which also houses all our songs.

Once it finds a song, it will make a waveform of it. By default, every 1024 pixels of width will equate to 1 minute of playback, so a 5-minute song will generate an image size of 5120px by 240px.

This newly generated image will automatically import itself into the Godot project as it’s running, and make sure that it has optimised values, such as a reduced colour palette *(it’s only the Alpha-channel, or transparency of the image we care about)*, and optimised for the Graphics Processing Unit in the Headset.

```python
def generate_waveform(path: Path) -> Path | None:
    """Generate one waveform PNG next to the audio file. Returns PNG path or None."""
    duration_sec = get_duration_seconds(path)
    if duration_sec <= 0:
        print(f"Skipping {path} (could not get duration)")
        return None

    duration_minutes = duration_sec / 60.0
    width = max(1, math.ceil(duration_minutes * PIXELS_PER_MINUTE))
    height = 240

    output_path = path.with_name(path.stem + "_WAVEFORM.png")

    filter_str = f"aformat=channel_layouts=mono,showwavespic=s={width}x{height}"

    cmd = [
        FFMPEG_BIN,
        "-y",
        "-i",
        str(path),
        "-filter_complex",
        filter_str,
        "-frames:v",
        "1",
        str(output_path),
    ]

    print(f"Generating waveform for {path} -> {output_path} (size {width}x{height})")
    result = subprocess.run(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=False)

    if result.returncode != 0:
        stderr_text = (result.stderr or b"").decode("utf-8", errors="ignore")
        print(f"ffmpeg failed for {path} (code {result.returncode})")
        print(stderr_text)
        return None

    if not output_path.is_file():
        print(f"Waveform PNG not created for {path}")
        return None

    return output_path
```

Figure IV-8 generate waveforms tool (waveform generation function)

```gdscript
# By defualt, GOdot's going to try make its own .import file for our waveform...
# usually it's good enough but we need to tweak it, otherwise we will have a capacity and rendering disaster
def patch_import_file(import_path: Path) -> None:
    try:
        text = import_path.read_text(encoding="utf-8")
    except Exception as e:
        print(f"Could not read {import_path}: {e}")
        return

    lines = text.splitlines()
    new_lines: list[str] = []

    current_section = None
    in_remap = False
    in_params = False

    # Track whether we have seen/overridden certain keys so we can append if missing
    seen_importer = False
    seen_type = False
    seen_metadata = False

    # Keys to replace in [params]
    params_keys = {
        "compress/mode",
        "compress/high_quality",
        "compress/lossy_quality",
        "compress/uastc_level",
        "compress/rdo_quality_loss",
        "compress/hdr_compression",
        "compress/normal_map",
        "compress/channel_pack",
        "mipmaps/generate",
        "mipmaps/limit",
        "roughness/mode",
        "roughness/src_normal",
        "process/channel_remap/red",
        "process/channel_remap/green",
        "process/channel_remap/blue",
        "process/channel_remap/alpha",
        "process/fix_alpha_border",
        "process/premult_alpha",
        "process/normal_map_invert_y",
        "process/hdr_as_srgb",
        "process/hdr_clamp_exposure",
        "process/size_limit",
        "detect_3d/compress_to",
    }
```

Figure IV-9 generate waveforms tool (Patching generated .import file function) part 1.

```
# End of file: if ended while still in remap/params, append our keys
if in_remap:
    if not seen_importer:
        new_lines.append('importer="texture"')
    if not seen_type:
        new_lines.append('type="CompressedTexture2D"')
    if not seen_metadata:
        new_lines.append(
            'metadata={"imported_formats": ["s3tc_bptc", "etc2_astc"], "vram_texture": true}'
        )

if in_params:
    new_lines.extend(
        [
            "compress/mode=2",
            "compress/high_quality=false",
            "compress/lossy_quality=0.7",
            "compress/uastc_level=0",
            "compress/rdo_quality_loss=0.0",
            "compress/hdr_compression=0",
            "compress/normal_map=2",
            "compress/channel_pack=1",
            "mipmaps/generate=false",
            "mipmaps/limit=-1",
            "roughness/mode=1",
            'roughness/src_normal=""',
            "process/channel_remap/red=8",
            "process/channel_remap/green=8",
            "process/channel_remap/blue=8",
            "process/channel_remap/alpha=3",
            "process/fix_alpha_border=false",
            "process/premult_alpha=false",
            "process/normal_map_invert_y=false",
            "process/hdr_as_srgb=false",
            "process/hdr_clamp_exposure=false",
            "process/size_limit=0",
            "detect_3d/compress_to=1",
        ]
    )

try:
    import_path.write_text("\n".join(new_lines) + "\n", encoding="utf-8")
    print(f"Patched import settings for {import_path}")
except Exception as e:
    print(f"Could not write {import_path}: {e}")
```

Figure IV-10 generate waveforms tool (Patching generated .import file function) part 2.

Please see section XI.A “Output of generate waveforms.py”, page 165 for Terminal output

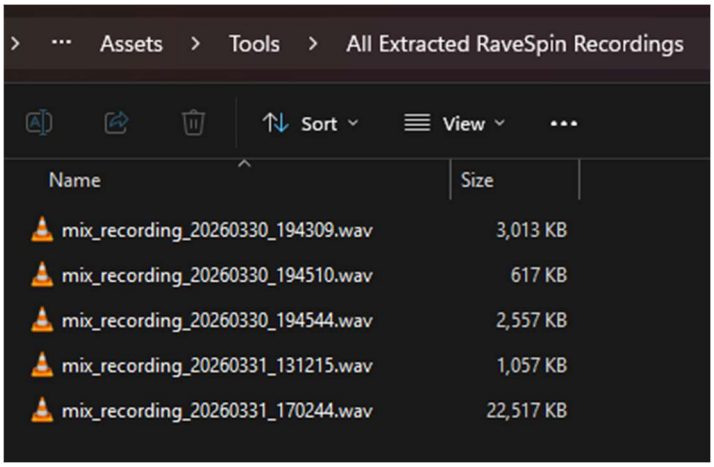
2. EXTRACT RECORDINGS FROM HEADSET (PYTHON SCRIPT)

Due to restrictions from Meta, you cannot simply share your files via Bluetooth using the Meta Quest 3S. Files can be saved to your Google-Drive, but this was not available to me as my Google-Drive had no storage left. I made a tool which will use the ADB interface to log into the connected headset and attempt to extract any recordings made in the RaveSpin folder on the Headset, to a local folder on the user's computer.

```
def list_wav_files():
    logger.info("Listing .wav files in remote directory...")
    cmd = f'adb shell run-as {PACKAGE} ls {REMOTE_DIR}'
    output = run_cmd(cmd)

    if output is None:
        logger.warning("Failed to list files or directory does not exist.")
        return []

    files = output.splitlines()
    wavs = [f for f in files if f.lower().endswith(".wav")]

    logger.info(f"Found {len(wavs)} .wav file(s).")
    return wavs


def pull_file(filename):
    remote_path = f"{REMOTE_DIR}/{filename}"
    local_path = os.path.join(LOCAL_DIR, filename)

    cmd = f'adb exec-out run-as {PACKAGE} cat "{remote_path}" > "{local_path}"'

    logger.info(f"Pulling {filename}...")
    result = os.system(cmd)

    if result != 0:
        logger.error(f"Failed to pull {filename}")
    else:
        logger.debug(f"Saved to {local_path}")


def main():
    logger.info(f"Starting extraction to {LOCAL_DIR}")

    os.makedirs(LOCAL_DIR, exist_ok=True)

    wav_files = list_wav_files()

    if not wav_files:
        logger.warning("No .wav files found. Exiting.")
        return

    for f in wav_files:
        pull_file(f)

    logger.info("Extraction complete.")
```

A) OUTPUT WHEN CONNECTED TO HEADSET

```
PS C:\Users\cesar> python -u "c:\Other\Git\RaveSpin\Assets\Tools\Extract Recordings from Headset.py"
2026-04-12 12:51:39,373 [INFO] Starting extraction to c:\Other\Git\RaveSpin\Assets\Tools\All Extracted RaveSpin Recordings
2026-04-12 12:51:39,373 [INFO] Listing .wav files in remote directory...
2026-04-12 12:51:44,492 [INFO] Found 5 .wav file(s).
2026-04-12 12:51:44,492 [INFO] Pulling mix_recording_20260330_194309.wav...
2026-04-12 12:51:44,645 [INFO] Pulling mix_recording_20260330_194510.wav...
2026-04-12 12:51:44,710 [INFO] Pulling mix_recording_20260330_194544.wav...
2026-04-12 12:51:44,839 [INFO] Pulling mix_recording_20260331_131215.wav...
2026-04-12 12:51:44,917 [INFO] Pulling mix_recording_20260331_170244.wav...
2026-04-12 12:51:45,772 [INFO] Extraction complete.
PS C:\Users\cesar>
```



3. GD-SCRIPT STRINGS TO ENUM (PYTHON SCRIPT)

When coding data structures and classes, one powerful element and data type to be aware of is the **Enum** *(Enumerator)*. In the case of Rave-Spin XR, if I wanted to keep track of where a Song came from, such as local disk storage, Spotify, Google Drive, etc, I could store the value as a String, like this:

Origin: "Spotify"

Problems may arise however when performing searches and saving this data for later versions. If I want to search a database of songs, each originating from different sources, I could have a search query for "Spotify" which in it of itself is expensive and slow, as each character of the string must be compared for a match. Problems also arise when we have slight variations.

For whatever reason, the String may change at any point from data storing and retrieval. It may become "spotify" or "SPOTIFY," or a slight typo like "Sppotify" could cause a search query to fail.

What an Enum allows us to do is to define a list of unique keys, which at runtime is treated as a single number by the computer, and is translated to a more readable name for the user.

We can see this with Enums such as **E_Track_Origins** or **E_MusicKeys**, which itself contains a unique value for each song origin, such as E_Track_Origins.Spotify, E_Track_Origins.SoundCloud, E_Track_Origins.YouTube_Music.

Because the computer treats these keys as just a single number, search operations are incredibly fast.

A downside to Enums in Godot is that there is no way to easily and automatically convert these Enum values as strings, and vice-versa. So, if I wanted to say, "This song came from **{ E_Track_Origins.Spotify }**", it would only come out as "This song came from **49**". There is no native function in Godot to automatically translate any Enum into a string.

To fix this, I made a tool called "**DEV_Create_Strings_As_Enum.py**" which will scan an array in Godot and insert an Enum version of it alongside it within the same script file. It will also generate the necessary translation functions to so we can easily have Enums convert to strings, and strings to Enums.

When running the script, you will just need to specify the initial parameters and then it will begin to run.

```
# What will we start looking for our array from? like 'const String_Array : Array[StringName] = ['
START_SEARCH_PHRASE = "const m_scales_str : Array[StringName] = ["
ORIGINAL_VAR_NAME = "m_scales_str"

# Where will we insert our Enum and String -> Enum function??
INJECTION_POINT_PHRASE = "### INSERT ENUM VERSION HERE ###"

# Name of the enum version of our string array
ENUM_VARIABLE_NAME = "m_scales_enum"

# MAKE SURE YOU'VE SAVED THIS FILE FIRST!
TARGET_GD_FILE = "E_MusicKey.gd"
```

A) TARGET SCRIPT

Here we can see a .GD script file from Rave-Spin XR. It's an array of scales that a song could be, in string format. We want to have an Enum version of this array and also be able to translate easily between the Enums and string versions.

```
const m_scales_str : Array[StringName] = [ "Unknown",
"Major",
"Natural Minor",
"Harmonic Minor",
"Melodic Minor",
"Gypsy Minor",
"Neapolitan Minor",
"Hungarian Minor",
"Major Pentatonic",
"Minor Pentatonic",
"Blues",
"Dorian",
"Phrygian",
"Lydian",
"Mixolydian",
"Locrian",
"Chromatic",
"Whole Tone",
"Octatonic",
"Arabic",
];
```

Figure IV-11 Screenshot of E_MusicKey.gd

After running the tool, it will automatically find the array of strings, and the point in the same file to insert the Enum version and translation functions into and begin to populate it.

B) OUTPUT

```
### INSERT ENUM VERSION HERE ##

## Enum version of `m_scales_str`.
enum m_scales_enum {UNKNOWN,               MAJOR,                NATURAL_MINOR,
HARMONIC_MINOR,         MELODIC_MINOR,          GYPSY_MINOR,
NEAPOLITAN_MINOR,       HUNGARIAN_MINOR,        MAJOR_PENTATONIC,
MINOR_PENTATONIC,       BLUES,                  DORIAN,
PHRYGIAN,               LYDIAN,                 MIXOLYDIAN,
LOCRIAN,                CHROMATIC,              WHOLE_TONE,
OCTATONIC,              ARABIC,                 }
## Converts scale text to scale enum.
## Input should be uppercase scale text.

static func m_scales_str_to_m_scales_enum(string_version : String):
    string_version = string_version.to_upper().strip_edges().strip_escapes()
    match string_version:
        "UNKNOWN":
            return m_scales_enum.UNKNOWN
        "MAJOR":
            return m_scales_enum.MAJOR
        "NATURAL MINOR":
            return m_scales_enum.NATURAL_MINOR
        "HARMONIC MINOR":
```

Figure IV-12 Automatically generated Enum translation code for E_MusicKey.gd.

V. TESTING AND EVALUATION

The testing of Rave-Spin XR was done incrementally throughout the entire development of the project. For items such as the sliders on the Controller, I would have made a base prototype that wasn't linked to anything of importance. Once I felt it was more than adequate, such as being responsive, consistent, and easy-to-use, I would then make multiple child nodes of that slider, to use for items such as the crossfaders and channel volumes.

This incremental testing approach made sure that any new item introduced could work independently on its own, have multiple objects derive from it, and be easy to reuse for other components. For example, this can be seen with the knob controls. Initially they were to have the ability to change their values by tracking the users hand rotation. Later, sliders were used instead, as mentioned in section IV.A.3 "Pioneer DDJ-FLX4 Controls". Because of the modular design pattern, changes such as this were possible and very easy to include, and almost no adjustments were required.

I wanted to make sure code was as clean and reuseable as possible. This is why when starting off this project, I made sure to account for potential scalability, such as including functionality for 4-deck controllers, even though the Pioneer DDJ-FLX4 is a 2-deck controller, because if I wanted to have the option to switch out controllers at runtime, this could be done easily.

This focus on code modularity and planning was to make sure the system was robust and to avoid any spaghetti code of dependencies.

A. SYSTEM TESTING

1. PARITY TESTING

The main idea behind Rave-Spin XR is to have a near 1-to-1 parity with the equipment in the application, and the real-world equipment it's based on. Any workflow that a user could do on a real physical Pioneer DDJ-FLX4, the exact same steps should result in the same outcome on the DDJ-FLX4 in Rave-Spin XR. To ensure this, I needed to have an actual DDJ-FLX4 to test on, as well as the User-Manual to ensure correct naming and workflow.

I requisitioned a DDJ-FLX4 controller from TUD – Grangegorman but was denied with no viable alternative. I instead had to borrow one from elsewhere and make do with the online emulator from Tribe XR when access to a physical board was unavailable. Although neither option was ideal and this did prove to be a major hinderance, it did get the job done, albeit painfully. If I were to redo the project, I would be much more persistent in ensuring I was requisitioned a physical controller.

I conducted parity testing throughout the development of Rave-Spin XR by making sure any new features, such as beat-sync, matched exactly how it worked in the application, and how it worked on the real-life equivalent. If I had Track A loaded and playing on the left deck, and Track B loaded and playing on the right deck, if I pressed the "Beat-Sync" button on the right deck, Track B would have its BPM changed to match the BPM of Track A, also making sure that Track B was playing on the same downbeat as Track A.

2. USER TESTING

When using the application, I always made sure to note down any grievances I personally had, be it minor such as controls being too small *(leading to the scale of the board being increased to allow for easier control)*, to major items of contention as well. Rave-Spin XR is as much a tool for the public, as it is a tool for myself.

Every couple of weeks, I would informally invite between one to four other students in TUD to try out the application. Before obtaining consent, I made sure the student knew that they were to test the system and that I would be noting down feedback for later iterations. I omitted all personal information from notes and observations as to avoid any potential breaches of GDPR.

I made sure to make a note of the users experience with DJ equipment and music theory and DJing so I could better accommodate for these groups. It's through this testing that I discovered items of interest like UI interaction being cumbersome, which is what led to The XR Hand-Touch node being created *(see section III.B.2.b) "XR Hand Touch Interaction")*, and how some users used their middle finger for pointing and clicking, rather than the index finger as I would have initially guessed.

During user testing, although it was all very informal and casual, I did make sure to ask Likert-Scale questions about certain features [61]. For example, one week I had just added Beat-Sync. On the Tuesday, when the other students came in, I offered them to try the latest build and ask them to try out Beat-Sync for one of the decks on the DJ Controller. During this time, I would gauge their thoughts about certain aspects such as visuals, UI responsiveness, audio-clarity / lack of any audio artefacts, etc.

I would make sure these questions could be answered back with a simple scale of 1-5, such as *"On a scale from 1 (not consistent) to 5 (most consistent) how often do you find Beat-Sync works at matching the tempo and the downbeat of both tracks?"*. Their responses and their existing DJ skill level would be noted down using my phone's notes-app or pen-&-paper.

Typically for every four students asked to participate:

- one had pre-existing experience or was an active DJ.
- two would know the basics of music theory but not DJ equipment and workflows.
- one would be a complete beginner on all aspects.

I wanted to make sure I had a good mix of people from diverse backgrounds trying the prototypes and final builds. In total, the approximate number of people who took part in user testing is approximately twenty.

Other changes that came due to user testing was making the Controller bigger by a factor of 20% so that it was easier to interact with the different controls even if they were close together. I also redesigned the FLX4 model used so that the controls could be even more spread apart to avoid any accidentally mis-interactions.

Inversely, I also shrunk down the Librebox UI Track Selection and the Librebox UI Hub. Some users had complained that they were moving their head back and forth too much to focus on the various components, which down the line could lead to neck strain, as the user is not only moving their head, but an XR headset on top of that too, leading to far greater fatigue. By making all the UIs smaller, I had to redesign the layout of some components, such as the Librebox Hub. The original version of the Hub can be seen here in Figure V-1 Screenshot of original Librebox Hub Before .

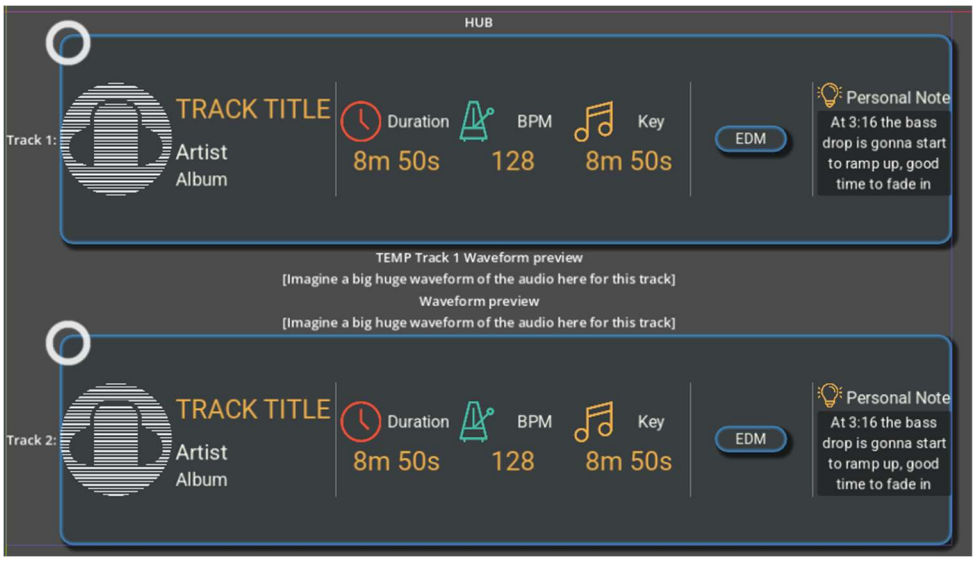


Figure V-1 Screenshot of original Librebox Hub Before redesign.

The redesigned version *(see Figure III-16 Screenshot of Librebox HUB – In English mode. On page 52)* let the elements be better placed and fit into a nicer 1:1 square aspect ratio, rather than the previous 16:9 aspect ratio that the old design in Figure V-1 was forced to use. The change in aspect ratio and better layout of elements led to less complaints.

3. RESULTS AND FINDINGS

After compiling all user feedback, I made the following findings regarding Rave-Spin XR. I asked users to complete a series of tasks, increasing in difficulty. First, I would ask them to select a song for a track and start playing it. Then I would ask them to explore further or "mess around some more", such as try Jog Wheel scratching, move the knobs to random values, slowly but surely I would go through most of, if not all of the use cases found in Section III.D "Requirements analysis" (page 82).

If the participant in question was a beginner DJ, or was not knowledgeable with music theory, I would not ask them to complete the more complicated use cases such as setting hot cues, loop cues, pitch-shifting, or more complicated specific use cases.

If the user was experienced with how to DJ using existing equipment, or knew music theory, then I asked them to complete as many of the use cases as possible.

A) EASE OF USE AND NAVIGATION

All users during testing showed no issues getting started. Starting off was no issue at all and almost everyone was able to select a track and start playing immediately. Buttons and sliders were no issue at all. Knobs and Jog wheels were the point of greatest contention. Many users expressed a desire to have the knobs be rotatable with the users' hands, as they would in real-life. Some users liked the spawning sliders and found it intuitive, but having the option to control the knobs via twisting would be something I'd have to find a solution for as it was brought up multiple times.

The same goes for jog wheel scratching. In its current state, the Jog Wheels can only be interacted with if the user is doing a fist pose or an index pinch pose as to avoid any accidental scratching. Many participants didn't like this feature and asked that it be changed to allow an open palm-pose to allow scratching also. This has been changed in the latest version of Rave-Spin XR.

If any other users ran into issues getting started with Rave-Spin XR, I also created a "Getting Started" video guide to help assist users. It is hosted publicly on YouTube and on the official Rave-Spin XR website. The video demonstrates how to use all aspects of the system and how they work.

B) USER INTERFACE

For the most part the UI got no major reactions. It appears it was serviceable for the most part. Several users mentioned that it would be nice to have the option to switch between a light mode and a dark mode for the user interface, like most modern UIs.

Comments were also made to allow the option to disable transparency on certain elements. When Rave-Spin XR is being used, the transparency of items, such as the waveform of the current active song proved to be quite distracting for certain users and made it harder to follow along. We can see this in Figure V-2 where there is no background behind the waveform and HUD elements, in busy environments, this could pose a problem. Asking these users if having the option to make the background opaque rather than transparent would be preferable and the consensus was a unanimous yes.

Work has started to make this a togglable option, but it has not finished implementation yet.

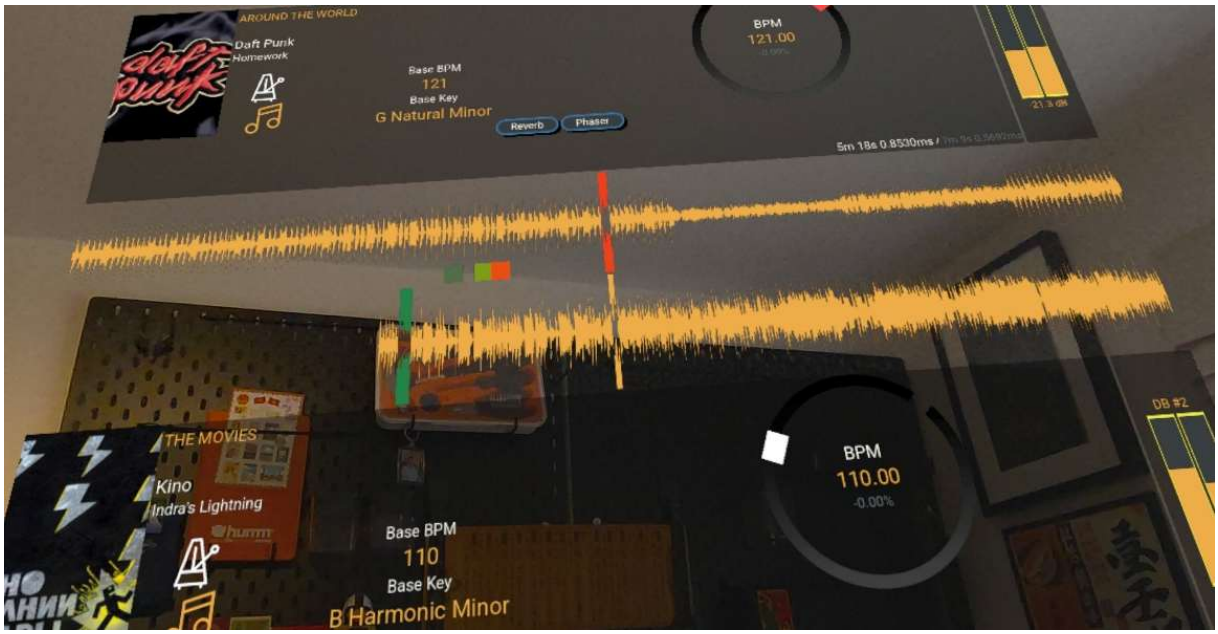


Figure V-2 RaveSpin in action, transparency of elements could make following information difficult.

Users who had previous DJ experience noticed how the layout and interface of Librebox UI, particularly the Librebox Hub, was supposed to take great inspiration from the UI of Rekordbox *(another popular industry standard software for DJs for managing DJ equipment and controllers)*. These users mentioned it would be nice to have the option to change the colour scheme and waveform images to match the UI of Rekordbox. We can see how the Librebox Hub *(see Figure V-2, Figure III-16 page 52, and Figure III-5 page 47 for screenshots of Librebox Hub)* compares to the UI of Rekordbox *(see Figure V-3 Screenshot of Rekordbox UI.)*.

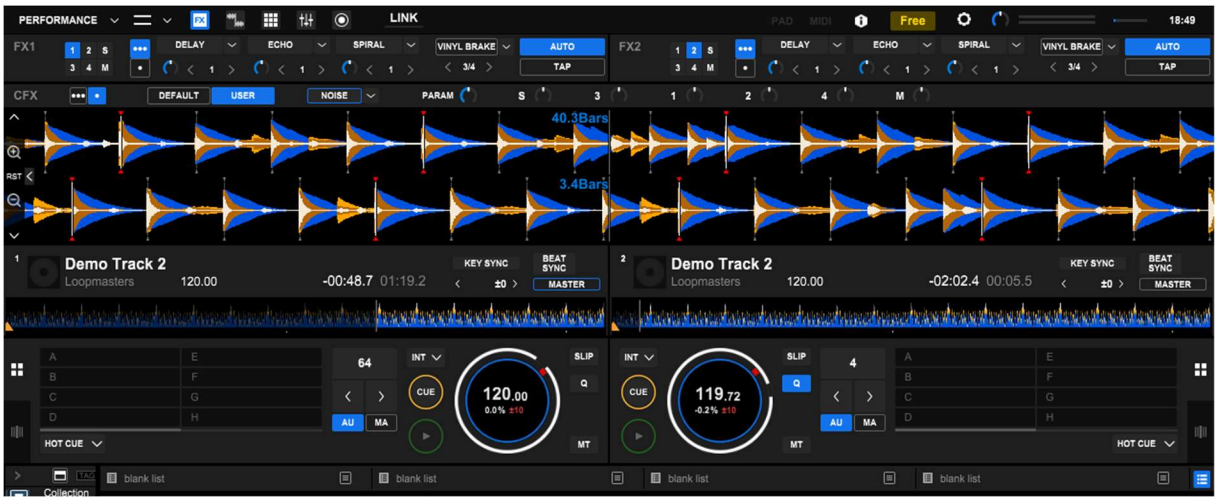


Figure V-3 Screenshot of Rekordbox UI.

C) FUNCTIONAL TESTING

Users commented that when starting Rave-Spin XR, the application can take some time to get running up to full-speed, several components lag and freeze for the first few seconds of runtime, but after this initial freeze it seemed to go away fully. I investigated this and realised it was due to shader compilation stutter. This is where the application has to compile the math and code that goes into a visual element, such as a material or a holographic effect, which is used extensively for the performance pads when in Hot-Cue mode, the different controls when hovering and activating them.

Godot will pause the entire application until this compilation is finished. The benefit is that we never see materials that are incorrectly rendered, and after this initial stutter the results are cached so no more compilation is ever required.

The downside to this is that this caching does not persist between sessions. Closing the application and restarting it will require this stutter to happen again. I am currently looking into how to cache any shader compilations but as of writing this, it is not enough of a detriment to the user experience, its priority is simply too low as it happens only within the first 10-15 seconds of a session.

Experienced DJs also commented on the jog wheels. Users mentioned that they would like the Jog wheel behaviour to be adjustable, either to be faster, or even have their spinning with track playback be disabled.

By default, when a song is playing at normal speed *(the normal BPM of the song matches the Current BPM that it is being played at)*, the jog wheel will spin at one full rotation per second. If the song has its speed changed to 1.2x speed, then the Jog wheel will turn at 1.2 full rotations per second. We can use BPM for this analysis because in Rave-Spin XR the BPM of a song is treated as a static value, rather than variable *(future edition of Rave-Spin XR would have the BPM be analysed at runtime to account for songs with variable tempos)*.

$$Rotations\ per\ second = \frac{Current\ BPM}{Base\ BPM}$$

Users mentioned that they would like this to be adjustable as some would have preferred the Jog wheels to the behaviour of something like the motorised Jog wheels of the Pioneer DDJ-REV7 *(Another popular DJ Controller, like the DDJ-FLX4 but with automatically spinning jog wheels)* [62] which have an arbitrary base rotation speed. Assuming this arbitrary rotation scaler is **X**, the rotation formula would become:

$$Rotations\ per\ second = X(\frac{Current\ BPM}{Base\ BPM})$$

This would be much more ideal for managing jog wheel spin as it would address all user concerns regarding jog wheel rotation. Users who want the jog wheels to not spin with the track playlist could have **X** set to zero, and users who want something faster, akin to the DDJ-REV7 or a Vinyl Record Player, they could set **X** to 0.55 or 0.75 to similar 33 RPM and 45 RPM Vinyl players.

Experienced users also mentioned that the sensitivity of the Colour FX was far too high. Normally, a value of 0.5 should disable any high pass filter and any low pass filter. The way Colour FX works now is that the Low Pass filter only becomes audibly noticeable when the Colour FX is set between 0.0 ~ 0.15. Any values from ~0.15 – 0.5 produce no major audible change.

The High pass filter also suffers from a similar issue. 0.5 ~ 0.6 produces the greatest change with the high pass filter. Any Colour FX value past 0.8 causes no noticeable change either.

From these findings I learnt that the low pass filter and high pass filter shouldn't have their thresholds set upon a linear progression ranging from one point to another. Instead, a floating-point curve would ideally be used instead, similar to how 2 floating point curves were used for the Crossfader slider found in Section III.B.3.b)(1) "Crossfader" *(page 68)*.

Another issue that experienced DJs brought up was with the audio effects functionality. Several users noted that it would be ideal if there was a separate interface to control the more specific aspects of certain audio effects. An example of which was the Panner effect. The way this effect works is that it confines all sound into either left or right speakers, and Pans it from one side to the other to give the illusion that the music is panning around you. The way this works at the moment is that the effect is activated, but there is no way to control this specific variable. The FX level knob could have the Panner effect at '1' which means that it will be confined to the right speaker.

If the FX level knob was 0.5, then the sound would be at both left and right speakers equally with no skew to either side, but the way the Bus Manager is implemented in its current state, it sets the influence of this effect to 0.5 as well.

This was noted and will be addressed in future versions of Rave-Spin XR.

B. SYSTEM EVALUATION

1. PERFORMANCE – GENERAL

After the initial shader compilation stutter mentioned above in Section V.A.3.c) “Functional Testing”, the frame rate of the Rave-Spin XR is able to hover at around 60fps *(frames per second)* or ~16.6ms per frame quite consistently.

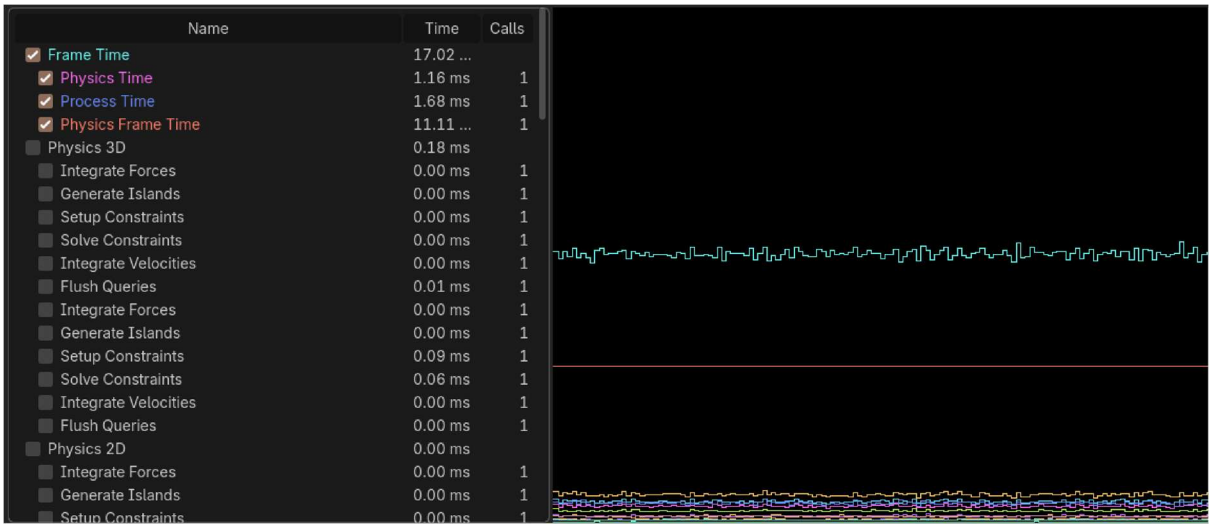


Figure V-4 Screenshot of profiler of RaveSpin XR

Several intensive tasks are handled by separate threads, and the main thread responsible for rendering is on a separate thread too. The separation of the render thread from the main thread proved to be a fantastic choice as if the FPS kept dropping, it could very easily disorient the user and potentially cause motion sickness. With a separated render thread, the FPS is much less susceptible to sudden hitches or drops.

As mentioned previously, attempting to use .MP3 files will cause the program to stutter immensely, as doing anything besides standard pause/play of songs will cause Godot to stall until enough of the data from a song has been decompressed and decoded. I mentioned this in on page 115, section IV.A.5.c) “On-Demand MP3 decoding”.

When using .WAV files, performance always remains consistent and the FPS doesn’t appear to be affected.

2. PERFORMANCE – AUDIO

Audio clarity has, for the most part, been very clear. Issues may arise however if the user enables Poly FX and begins stacking audio effects on top of each other. The Meta Quest 3S is not the most powerful headset on the market today. When multiple effects, particularly complex ones such as Reverb, Delay, and Pitch-shift are enabled and stacked on top of each other, audio clipping can occur. I had initially addressed this by giving the Godot Audio server more leeway by giving it 30ms to render its audio before it had to be output *(see Figure V-5)* instead of the default 15ms, but multiple audio effects still pushes the headset past its limits.

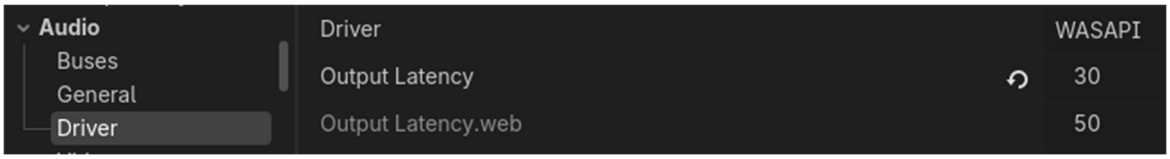


Figure V-5 Screenshot of Project settings for the Audio Server in RaveSpin

Other actions such as Jog wheel scratching, jumping between hot cues, loops, pitch-shifts, and firing samples using the performance pad in Sampler Mode, even when done together, causes no discernible performance loss.

3. USABILITY

The goals that this project set out to achieve have, for all intents and purposes, been accomplished. With a real physical Pioneer DDJ-FLX4, a DJ needs some way to load their music onto the controller, manipulate that audio through mixing, effects, and EQ, and optionally record their sessions. RaveSpin XR replicates this entire workflow. Tracks can be imported from local storage, loaded onto either deck, mixed using the crossfader and channel faders, shaped with EQ and colour FX, layered with beat effects, and recorded as a WAV file, all from within the headset. The same sequence of actions that a DJ would perform on a physical setup can be done in RaveSpin XR with little to no adjustment, which was the core objective of the project from the very beginning.

Beyond simply matching the hardware's functionality, the application also adds conveniences that physical equipment does not offer. The Hub provides a persistent at a glance overview of both decks, including live waveforms with hot cue markers, current BPM, active effects, and dB metering, all visible without needing to glance at a separate laptop screen. The tooltip system means a new user can hover over any control and immediately learn what it does, which is something that no physical controller provides out of the box. These additions do not replace the need for real hardware in a professional setting, but they do make RaveSpin XR a genuinely useful tool for learning, practising, and mixing on the go.

To further assist users when using Rave-Spin XR for the first time, a tutorial was recorded and publicly released on YouTube and the official Rave-Spin XR website [63]. This tutorial was to give users a quick guide about how the application works and how to get started using the different features.

C. PROJECT MANAGEMENT EVALUATION

In this section we will cover the planning process of Rave-Spin XR.

I had originally drawn up a Gantt chart to help guide the general trajectory of the project progress. It's broken down into 5 distinct phases with milestones attached. This was to make sure that development progressed at a steady pace, but also to give me a clear progress bar I could check during the semester to gauge whether I was on track or falling behind.

Figure V-6 Original Gantt Chart of RaveSpin XR

For the most part, the chart was followed quite closely and most of the milestones were hit as planned.

The project deadline was due in late April. The project was fully completed in late March.

1. PROTOTYPING AND DESIGN

Unfortunately, as I've never used Godot in any huge capacity, especially none remotely to the scale of RaveSpin, I ran into many roadblocks that caused the timeline to shift a couple of times. Certain features that I had initially estimated as small tasks ended up requiring significantly more time to research, prototype, and debug than I had originally accounted for, and this knock-on effect pushed other planned work further down the schedule.

As a result of these delays, I was unable to implement the bonus features I had originally intended. An example of which was local WLAN multiplayer, which would have allowed two users who were on the same network to DJ together in the same shared session.

After consulting with several users, both experienced and beginner, it was unanimously decided that the feature was better off being set aside to a low priority item, especially if it would come at the detriment of other higher priority features which were wanted at the time, such as Beat-Sync and Jog wheel scratching.

Throughout the development of Rave-Spin XR, including prototyping and design, I always made sure to invite other users in TUD to try out the application. It's thanks to this input that I gained further clarification and insight on what to focus on and polish and develop, and what could be relegated to the backburner.

2. IMPLEMENTATION

The Implementation phase of the project, such as extensive user testing, final iterations, and writeup, were all stretched across roughly four months alongside other college modules and assignments. I think the time was used well, and I managed to keep steady momentum on both the development side and the written side without either one slipping too far behind the other.

Reflecting on the project, I am genuinely happy with where it ended up. It lines up closely with the goals I set out for myself at the very beginning, and the finished application captures exactly what I had in mind back when RaveSpin XR was originally being thought up.

3. SUPERVISOR ENGAGEMENT

Dr. Bryan Duggan was my supervisor for Rave-Spin XR. He has extensive expertise in music and the theory behind it, XR, the Meta platform, and Godot. Without his input I don't think Rave-Spin XR could have come as far as it has. To make sure things were progressing well and there was a constant and clear line of communication between myself and my supervisor, we had set up weekly online meetings.

Similar to a Scrum standup, we would catch up on all progress from the previous week and also addressed what issues were encountered, what observations were made and noting anything else of importance. I would also have the Git repository of the project be up to date so that the project could downloaded and tested live [64].

We would also address any accomplishments, new features, what could be improved or changed on said features, general feedback, and what to focus on for the next coming week(s).

A weekly log about the project was also kept. The logs can be found in section XIV "Appendix 6. Weekly Logs" *(page 200)*. They contain the status of the project at that point, such as what progress had been completed, future aims, and any issues encountered.

4. MANAGING RISK

A) VERSION CONTROL

To make sure code was clean and mostly devoid of bugs, I made sure that a Git repository was set up and hosted on GitHub from the project's initial onset [64]. If I encountered any huge loss of data, or created a new bug, thanks to Git, I was able to rollback any changes or see had changed between commits and deduce what the error was from that.

Once I was happy that the changes I had made were properly implemented and did not introduce bugs after extensive testing of all systems, I pushed my changes to the repository. It proved to be invaluable.

After the issues I encountered with the official GitHub Desktop tool *(see section IV.B.3 "GitHub Desktop" page 119)*, I had made an active effort to increase the frequency of my git commits as to avoid another huge loss of progress.

B) MODULARITY

As mentioned previously, I have more plans for this project. I want to add in new DJ controllers, MIDI players, and more. To make sure this is feasible, it was imperative that an Object-Oriented design pattern was followed. Any new features I want to add could be easily swapped in with very little hassle. For example, the Pioneer DDJ-FLX4 that is in Rave-Spin XR, is derived from a DJ Controller class.

At no point ever in the rest of the codebase, is there any hard reference to a specific item belonging to the FLX4 specifically. All data is queried to and from the DJ Controller parent class instead. This means that if we want to swap out the FLX4 at runtime to another DJ Controller object, such as DDJ-REV7 or a FLX10, it can be done so easily.

The same can be said for the Librebox UI and even the individual controls and buttons on the DJ controller. All calls and updates are made to and from a parent class, whose child objects can override and implement their own custom logic if need be *(Polymorphism)*.

This also has the added benefit of making code organisation, reusability, maintainability, and debugging an absolute breeze. Although this is ideal from a personal perspective, as any updates and future work I may want to do can be done so with no major system-breaking changes, it also means that as an open-source project, anyone else can more easily understand the codebase and make their own changes. They can spend more time coding and less time figuring out a jumbled mess of code.

C) USER SATISFACTION

To make sure users were never frustrated, user testing was done almost every week or 2 and any pain points mentioned were noted and addressed for next time *(see Section V.A.2, page 131, "User testing")*

D) SCOPE CREEP

To make sure the project remained concise and focused on its goals, I always made sure that the core features were prioritised first. Then any extra features would be added to the backlog and sorted based on their priority. Items of highest priority were always done first and then items of less importance were only included if there was enough time. This made sure that the project prioritised quality over quantity.

Having the project scope be tightly controlled helped hugely making sure that everything stuck to schedule and no overloading of tasks occurred.

E) LEGAL RISKS - COPYRIGHT

Under Irish law, I cannot say that the Pioneer DDJ-FLX4 is being accurately emulated in this project. When making any public facing statement or post, I must make sure to emphasise the point that the DJ Controller used in Rave-Spin XR is **based on** the Pioneer DDJ-FLX4 and is **not an emulation**. The layout of the controller in the final design has been modified as I did use images of the Pioneer DDJ-FLX4 in early prototypes. Efforts have been made to remove the Pioneer logo from final models as seen in the final model used in Figure IX-9, page 161.

F) LEGAL RISKS – GDPR AND USER DATA

Rave-Spin XR is a primarily offline application that does not require an internet connection. This is for multiple reasons, the main one being that if there is any reliance at all on an internet connection, considering where DJs typically perform gigs and the number of people and means of connection interference, access to a stable online connection is nearly never guaranteed.

The application runs offline, and because of this, everything is run and stored locally and uses music that is stored on the device's local storage. Personal user data is never accessed nor is it required.

Rave-Spin XR is an open-source tool that is available for anyone. It is down to the individual user to make sure that the application is used in a lawful manner.

D. CONCLUSIONS

By examining the functional requirements and testing them against the use cases in section III.D "Requirements analysis" *(page 82)*, we can see if the system has met the requirements.

Functional Requirements	Result
XR passthrough	Met
Accurate hand tracking	Partially met, see section IV.B.1 "Inaccurate Hand-Tracking" page 116
Import audio files from disk	Met
Audio metadata extraction	Partially met, see section IV.B.2 "Using External Compiled Binaries with Android/Meta" page 117
Load music into DJ Controller deck(s)	Met
Control playback of a song on a deck	Met
Adjust tempo of a song on a deck	Met
Adjust EQ and audio effects of a song on a deck	Met
Set and manage loop regions	Met
DDJ-FLX4 performance pad functionality	Met
Recording and saving of DJ sessions	Met
Localisation	Met
Jog wheel functionality	Met
Beat sync	Met
Downbeat alignment	Met

Overall, I am very happy with the planning that went into this project, and the execution of said plan. Although progress at the start of the project was not ideal, as issues with getting up to speed with Godot and learning its oddities mid-development, and Spotify changing its' API policy mid-development, and other unforeseen issues from developing on the Meta Headset platform, I am happy with how things progressed.

The milestones were met and the project finished way before due deadlines.

VI. CONCLUSIONS AND FUTURE WORK

The main core features of Rave-Spin XR have been implemented and work with a high degree of polish.

A. SUMMARY OF PROGRESS

1. PROTOTYPE - DRAFT

Compared to the initial prototypes of Rave-Spin XR, the project has indeed come a very long way. Many features such as the entirety of Librebox were introduced after the initial designs and after a large refactor to enforce the separation of responsibilities from the UI and the DJ Controller and enforcing a stricter OOP design approach.

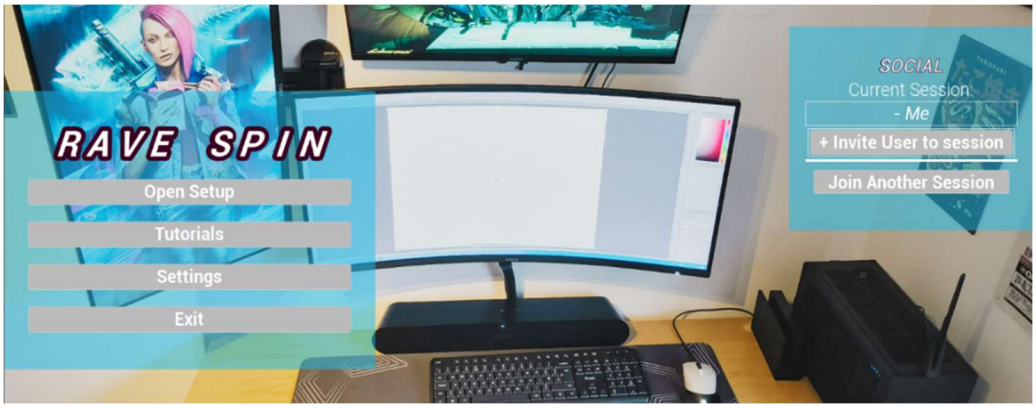


Figure VI-1 Initial UI Draft #1 of RaveSpin Menu

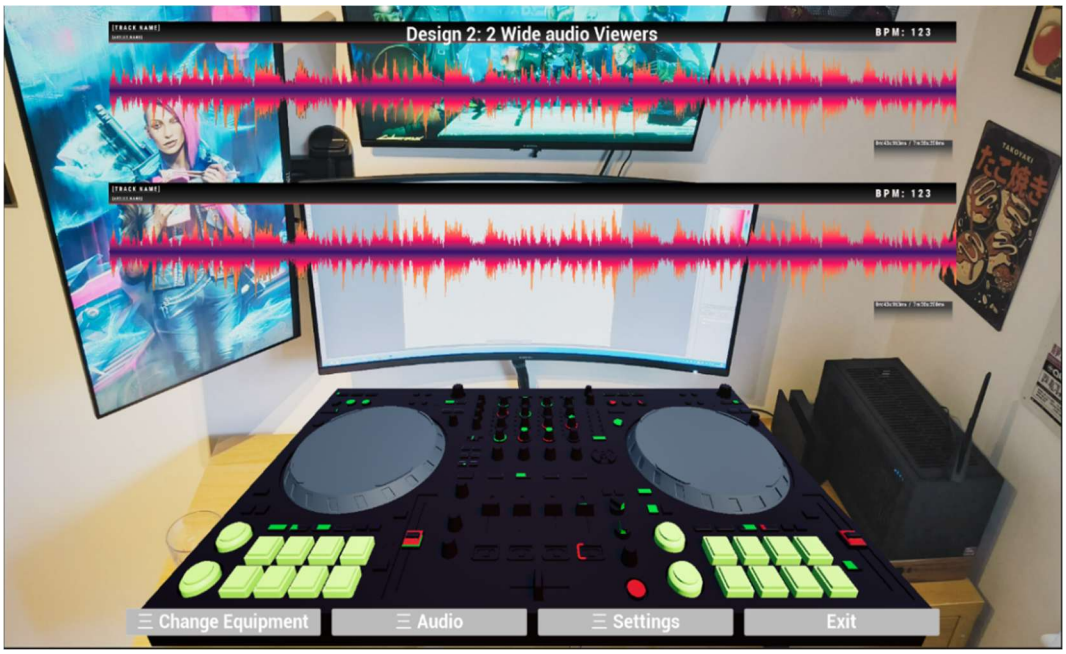


Figure VI-2 Initial UI Draft #1 of RaveSpin in-action.

2. PROTOTYPE – INTERIM

After the project was approved, work began immediately. The first couple of months were predominately me learning Godot. It was here that I ran into several issues and oddities of the software. I was also learning loads about DJs, music theory, and the underlying hardware. From the following images in Figure VI-3 and Figure VI-4, we can see an early version of Rave-Spin XR, all before Librebox was introduced and when functionality was heavily limited. The DJ Controller directly drove and made all changes to the HUD elements, rather than the HUD listening to signals given by DJ Controller. The Arena scene listened for changes and signals on the DJ Controller to drive changes such as Channel Volumes and Play/Pause state of songs.

There was a very inconsistent separation of concerns and poor application of OOP principles.

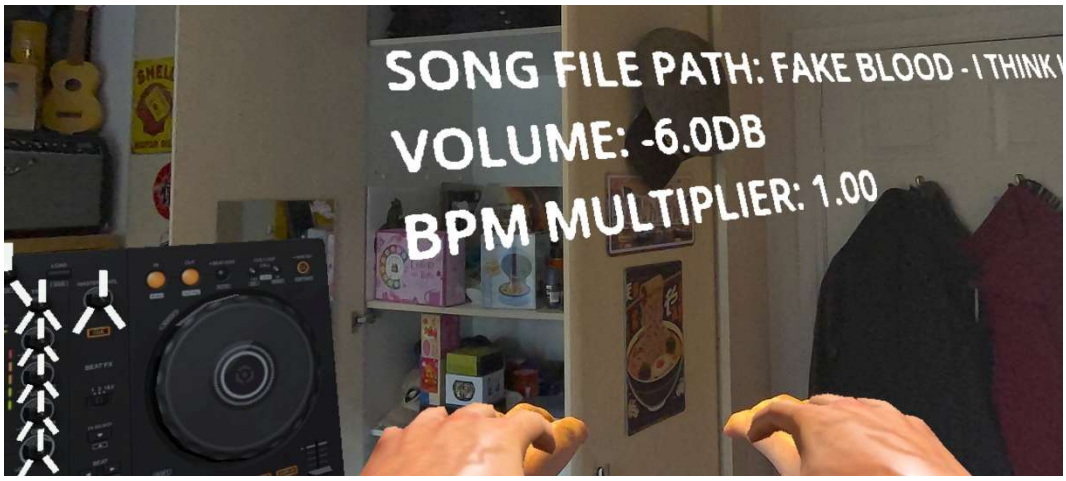


Figure VI-3 Screenshot of RaveSpin HUD before Librebox

Figure VI-4 Screenshot of RaveSpin Prototype in action, Only Pause/Play and Sliders worked.

B. LIBREBOX

1. BETTER METADATA EXTRACTION AND FFMPEG INTEGRATION

In its current form, Rave-Spin XR struggles to get the correct metadata from imported audio files. To do this I had wanted to use FFMPEG, but due to restrictions on external library use in applications on the Meta platform, much of the code fails silently. The project in its current state, tries to use FFMPEG first to gather metadata. If this fails, it will then fall to backup methods, such as manually parsing the raw data of the files to gather the correct tags regarding the artist's name, genres, album names, and try generating the waveform images *(only works for .WAV format files)*. Some of these functions only work for certain file formats, and only .WAV and .MP3 files have been tested.

To remedy this, I would like to edit the source code of the Godot engine to include FFMPEG functionality natively. This should remedy any issues with external libraries as the code will instead be called from Godot itself and thus shouldn't run into issues.

Once this is implemented, it would allow Rave-Spin XR to extract even more metadata from audio files such as images of the album art and allow for native generation of Waveform images, regardless of file type. This would allow easy parsing of .OGG music files, .FLAC, and almost any other media file.

2. TUTORIALS

In the Librebox scene, there is a node responsible for showing tooltips for the various components to the user. Although I'm happy with how the tooltips turned out, and they do a good job at explaining the role of the different components, I would like to expand it to make DJing even more accessible to newcomers.

As outlined in my original project brief, I would like to include tutorials with gamification elements. This can be seen with games like DJ Hero *(although this leans more heavily into gamification and less into educational)* and I think Rave-Spin XR would benefit hugely from this.

3. THIRD-PARTY MUSIC PLATFORM INTEGRATION

As mentioned in the original project aims and objects in the interim report, one of the goals of Rave-Spin XR was to include 3rd party music integration so that users could download songs from their accounts from Spotify, SoundCloud, YouTube Music, etc…

I initially tried to implement Spotify integration but due to changes to how the Spotify API can be used *(i.e. no downloading. Streaming audio only)*. I would like to explore the ability to have integration with 3rd party providers and allow users to download their songs easily. More research would be required to figure out what the different platforms allow with their API's, and most importantly, if they allow downloading of songs. As mentioned previously, Rave-Spin XR must be able to operate with no internet connection as DJs may frequently find themselves in busy environments which could prove making finding a stable connection difficult.

Much of the infrastructure is already in place to allow for integration of 3rd party platforms. We can see in the folder "**Art/Icon/Streaming_Platforms/**" folder that the art is already to go, and the functionality for getting the correct icon depending on what platform is implemented in ETrackOrigins.gd.

```gdscript
class_name ETrackOrigins

## Catalogue of where a Track can origininate from
## Maps platform labels to enums and logos.

## All known source platform labels used by metadata and UI
const Track_Origins_str : Array[StringName] = [
    "Other",

    # MAIN Local options
    "Local Storage",
    "Local Network (SMB)",         # Windows/NAS file share
    "Local Network (DLNA/UPnP)",   # Media server discovery/streaming on LAN
    "Direct URL (HTTP)",
    "FTP",                         # Legacy file transfer protocol
    "SFTP",                        # SSH File Transfer Protocol

    # major streaming
    "Spotify",
    "Apple Music",
    "Amazon Music",
    "YouTube Music",
    "Deezer",
    "TIDAL",
    "Qobuz",
    "Pandora",
    "iHeartRadio",
    "Napster",      # Apparantly it STILL exists??
    "iTunes Store",

    # cloud storage - If someone else makes a good enough API?
    "OneDrive",
    "Google Drive",
    "Dropbox",
    "iCloud Drive",
    "MEGA",
    "Nextcloud",

    # misc platforms
    "SoundCloud",
    "Bandcamp",
    "YouTube",
    "TikTok",
    "Instagram (Music)",
    "Mixcloud",
    "Audiomack"

    # Russia / bloc
    "Yandex Music",
    "VK Music",
    "Zvuk", # СберЗвук
    "Odnoklassniki Music",
    "MTS Music",
    "BOOM (VK/UMA)",

    # China
    "QQ Music",
    "NetEase Cloud Music", # 163 music
    "KuGou Music",
    "KuWo Music",
    "Migu Music",
    "Xiami (legacy)", #metadata only

    # Japan / Korea / Taiwan
    "Line Music",
    "AWA",
    "Recochoku",
    "KKBOX",
    "Melon",
    "Genie Music",
    "FLO",

    # India / South Asia
    "JioSaavn", # Jio + Saavn merge
    "Gaana",
    "Wynk Music",
    "Hungama Music", # metadata only

    # Vietnam + neighbours
    "MOOV",
    "JOOX",
    "Zing MP3",
    "NhacCuaTui",
    "Langit Musik",

    # Africa + Arabia
    "Anghami",
    "Boomplay",
    "Mdundo",

    # mostly classics
    "Naxos Music Library",
```

Figure VI-5 Screenshot of E_Track_Origins... what music platforms could a track come from?

```gdscript
## Returns UI logo texture path for a given source platform.
static func Get_Origin_Platform_Logo(platform : Track_Origins_enum) -> StringName:
    match platform:
        Track_Origins_enum.LOCAL_STORAGE:
            return "res://Art/Icon/Streaming_Platforms/T_LocalStorage.png"

        Track_Origins_enum.LOCAL_NETWORK_SMB:
            return "res://Art/Icon/Streaming_Platforms/T_LAN.png"
        Track_Origins_enum.LOCAL_NETWORK_DLNA_UPNP:
            return "res://Art/Icon/Streaming_Platforms/T_LAN.png"

        Track_Origins_enum.FTP:
            return "res://Art/Icon/Streaming_Platforms/T_FTP.png"
        Track_Origins_enum.SFTP:
            return "res://Art/Icon/Streaming_Platforms/T_FTP.png"

        Track_Origins_enum.SPOTIFY:
            return "res://Art/Icon/Streaming_Platforms/T_Spotify.png"

        Track_Origins_enum.APPLE_MUSIC:
            return "res://Art/Icon/Streaming_Platforms/T_AppleMusic.png"
        Track_Origins_enum.ITUNES_STORE:
            return "res://Art/Icon/Streaming_Platforms/T_iTunes.png"

        Track_Origins_enum.AMAZON_MUSIC:
            return "res://Art/Icon/Streaming_Platforms/T_AmazonMusic.png"

        Track_Origins_enum.YOUTUBE_MUSIC:
            return "res://Art/Icon/Streaming_Platforms/T_YouTube_Music.png"
        Track_Origins_enum.YOUTUBE:
            return "res://Art/Icon/Streaming_Platforms/T_YouTube_Regular.png"

        Track_Origins_enum.DEEZER:
            return "res://Art/Icon/Streaming_Platforms/T_Deezer.png"

        Track_Origins_enum.TIDAL:
            return "res://Art/Icon/Streaming_Platforms/T_Tidal.png"

        Track_Origins_enum.NAPSTER:
            return "res://Art/Icon/Streaming_Platforms/T_Napster.png"

        Track_Origins_enum.ONEDRIVE:
            return "res://Art/Icon/Streaming_Platforms/T_OneDrive.png"

        Track_Origins_enum.GOOGLE_DRIVE:
            return "res://Art/Icon/Streaming_Platforms/T_GoogleDrive.png"
```

Figure VI-6 Screenshot of code to get the logo for a given music platform.

From the screenshots of the code in Figure VI-5 and Figure VI-6, we can see that the functionality is there and is ready to be used to fetch what logos and art we may need for the different platforms. Thanks to the Strings to Enum conversion mentioned in Section IV.C.3, page 127 “GD-Script Strings to Enum”, the names of all these platforms were able to be quickly converted into Enums which would for easy metadata storage.

C. DJ CONTROLLER

1. ADDITIONAL CONTROLLER

The reason why I chose the Pioneer DDJ-FLX4 as the preferred controller for this project was because it is a fantastic controller that has many of the features expert DJs look for and is ideal for beginners who want to break into the artform and learn how to DJ. It's a fantastic middle-ground controller that does almost everything well enough.

I always wanted to have the option to switch out controllers at runtime so I made sure the DJ Controller parent class had all of the logic that one may need for a controller, and the FLX4 in Rave-Spin XR is an child object of that parent class.

This means that making a new potentially more specialised and more advanced controller, such as the Pioneer DDJ-FLX10 would be very straightforward.

All nodes in the scene use **DJ_Controller.Get_Instance()** for any API calls so the Get_Instance() could point to any DJ Controller we may want.

2. CUSTOM SAMPLES FOR PERFORMANCE PADS

In its current state, the performance pads *(When in sampler mode)* fire off pre-defined audio snippets. In future versions, I would like to have the user be able to set what audio will be used and save them so that they persist between sessions.

I chose generic audio samples from an online soundboard, and I believe that having the option to choose these samples at runtime would be ideal for everyone.

The logic for selecting audio files from local disk storage is already implemented for importing new tracks so having the same functionality for selecting new audio samples would be no issue.

VII. PROJECT CONCLUSION

Rave-Spin XR as it currently stands, has come a long way since its first inception. The project development has stuck to schedule and was able to be completed ahead of project deadlines.

Rave-Spin XR successfully accomplishes its main goals and can be used in casual mixing environments, or in gigs as well. Performance is consistent, and even though the Meta Quest 3s is not the most powerful XR headset on the market, it still runs perfectly smooth on the underpowered hardware.

Beginner DJs who have an XR Headset can easily download the application and start mixing straight away thanks to the informative tooltips and easy to follow controls. Users who may not have enough money to afford advanced DJ controllers or other equipment can use Rave-Spin XR to get started as a much more affordable but still viable alternative.

Users from other countries can also use the app thanks to localisation support, and if developers want to localise the project into their own language, all that's required is editing a large .CSV file in MS Excel.

More experienced DJs can travel the world without needing to bring their equipment with them everywhere, ideally making logistics much easier.

Although Rave-Spin XR still has many more features it can include, and more polish could be used for other areas, as the subtitle says in page 1 of this report, the project has accomplished its goal of being "A PORTABLE XR COMPANION APP FOR DJS".

I am incredibly proud of the work displayed and am grateful to the invaluable feedback to all the people who participated in the user testing, and to my Supervisor Dr. Bryan Duggan for his insight and making sure the project stayed polished and concise.

VIII. REFERENCES

[1] L. Tremosa, 'Beyond AR vs. VR: What is the Difference between AR vs. MR vs. VR vs. XR?', IxDF - Interaction Design Foundation. Accessed: Apr. 16, 2026. [Online]. Available: https://ixdf.org/literature/article/beyond-ar-vs-vr-what-is-the-difference-between-ar-vs-mr-vs-vr-vs-xr

[2] 'RaveSpin — DJ in XR'. Accessed: Apr. 16, 2026. [Online]. Available: https://marc-rene.github.io/RaveSpin-Documentation/

[3] M. Conrad, D. Kablitz, and S. Schumann, 'Learning effectiveness of immersive virtual reality in education and training: A systematic review of findings', *Comput. Educ. X Real.*, vol. 4, p. 100053, Jan. 2024, doi: 10.1016/j.cexr.2024.100053.

[4] 'Pioneer DJ DDJ-FLX10: All specifications & features', Pioneer DJ. Accessed: Nov. 23, 2025. [Online]. Available: https://www.pioneerdj.com/en/product/controller/ddj-flx10/black/specifications/

[5] 'Pioneer DJ DDJ-FLX10 Controller', Digital DJ Tips. Accessed: Apr. 16, 2026. [Online]. Available: https://www.digitaldjtips.com/reviews/pioneer-dj-ddj-flx10/

[6] Recordcase, 'Pioneer DJ DDJ-FLX10 in review', Recordcase.de. Accessed: Apr. 16, 2026. [Online]. Available: https://www.recordcase.de/en/pioneer-dj-ddj-flx10-in-review

[7] H. H. published, '"One of the FLX10's greatest strengths is its layout and ease of use": Pioneer DDJ-FLX10 review', MusicRadar. Accessed: Apr. 16, 2026. [Online]. Available: https://www.musicradar.com/reviews/pioneer-ddj-flx10-review

[8] H. H. published, '"A strong contender for the best beginner's DJ controller on the market right now": Pioneer DDJ-FLX4 review', MusicRadar. Accessed: Nov. 23, 2025. [Online]. Available: https://www.musicradar.com/reviews/pioneer-ddj-flx4-review

[9] 'Tribe XR partners with AlphaTheta Corporation to bring Pioneer DJ – the world's leading DJ hardware and software brand – to virtual reality - News - Pioneer DJ News', Pioneer DJ. Accessed: Oct. 12, 2025. [Online]. Available: https://www.pioneerdj.com/en-us/news/2021/tribe-xr-partners-with-alphatheta-corporation/?utm_source=chatgpt.com/

[10] 'Tribe XR | DJ Academy on Steam'. Accessed: Oct. 12, 2025. [Online]. Available: https://store.steampowered.com/app/877850/Tribe_XR__DJ_Academy/

[11] 'Ableton Live in VR lets you play as a disembodied Daft Punk head', CDM Create Digital Music. Accessed: Oct. 12, 2025. [Online]. Available: https://cdm.link/newswires/ableton-live-vr-lets-play-disembodied-daft-punk-head/

[12] 'AliveInVR on Steam'. Accessed: Oct. 12, 2025. [Online]. Available: https://store.steampowered.com/app/674180/AliveInVR/

[13] 'Vinyl Reality - DJ in VR', Vinyl Reality. Accessed: Oct. 12, 2025. [Online]. Available: https://www.vinyl-reality.com/

[14] 'Vinyl Reality - DJ in VR on Steam'. Accessed: Oct. 12, 2025. [Online]. Available: https://store.steampowered.com/app/642770/Vinyl_Reality__DJ_in_VR/

[15] ‘Algoriddim djay Pro 5’, Digital DJ Tips. Accessed: Oct. 12, 2025. [Online]. Available: https://www.digitaldjtips.com/reviews/algoriddim-djay-pro-5/

[16] ‘Algoriddim DJay Pro review & performance tests’, DeeJay Plaza. Accessed: Oct. 12, 2025. [Online]. Available: https://www.deejayplaza.com/en/articles/djay-pro-review

[17] J. Hartley, ‘Algoriddim djay Pro Review - We Are Crossfader’, We Are Crossfader - Learn How To DJ Online. Accessed: Oct. 12, 2025. [Online]. Available: https://wearecrossfader.co.uk/blog/algoriddim-djay-pro-review/

[18] ‘OpenXR - High-performance access to AR and VR —collectively known as XR— platforms and devices’, The Khronos Group. Accessed: Apr. 13, 2026. [Online]. Available: https://www.khronos.org/openxr/

[19] ‘Global XR (AR & VR Headsets) Market Share: Quarterly’. Accessed: Apr. 13, 2026. [Online]. Available: https://counterpointresearch.com/en/insights/global-xr-ar-vr-headsets-market-share-quarterly

[20] T. P. published, ‘Meta Quest 3S review: The best VR headset for the money’, Tom’s Guide. Accessed: Apr. 16, 2026. [Online]. Available: https://www.tomsguide.com/computing/vr-ar/meta-quest-3s-review

[21] marc-rene, ‘Commits · marc-rene/Segfault-Game-Engine’, GitHub. Accessed: Apr. 16, 2026. [Online]. Available: https://github.com/marc-rene/Segfault-Game-Engine

[22] ‘Forward Shading Renderer’. Accessed: Apr. 13, 2026. [Online]. Available: https://developers.meta.com/horizon/documentation/unreal/unreal-forward-renderer/

[23] ‘Behavior changes: apps targeting API 29+’, Android Developers. Accessed: Apr. 14, 2026. [Online]. Available: https://developer.android.com/about/versions/10/behavior-changes-10

[24] ‘Unreal on Android’, Android Developers. Accessed: Apr. 15, 2026. [Online]. Available: https://developer.android.com/games/engines/unreal/unreal-on-android

[25] ‘Make epic mobile games with Unreal Engine’, Unreal Engine. Accessed: Apr. 15, 2026. [Online]. Available: https://www.unrealengine.com/uses/mobile-games

[26] ‘Unreal Engine EULA’, Unreal Engine. Accessed: Apr. 16, 2026. [Online]. Available: https://www.unrealengine.com/eula/unreal

[27] ‘UGCExample/Documentation/QuickStart.md at release · EpicGames/UGCExample’, GitHub. Accessed: Apr. 15, 2026. [Online]. Available: https://github.com/EpicGames/UGCExample/blob/release/Documentation/QuickStart.md

[28] ‘New example project and plugin for mod support released’, Unreal Engine. Accessed: Apr. 15, 2026. [Online]. Available: https://www.unrealengine.com/blog/new-example-project-and-plugin-for-mod-support-released

[29] ‘How do I access read-only source code?’, Unity Support Help Center. Accessed: Apr. 16, 2026. [Online]. Available: https://support.unity.com/hc/en-us/articles/10033332561812-How-do-I-access-read-only-source-code

[30] ‘Audio has TOO MUCH latency. - Unity Engine’, Unity Discussions. Accessed: Apr. 13, 2026. [Online]. Available: https://discussions.unity.com/t/audio-has-too-much-latency/679436

[31] ‘I am getting a lot of sound latency when developing my game for Android. How can I get my sounds to play immediately?’, Unity Support Help Center. Accessed: Apr. 13, 2026. [Online]. Available: https://support.unity.com/hc/en-us/articles/206116316-I-am-getting-a-lot-of-sound-latency-when-developing-my-game-for-Android-How-can-I-get-my-sounds-to-play-immediately

[32] ‘Flax Roadmap | Trello’. Accessed: Apr. 13, 2026. [Online]. Available: https://trello.com/b/NQjLXRCP/flax-roadmap

[33] ‘Introducing XR tools’, Godot Engine documentation. Accessed: Apr. 16, 2026. [Online]. Available: https://docs.godotengine.org/en/latest/tutorials/xr/introducing_xr_tools.html

[34] ‘RhythmNotifier - Sync Your Game to the Beat of the Music (Sound & Audio) - Godot Asset Library’. Accessed: Apr. 16, 2026. [Online]. Available: https://godotengine.org/asset-library/asset/3417

[35] ‘Godot XR Hand Pose Detector - Godot Asset Library’. Accessed: Apr. 16, 2026. [Online]. Available: https://godotengine.org/asset-library/asset/3097

[36] E. Obedkov, ‘Devs fume over Runtime per-install fees: “The breach of trust is so insane that it’s impossible to stay with Unity”’, Game World Observer. Accessed: Apr. 13, 2026. [Online]. Available: https://gameworldobserver.com/2023/09/13/unity-runtime-fee-devs-reaction-switching-to-other-engines

[37] ‘GDScript reference’, Godot Engine documentation. Accessed: Apr. 16, 2026. [Online]. Available: https://docs.godotengine.org/en/stable/tutorials/scripting/gdscript/gdscript_basics.html

[38] KhronosGroup, ‘glTF/specification/2.0 at main · KhronosGroup/glTF’, GitHub. Accessed: Apr. 15, 2026. [Online]. Available: https://github.com/KhronosGroup/glTF/tree/main/specification/2.0

[39] ‘What is glTF: A Complete Guide’. Accessed: Apr. 14, 2026. [Online]. Available: https://www.vividworks.com/blog/what-is-gltf-a-complete-guide

[40] ‘ffprobe Documentation’. Accessed: Apr. 13, 2026. [Online]. Available: https://ffmpeg.org/ffprobe.html

[41] DJ.Studio, ‘Mixing dB Levels for DJs - Understanding Gain Staging’, DJ.Studio | Mixing dB Levels for DJs - Understanding Gain Staging. Accessed: Nov. 23, 2025. [Online]. Available: https://dj.studio/blog/mixing-db-levels

[42] R. Dejaegher, ‘DJ Audio Routing 303: Analogue Mixers, Guitar Pedals, and External Soundcards’, DJ TechTools. Accessed: Nov. 23, 2025. [Online]. Available: https://djtechtools.com/2015/06/28/audio-routing-101-analog-mixers-guitar-pedals-and-external-soundcards/

[43] T. and How-To’s and N. and Reviews, ‘How does a mixer work? | The Basics’, reverb.com. Accessed: Nov. 23, 2025. [Online]. Available: https://reverb.com/fr/news/how-does-a-mixer-work-the-basics

[44] J. Hartley, ‘The Inputs and Outputs on a DJ mixer - We Are Crossfader’, We Are Crossfader - Learn How To DJ Online. Accessed: Nov. 23, 2025. [Online]. Available: https://wearecrossfader.co.uk/blog/the-inputs-and-outputs-on-a-dj-mixer/

[45] P. Morse, ‘The 7 Best DJ Controllers For 2025’, Digital DJ Tips. Accessed: Oct. 08, 2025. [Online]. Available: https://www.digitaldjtips.com/best-dj-controllers-2025/

[46] ‘Pioneer DDJ-FLX4 — Mixxx User Manual’. Accessed: Apr. 14, 2026. [Online]. Available: https://manual.mixxx.org/2.5/en/hardware/controllers/pioneer_ddj_flx4

[47] ‘How to use your Pioneer DJ DDJ-FLX4 controller (Instruction Manual) - AlphaTheta Help Center’. Accessed: Apr. 14, 2026. [Online]. Available: https://support.alphatheta.com/en-US/articles/12267724407961

[48] ‘FMOD’, *Wikipedia*. Mar. 28, 2026. Accessed: Apr. 14, 2026. [Online]. Available: https://en.wikipedia.org/w/index.php?title=FMOD&oldid=1345884699

[49] ‘FMOD Studio’. Accessed: Apr. 17, 2026. [Online]. Available: https://www.fmod.com/studio

[50] ‘FMOD - Welcome to the FMOD Engine’. Accessed: Apr. 17, 2026. [Online]. Available: https://www.fmod.com/docs/2.03/api/welcome.html

[51] ‘MusicBrainz - the open music encyclopedia’. Accessed: Apr. 14, 2026. [Online]. Available: https://musicbrainz.org/

[52] ‘Manifesto for Agile Software Development’. Accessed: Apr. 14, 2026. [Online]. Available: https://agilemanifesto.org/iso/en/manifesto.html

[53] ‘Rapid Application Development Model (RAD) - Software Engineering’, GeeksforGeeks. Accessed: Apr. 14, 2026. [Online]. Available: https://www.geeksforgeeks.org/software-engineering/software-engineering-rapid-application-development-model-rad/

[54] ‘Rapid Application Development (RAD): Complete 2026 Guide’. Accessed: Apr. 14, 2026. [Online]. Available: https://www.weweb.io/blog/rapid-application-development-rad-complete-guide

[55] S. Stiner, ‘Rapid Application Development (RAD): A Smart, Quick And Valuable Process For Software Developers’, Forbes. Accessed: Apr. 14, 2026. [Online]. Available: https://www.forbes.com/sites/forbestechcouncil/2016/08/24/rapid-application-development-rad-a-smart-quick-and-valuable-process-for-software-developers/

[56] ‘Feature-Driven Development vs. Test-Driven Development’, LaunchDarkly. Accessed: Apr. 14, 2026. [Online]. Available: https://launchdarkly.com/blog/feature-driven-development-versus-test-driven-development/

[57] S. O’Connor, ‘Feature driven development (FDD): the complete guide for 2026’, monday.com Blog. Accessed: Apr. 14, 2026. [Online]. Available: https://monday.com/blog/rnd/feature-driven-development-fdd/

[58] ‘Home - ID3.org’. Accessed: Apr. 14, 2026. [Online]. Available: https://id3.org/

[59] P. Marks, ‘Why Do Photos Look Grainy at Night? A Pro Photographer’s Guide to Fixing Digital Noise’, Medium. Accessed: Apr. 14, 2026. [Online]. Available: https://medium.com/@kashafshahid467/why-do-photos-look-grainy-at-night-a-pro-photographers-guide-to-fixing-digital-noise-01e49cf16956

[60] ‘Update on Developer Access and Platform Security | Spotify for Developers’. Accessed: Apr. 13, 2026. [Online]. Available: https://developer.spotify.com/blog/2026-02-06-update-on-developer-access-and-platform-security

[61] ‘Likert Scale Questionnaire: Examples & Analysis’. Accessed: Apr. 15, 2026. [Online]. Available: https://www.simplypsychology.org/likert-scale.html

[62] ‘DDJ-REV7 - Scratch-style 2-channel professional DJ controller for multiple DJ applications’, Pioneer DJ. Accessed: Apr. 14, 2026. [Online]. Available: https://www.pioneerdj.com/en/product/dj-controllers/ddj-rev7/

[63] ‘RaveSpin — DJ in XR’. Accessed: Apr. 15, 2026. [Online]. Available: https://marc-rene.github.io/RaveSpin-Documentation/

[64] marc-rene, *marc-rene/RaveSpin*. (Apr. 13, 2026). GDScript. Accessed: Apr. 16, 2026. [Online]. Available: https://github.com/marc-rene/RaveSpin

IX. APPENDIX 1. DDJ-FLX4 RENDITIONS

A. SOURCE IMAGES

Figure IX-1 Top-Down View of Pioneer DDJ-FLX4

Figure IX-2 Orthographic side view of Pioneer DDJ-FLX4

B. VERSION 1

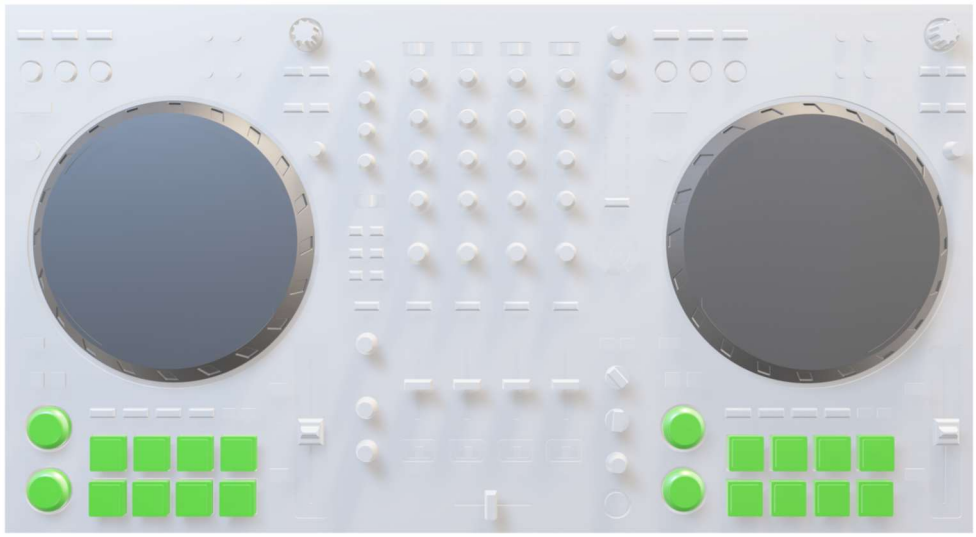


Figure IX-3 Top-Down View of version 1 of DJ Controller (Used a DDJ-FLX6 model as a stopgap)

Figure IX-4 Orthographic side view of version 1 of DJ Controller (Used a DDJ-FLX6 model as a stopgap)

C. VERSION 2

Figure IX-5 Top-Down View of version 2 of DJ Controller (Just learnt how to export Textures)

Figure IX-6 Orthographic side view of version 2 of DJ Controller (Notice how there's no buttons or bumps. It is completely flat. It's just a plain texture)

D. VERSION 2.1

Figure IX-7 Top-down view of heavily improved DDJ-FLX4 model.

Figure IX-8 Orthographic side view of heavily improved DDJ-FLX4 model (Notice how buttons, Jog wheels, etc... all exist now. It's no longer just a flat texture)

E. VERSION 3 – FINAL VERSION

Figure IX-9 Top-down view of DDJ-FLX4 model that's used in RaveSpin XR. Redundant items have been removed; controls have been spaced better. Easier to access and use. Less risk of triggering other controls. Reduced texture size also increases performance.

X. APPENDIX 2. LOCALISED CONTROLS

A. SCREENSHOT OF LOCALISED CONTROLS ON DDJ-FLX4

1. ENGLISH VERSION

2. IRISH VERSION

Figure X-1 Picture of Controls on In-game FLX4 model in action (Irish language selected)

3. MANDARIN VERSION

Figure X-2 Picture of Controls on In-game FLX4 model in action (Mandarin language selected)

XI. APPENDIX 3. CODE OUTPUTS

A. OUTPUT OF GENERATE WAVEFORMS.PY

1. COPY + PASTED OUTPUT

- PS C:\Users\cesar> python -u "c:\Other\Git\RaveSpin\Assets\Tools\generate_waveforms.py"
- Generating waveform for C:\Other\Git\RaveSpin\Godot\Music\Song Data\Alla Pugacheva\Поднимись над суетой\Audio\Звёздное лето.wav -> C:\Other\Git\RaveSpin\Godot\Music\Song Data\Alla Pugacheva\Поднимись над суетой\Audio\Звёздное лето_WAVEFORM.png (size 4853x240)

- Now open the project in Godot so it scans and imports the new waveform PNGs.
- This script will wait until all corresponding .import files are created.

- Detected import file: C:\Other\Git\RaveSpin\Godot\Music\Song Data\Death Grips\Exmilitary\Audio\Death Grips - Exmilitary - 9 - 5D_WAVEFORM.png.import
- Detected import file: C:\Other\Git\RaveSpin\Godot\Music\Song Data\Daft Punk\Discovery\Audio\Daft Punk - Harder, Better, Faster, Stronger (Official Audio)_WAVEFORM.png.import
- Detected import file: C:\Other\Git\RaveSpin\Godot\Music\Song Data\Death Grips\Exmilitary\Audio\Death Grips - Exmilitary - 1 - Beware_WAVEFORM.png.import
- Detected import file: C:\Other\Git\RaveSpin\Godot\Music\Song Data\Kino\МОЛНИИ ИНДРЫ\Audio\Ты мог быть героем (2025)_WAVEFORM.png.import
- Detected import file: C:\Other\Git\RaveSpin\Godot\Music\Song Data\Crusy\Love is the Answer\Audio\Crusy - Love Is The Answer - 126bpm_WAVEFORM.png.import
- Detected import file: C:\Other\Git\RaveSpin\Godot\Music\Song Data\Kino\МОЛНИИ ИНДРЫ\Audio\Звезда (2025) - Kino_WAVEFORM.png.import
- Detected import file: C:\Other\Git\RaveSpin\Godot\Music\Song Data\Daft Punk\Homework\Audio\Daft Punk - Around the World (Official Audio)_WAVEFORM.png.import
- Detected import file: C:\Other\Git\RaveSpin\Godot\Music\Song Data\Daft Punk\Discovery\Audio\Daft Punk - One More Time (Official Audio)_WAVEFORM.png.import
- Detected import file: C:\Other\Git\RaveSpin\Godot\Music\Song Data\Fake Blood\Fix your Accent\Audio\Fake Blood - I Think I Like It - 126bpm_WAVEFORM.png.import
- Detected import file: C:\Other\Git\RaveSpin\Godot\Music\Song Data\Test\109bpm Test - Synth (C-E)_WAVEFORM.png.import
- Detected import file: C:\Other\Git\RaveSpin\Godot\Music\Song Data\Test\145bpm Test - Synth (C-E)_WAVEFORM.png.import
- Detected import file: C:\Other\Git\RaveSpin\Godot\Music\Song Data\Alla Pugacheva\Поднимись над суетой\Audio\Звёздное лето_WAVEFORM.png.import

2. TERMINAL OUTPUT

```
PS C:\Users\cesar> python -u "c:\Other\Git\RaveSpin\Assets\Tools\generate_waveforms.py"
Generating waveform for C:\Other\Git\RaveSpin\Godot\Music\Song Data\Alla Pugacheva\Поднимись над суетой\Audio\Звёздное лето.wav -> C:\Other\Git\RaveSpin\Godot\Music\Song Data\
ugacheva\Поднимись над суетой\Audio\Звёздное лето_WAVEFORM.png (size 4853x240)
Generating waveform for C:\Other\Git\RaveSpin\Godot\Music\Song Data\Crusy\Love is the Answer\Audio\Crusy - Love Is The Answer - 126bpm.wav -> C:\Other\Git\RaveSpin\Godot\Music
Data\Crusy\Love is the Answer\Audio\Crusy - Love Is The Answer - 126bpm_WAVEFORM.png (size 4212x240)
Generating waveform for C:\Other\Git\RaveSpin\Godot\Music\Song Data\Daft Punk\Discovery\Audio\Daft Punk - Harder, Better, Faster, Stronger (Official Audio).wav -> C:\Other\Git
pin\Godot\Music\Song Data\Daft Punk\Discovery\Audio\Daft Punk - Harder, Better, Faster, Stronger (Official Audio)_WAVEFORM.png (size 3836x240)
Generating waveform for C:\Other\Git\RaveSpin\Godot\Music\Song Data\Daft Punk\Discovery\Audio\Daft Punk - One More Time (Official Audio).wav -> C:\Other\Git\RaveSpin\Godot\Mus
g Data\Daft Punk\Discovery\Audio\Daft Punk - One More Time (Official Audio)_WAVEFORM.png (size 5469x240)
Generating waveform for C:\Other\Git\RaveSpin\Godot\Music\Song Data\Daft Punk\Homework\Audio\Daft Punk - Around the World (Official Audio).wav -> C:\Other\Git\RaveSpin\Godot\M
ong Data\Daft Punk\Homework\Audio\Daft Punk - Around the World (Official Audio)_WAVEFORM.png (size 7332x240)
Generating waveform for C:\Other\Git\RaveSpin\Godot\Music\Song Data\Death Grips\Exmilitary\Audio\Death Grips - Exmilitary - 1 - Beware.wav -> C:\Other\Git\RaveSpin\Godot\Music
Data\Death Grips\Exmilitary\Audio\Death Grips - Exmilitary - 1 - Beware_WAVEFORM.png (size 6017x240)
Generating waveform for C:\Other\Git\RaveSpin\Godot\Music\Song Data\Death Grips\Exmilitary\Audio\Death Grips - Exmilitary - 9 - 5D.wav -> C:\Other\Git\RaveSpin\Godot\Music\Son
\Death Grips\Exmilitary\Audio\Death Grips - Exmilitary - 9 - 5D_WAVEFORM.png (size 738x240)
Generating waveform for C:\Other\Git\RaveSpin\Godot\Music\Song Data\Fake Blood\Fix your Accent\Audio\Fake Blood - I Think I Like It - 126bpm.wav -> C:\Other\Git\RaveSpin\Godot
\Song Data\Fake Blood\Fix your Accent\Audio\Fake Blood - I Think I Like It - 126bpm_WAVEFORM.png (size 5823x240)
Generating waveform for C:\Other\Git\RaveSpin\Godot\Music\Song Data\Kino\МОЛНИИ ИНДРЫ\Audio\Звезда (2025) - Kino.wav -> C:\Other\Git\RaveSpin\Godot\Music\Song Data\Kino\МОЛНИИ
\Audio\Звезда (2025) - Kino_WAVEFORM.png (size 4448x240)
Generating waveform for C:\Other\Git\RaveSpin\Godot\Music\Song Data\Kino\МОЛНИИ ИНДРЫ\Audio\Ты мог быть героем (2025).wav -> C:\Other\Git\RaveSpin\Godot\Music\Song Data\Kino\M
ИНДРЫ\Audio\Ты мог быть героем (2025)_WAVEFORM.png (size 4071x240)
Generating waveform for C:\Other\Git\RaveSpin\Godot\Music\Song Data\Kino\МОЛНИИ ИНДРЫ\Audio\Фильмы (2025) - Kino.wav -> C:\Other\Git\RaveSpin\Godot\Music\Song Data\Kino\МОЛНИИ
\Audio\Фильмы (2025) - Kino_WAVEFORM.png (size 3817x240)
Generating waveform for C:\Other\Git\RaveSpin\Godot\Music\Song Data\Test\109bpm Test - Synth (C-E).wav -> C:\Other\Git\RaveSpin\Godot\Music\Song Data\Test\109bpm Test - Synth
WAVEFORM.png (size 2255x240)
Generating waveform for C:\Other\Git\RaveSpin\Godot\Music\Song Data\Test\145bpm Test - Synth (C-E).wav -> C:\Other\Git\RaveSpin\Godot\Music\Song Data\Test\145bpm Test - Synth
WAVEFORM.png (size 1695x240)

Now open the project in Godot so it scans and imports the new waveform PNGs.
This script will wait until all corresponding .import files are created.

Detected import file: C:\Other\Git\RaveSpin\Godot\Music\Song Data\Death Grips\Exmilitary\Audio\Death Grips - Exmilitary - 9 - 5D_WAVEFORM.png.import
Detected import file: C:\Other\Git\RaveSpin\Godot\Music\Song Data\Daft Punk\Discovery\Audio\Daft Punk - Harder, Better, Faster, Stronger (Official Audio)_WAVEFORM.png.import
Detected import file: C:\Other\Git\RaveSpin\Godot\Music\Song Data\Death Grips\Exmilitary\Audio\Death Grips - Exmilitary - 1 - Beware_WAVEFORM.png.import
Detected import file: C:\Other\Git\RaveSpin\Godot\Music\Song Data\Kino\МОЛНИИ ИНДРЫ\Audio\Ты мог быть героем (2025)_WAVEFORM.png.import
Detected import file: C:\Other\Git\RaveSpin\Godot\Music\Song Data\Crusy\Love is the Answer\Audio\Crusy - Love Is The Answer - 126bpm_WAVEFORM.png.import
Detected import file: C:\Other\Git\RaveSpin\Godot\Music\Song Data\Kino\МОЛНИИ ИНДРЫ\Audio\Звезда (2025) - Kino_WAVEFORM.png.import
Detected import file: C:\Other\Git\RaveSpin\Godot\Music\Song Data\Daft Punk\Homework\Audio\Daft Punk - Around the World (Official Audio)_WAVEFORM.png.import
Detected import file: C:\Other\Git\RaveSpin\Godot\Music\Song Data\Daft Punk\Discovery\Audio\Daft Punk - One More Time (Official Audio)_WAVEFORM.png.import
Detected import file: C:\Other\Git\RaveSpin\Godot\Music\Song Data\Fake Blood\Fix your Accent\Audio\Fake Blood - I Think I Like It - 126bpm_WAVEFORM.png.import
Detected import file: C:\Other\Git\RaveSpin\Godot\Music\Song Data\Test\109bpm Test - Synth (C-E)_WAVEFORM.png.import
Detected import file: C:\Other\Git\RaveSpin\Godot\Music\Song Data\Test\145bpm Test - Synth (C-E)_WAVEFORM.png.import
Detected import file: C:\Other\Git\RaveSpin\Godot\Music\Song Data\Alla Pugacheva\Поднимись над суетой\Audio\Звёздное лето_WAVEFORM.png.import
Detected import file: C:\Other\Git\RaveSpin\Godot\Music\Song Data\Kino\МОЛНИИ ИНДРЫ\Audio\Фильмы (2025) - Kino_WAVEFORM.png.import
Patched import settings for C:\Other\Git\RaveSpin\Godot\Music\Song Data\Alla Pugacheva\Поднимись над суетой\Audio\Звёздное лето_WAVEFORM.png.import
Patched import settings for C:\Other\Git\RaveSpin\Godot\Music\Song Data\Crusy\Love is the Answer\Audio\Crusy - Love Is The Answer - 126bpm_WAVEFORM.png.import
Patched import settings for C:\Other\Git\RaveSpin\Godot\Music\Song Data\Daft Punk\Discovery\Audio\Daft Punk - Harder, Better, Faster, Stronger (Official Audio)_WAVEFORM.png.im
Patched import settings for C:\Other\Git\RaveSpin\Godot\Music\Song Data\Daft Punk\Discovery\Audio\Daft Punk - One More Time (Official Audio)_WAVEFORM.png.import
Patched import settings for C:\Other\Git\RaveSpin\Godot\Music\Song Data\Daft Punk\Homework\Audio\Daft Punk - Around the World (Official Audio)_WAVEFORM.png.import
Patched import settings for C:\Other\Git\RaveSpin\Godot\Music\Song Data\Death Grips\Exmilitary\Audio\Death Grips - Exmilitary - 1 - Beware_WAVEFORM.png.import
Patched import settings for C:\Other\Git\RaveSpin\Godot\Music\Song Data\Death Grips\Exmilitary\Audio\Death Grips - Exmilitary - 9 - 5D_WAVEFORM.png.import
Patched import settings for C:\Other\Git\RaveSpin\Godot\Music\Song Data\Fake Blood\Fix your Accent\Audio\Fake Blood - I Think I Like It - 126bpm_WAVEFORM.png.import
Patched import settings for C:\Other\Git\RaveSpin\Godot\Music\Song Data\Kino\МОЛНИИ ИНДРЫ\Audio\Звезда (2025) - Kino_WAVEFORM.png.import
Patched import settings for C:\Other\Git\RaveSpin\Godot\Music\Song Data\Kino\МОЛНИИ ИНДРЫ\Audio\Ты мог быть героем (2025)_WAVEFORM.png.import
Patched import settings for C:\Other\Git\RaveSpin\Godot\Music\Song Data\Kino\МОЛНИИ ИНДРЫ\Audio\Фильмы (2025) - Kino_WAVEFORM.png.import
Patched import settings for C:\Other\Git\RaveSpin\Godot\Music\Song Data\Test\109bpm Test - Synth (C-E)_WAVEFORM.png.import
Patched import settings for C:\Other\Git\RaveSpin\Godot\Music\Song Data\Test\145bpm Test - Synth (C-E)_WAVEFORM.png.import
```

XII. APPENDIX 4. GENERATIVE AI PROMPTS

Generative AI was used for this project to assist with research and to automate data entry such as section XII.A.3 “Chat 3: Listing of all music scales” on page 173, and for translations of text to other languages as seen in section XII.A.8 “Chat 8. Generating Translations” on page 183.

A. CHATGPT

1. CHAT 1: FLX4 POPULARITY RESEARCH

Prompt
I said The FLX4 is possibly the most common DJ controller board out there. It does almost everything a DJ could want and it’s fantastic for beginners to. It would be like the Ford Fiesta of the DJ world. It does 90% of everything you need, and if you know how to use this board, you’ll likely have no issue with most other Controllers. Thankfully there’s loads of resources out there about how this board works exactly. Please check Appendix E for the diagram of how the controller is laid out Can you give me links and references which back this up?
Response
Short answer: yes, you *can* back that paragraph up, but you should probably tone down “most common” to “one of the most popular beginner controllers” so you’re not over-claiming. Here’s a set of solid references you can drop into your report.
1. Popular / “go-to” beginner controller These support the idea that the DDJ-FLX4 is *very* widely recommended as a starter controller and covers most of what a new DJ needs. • **MusicRadar – Pioneer DDJ-FLX4 review** Calls it “a strong contender for the best beginner’s DJ controller on the market right now”, and positions it as the successor to the hugely popular DDJ-400. • **MusicRadar – Best beginner DJ controllers 2025** In their round-up of beginner controllers, they describe the DDJ-FLX4 as “one of the most versatile beginner DJ controllers on the market”, with enough hardware and software features for novices to develop their skills. • **Recordcase – “Which DJ controller is best for beginners?”** They explicitly recommend the DDJ-FLX4 as *their* clear choice in the sub-€350 price range and call it the successor to the “extremely popular” DDJ-400, noting that it “sets the new standard for entry-level DJ controllers”. • **Future Sound Academy – DDJ-FLX4 Review (2025)** Describes the FLX4 as a “smart, flexible, and beginner-friendly controller that lowers the barrier to entry without sacrificing quality or creativity”, clearly framing it as the go-to entry-level option in 2025. • **TurntableTrader – Beginner’s guide to buying DJ equipment (2025)** In their “Budget under £500” section, they list the DDJ-FLX4 as the main 2-channel controller

recommendation for learning beatmatching, EQing, looping and FX, explicitly labelling it "compact, beginner-friendly".

- **Community consensus (optional / less formal)**
  On r/Beatmatch, multiple users say the DDJ-FLX4 is "literally the recommended beginner controller" on that subreddit and describe it as "the best beginner controller" in their opinion.

If you phrase it as "one of the most popular and widely recommended beginner DJ controllers", these sources have your back.

2. "Does almost everything a DJ could want" / "90% of what you need"

You're basically saying: it covers nearly all core DJ functions and is something you can grow with.

- **MusicRadar review** – praises the FLX4's smart mixing tools, Bluetooth, mic integration and general functionality, calling it a likely "successful successor" to the DDJ-400 and a top beginner option.
- **MusicRadar beginner-controller guide** – calls it "one of the most versatile beginner DJ controllers", emphasising its hardware capability and wide software compatibility (Rekordbox + Serato, multiple devices).
- **Recordcase review** – says the FLX4 "sets the new standard for entry-level DJ controllers" thanks to its feature set (streaming integration, smart-mixing aids, multi-device support).
- **DJ Hookup deep-dive** – concludes that "the DDJ-FLX4 is a great starter controller" aimed at beginners and hobbyists, and explicitly calls it "a controller that you can grow with", i.e. not something you outgrow immediately.
- **Equipboard summary** – says the FLX4 "redefines entry-level controllers" by combining the strengths of the DDJ-400 with new tools like Smart CFX and Smart Fader, improving mixing for beginners.

These support your "does 90% of everything you need" angle without you needing a precise number.

3. "Fantastic for beginners" & transferable skills

Here you want: beginner-friendly, and skills that map to more serious Pioneer gear.

- **Pioneer official product page** – states that the DDJ-FLX4 has "a professional feel and a simple, user-friendly design that's perfect for beginners" and that its layout is "inherited from professional Pioneer DJ products". This directly supports both "great for beginners" and the idea that if you can use it, you'll be comfortable on other Pioneer controllers/mixers.
- **Crossfader review** – says the FLX4 is a good choice for beginners and intermediate DJs, "easy to learn and use" while still offering enough features to be creative. They explicitly pitch it as suitable for learning and as a budget option for serious practice.
- **MusicRadar review** – again, calling it one of the best beginner controllers reinforces that it's "fantastic for beginners".
- **Irish and UK retailers** – several product pages present it straightforwardly as a beginner controller:
  - ROR (Irish retailer): "The Pioneer DJ DDJ-FLX4 is the perfect beginner DJ controller."
  - Gear4Music: describes it as "ideal for beginner DJs and creative selectors on the move."

So instead of saying "you'll likely have no issue with most other controllers", you can say something like "its layout is based on Pioneer's professional club gear, so skills transfer easily to other Pioneer controllers and mixers", backed directly by Pioneer's own wording.

4. “Loads of resources out there about how this board works exactly”

You can absolutely justify that – there’s a *ton* of official and third-party material.

- **Pioneer official manuals & support**
  Official “How to use your Pioneer DJ DDJ-FLX4” page with downloadable full manual, quick-start guides and usage notes.
- **Pioneer official tutorial playlist**
  YouTube playlist “Tutorials – DDJ-FLX4 2-channel DJ Controller”, produced by Pioneer / AlphaTheta, with short videos explaining key features.
- **Crossfader “Getting started with the DDJ-FLX4”**
  Detailed article and accompanying videos walking beginners through setup, basic mixing and smart-features on the FLX4; explicitly frames it as “a great choice for beginners”.
- **Beginner DJ tutorials specifically on the FLX4**
  There are multiple beginner-focused YouTube videos like:
  - “Getting Started With The DDJ-FLX4 – Beginner DJ Tutorial”
  - “Learn to DJ in under 5 mins with the DDJ-FLX4”
  - Beatmatching, drop-mixing etc. tutorials centred entirely on the FLX4.
- **Courses & bundles built around the FLX4**
  Crossfader and others sell full “DDJ-FLX4 Pro Course” and “Beginner to Pro Bundle”, showing that there is a whole ecosystem of structured teaching built specifically on this controller.

That’s easily enough to justify “loads of resources”.

5. How you might actually write it

If you want to keep your original tone but make it academically defensible, you could rewrite the start of that paragraph roughly like this (you can adjust the wording to match the rest of your thesis):

The Pioneer DDJ-FLX4 is widely regarded as one of the most popular and versatile beginner DJ controllers on the market, and is frequently recommended in “best beginner controller” guides and buyer’s articles. It offers almost all of the core features a new DJ needs – including full mixer controls, performance pads, smart mixing tools and compatibility with both Rekordbox and Serato – while remaining affordable and portable. Its control layout is inherited from Pioneer’s professional club products, so skills learned on the DDJ-FLX4 transfer well to other Pioneer controllers and mixers. In practice, this means that if a beginner learns on an FLX4, moving to most other modern Pioneer controllers should be relatively straightforward. Finally, there is an extensive ecosystem of official manuals, tutorials and third-party courses focused specifically on the DDJ-FLX4, which makes it especially suitable as a teaching tool.

Your “Ford Fiesta of the DJ world” line can stay as your own analogy – that doesn’t need a citation.

2. CHAT 2: FLX4 TEMPO ADJUSTMENT RESEARCH

Prompt
On a DDJ-FLX4 How is the range of the tempo adjust calculated? Like if I have a song that's 120 bpm, can the lowest the tempo adjust go to is 100bpm and the highest is 140 bpm??? How is it calculated? provide references to backup your answer

Response
Short version: the FLX4 tempo fader is **percentage-based**, not "120 → 100–140" fixed. The range is set in your DJ software (Rekordbox / Serato / djay), and the fader just applies **±X % of the original BPM**.

1. What the tempo range actually is

On the DDJ-FLX4, the tempo fader sends a value to the software; the **software** decides the range:

- **Rekordbox**: tempo range options are **±6 %, ±10 %, ±16 % and ±100 % (WIDE)**.
  At ±6 %, the resolution is 0.02 % per fader "step"; at ±10 %/±16 % it's 0.04 %, and at WIDE it's 0.5 %.
- **Serato DJ Lite / Pro**: tempo range on the virtual decks is typically **±8 %, ±16 %, ±50 %**, depending on hardware.
- **Algoriddim djay** (iPad/FLX4): users commonly use **±8 %, ±16 %, ±50 % or ±75 %** as ranges.

On the FLX4 itself you change the range with:

- **Rekordbox / djay**: SHIFT + BEAT SYNC on that deck → cycles the tempo range.
- **Serato**: SHIFT + SYNC toggles tempo range per deck (same idea, slightly different mapping).

Reddit people complaining about "small tempo faders" on FLX4 are literally being told to **change the range from +6 to +10/+16/WIDE**, which matches the official Rekordbox ranges.

2. How it's calculated (your 120 BPM example)

General formula:

New BPM = Original BPM × (1 + p/100)
where **p** is between **–R** and **+R**, and **R** is your chosen range (6, 8, 10, 16, 50, 100…).

So for a track at **120 BPM**:

- **±6 % range (Rekordbox)**
  - Min: 120 × 0.94 = **112.8 BPM**
  - Max: 120 × 1.06 = **127.2 BPM**
- **±10 % range**
  - Min: 120 × 0.90 = **108 BPM**
  - Max: 120 × 1.10 = **132 BPM**
- **±16 % range**
  - Min: 120 × 0.84 = **100.8 BPM**
  - Max: 120 × 1.16 = **139.2 BPM**
  -

So your "120 → 100–140" is **basically ±16.7 %**. With the real ±16 % range you'll see about **101–139 BPM**, not exactly 100/140.

For big ranges:

- **±50 %** → 120 × (0.5–1.5) = **60–180 BPM**
- **±100 % (WIDE)** → 120 × (0–2) = **0–240 BPM** (practically you don't go near 0, but that's the math).

The BPM numbers you see on screen are just that formula, rounded to 0.01 or 0.1 BPM depending on the software and range.

3. Direct answer to your question

"If I have a song that's 120 BPM, can the lowest the tempo adjust go to is 100 BPM and the highest 140 BPM?"

Only if you pick a range that's **about ±16–17 %**. On an FLX4 with Rekordbox:

- **At ±16 %** you'll get **~101–139 BPM**, very close to 100–140 but not exact.
- If you need exactly 100–140, you'd need a **custom ~±16.7 % range**, which Rekordbox doesn't offer out of the box; it's fixed to specific percentage presets.

So:

- The FLX4 **doesn't care about absolute BPM numbers**.
- You pick a **percentage range** in your DJ software.
- The fader then moves the track's speed within that **±percentage of the original BPM** using the formula above.

Next Prompt
How can i change tempo of a stream in godot?
Response
In Godot you **can't** directly change "tempo" like a DJ time-stretch (keep pitch the same but change speed). What you *can* do out of the box is: • Change **speed + pitch together** using pitch_scale. That's it. No fancy Elastic Pro crap built in.
Basic way: use pitch_scale on AudioStreamPlayer If you've an AudioStreamPlayer (or AudioStreamPlayer3D) node: # Assume this is on the same node or you got a reference to it @onready var player: AudioStreamPlayer = $AudioStreamPlayer func set_playback_speed(multiplier: float) -> void: # 1.0 = normal speed # 2.0 = double speed (1 octave up) # 0.5 = half speed (1 octave down) player.pitch_scale = multiplier So if you want to slow the track to 80% speed: set_playback_speed(0.8) That's literally how you "change tempo" in Godot, engine-wise.
For BPM-based tempo (e.g. DJ controller stuff) Say your track is analysed as **track_bpm**, and you want it to play at **target_bpm**, you just: func set_tempo_from_bpm(track_bpm: float, target_bpm: float) -> void: if track_bpm <= 0.0: return var ratio = target_bpm / track_bpm player.pitch_scale = ratio Example: track is 120 BPM, you want 128 BPM: set_tempo_from_bpm(120.0, 128.0) # pitch_scale ≈ 1.0666 That will speed it up and pitch it up, just like playing a record faster.
What you *cannot* do natively • No **time-stretch with constant pitch** (Serato/Rekordbox style key lock). • No independent control of pitch vs tempo. • No built-in phase vocoder / elastique-style algorithm. If you want real DJ-style tempo changes with **key lock** you'd need to: • Use an external audio processing lib (C++/GDExtension, e.g. rubberband, SoundTouch, etc.), or • Offline process the audio yourself and load pre-stretched files. So: in Godot itself, "change tempo of a stream" = **set pitch_scale**. Anything more advanced, you're building or binding it yourself.

3. CHAT 3: LISTING OF ALL MUSIC SCALES

Prompt
Please make me a list of every scale that there is, dont say "A# Major" just say "Major" In the format [{Scale_1}, {Scale_2}, ...]
Response
[Major, Ionian, Natural_Minor, Aeolian, Dorian, Phrygian, Lydian, ...

4. CHAT 4: AUDIO METADATA RESEARCH

Prompt
So my project revolves around importing music and using those tracks in a standardised way. All the different songs I found have different metadata, Godot 4.6 treats Mp3, Wav, OGG files all differently. Is there a standard format or anywhere where I can find this data all standardised and free? or even a standard convention?
Response
Yeah… welcome to audio. There **isn't one universal, clean "standard metadata format"** across MP3/WAV/OGG that engines magically expose the same way. Each container does its own thing, and "DJ-relevant" metadata (BPM, key, beatgrid, cue points) is often **proprietary** anyway. Here's the least-painful way to make it sane: • **1) Separate two problems: "tags" vs "DJ analysis"** • **File tags (sometimes present, often messy)** • MP3: typically ID3 tags • OGG: Vorbis comments • WAV: RIFF INFO / BWF chunks / etc (less consistent) Godot will happily *play* these formats, but it won't reliably give you a standardised tag view out of the box (especially MP3 ID3). Godot's MP3 class docs focus on playback, not tag extraction. **Practical fix:** treat embedded tags as "best-effort". • **DJ analysis (usually NOT in tags)** • BPM, musical key, beatgrid, cue points, energy, waveform peaks… This stuff is rarely standardised in the audio file itself, and when it is, it's commonly stored in **app-specific databases** (Rekordbox/Serato/Traktor etc). So you'll end up computing/storing it yourself. • **2) Use a *real* free metadata source: MusicBrainz (+ AcoustID)** If you want **standardised, free, legal metadata**, MusicBrainz is the obvious choice: • MusicBrainz **core data is CC0** (basically public domain) To reliably match a random file to MusicBrainz, don't depend on filenames/tags—use audio fingerprinting: • **Chromaprint** generates a fingerprint from the audio • **AcoustID** uses those fingerprints to identify tracks and then suggests tags using MusicBrainz metadata So your pipeline becomes: 1. Fingerprint audio (Chromaprint) 2. Lookup in AcoustID 3. Get MusicBrainz IDs + metadata 4. Store it in *your* schema • **3) "Free" alternatives (with strings attached)**

Discogs can be useful, but be careful: they split data into CC0 vs "Restricted Data" with extra usage rules.
So it's not the clean "free and do anything" situation MusicBrainz gives you.

- **4) The actually-solid convention: make your own canonical schema**

Pick a single internal structure and always convert into it, e.g.:

- title, artist, album, duration, year, genre
- bpm, key, beatgrid, cue_points
- loudness, waveform_peaks_path
- source_file, import_hash, mbid (optional)

Store it as a **sidecar JSON** (or a tiny SQLite DB) keyed by a stable ID (hash of audio or AcoustID/MusicBrainz IDs). That way Godot's inconsistent tag support stops mattering.

- **5) Godot-specific reality check**

If your pain is "Godot treats MP3/WAV/OGG differently": don't fight it at runtime.

- Either **standardise files at import** (e.g., convert everything to OGG or WAV in a pre-processing step),
- Or keep originals but **always generate/maintain your metadata in your own DB**.

Also: there are community plugins that add MP3 ID3 extraction / unify metadata access in Godot, if you want to avoid writing a parser yourself.

- **Bottom line**
- **Standard convention for tags across all audio formats?** Not really.
- **Free, standardised metadata source?** Yes: **MusicBrainz (CC0)** + **AcoustID/Chromaprint** for matching.
- **DJ metadata like BPM/key/beatgrid?** Assume you'll compute + store it yourself.

If you want, I'll sketch a clean import pipeline design for your Godot project (classes, data flow, where to cache, and what you should compute vs lookup).

5. CHAT 5: RESEARCH OTHER MUSIC STREAMING PLATFORMS

Prompt

Could you fill this gdscript enum out with all possible 3rd party music platforms that a song may originate from, include weird ones from russia and china too, like Yandex music, QQ, Gaana, VK, Tidal? what i have so far is enum E_Track_Origin_Type { LOCAL_STORAGE, SPOTIFY, APPLE_MUSIC, DEEZER, YT_MUISC, # YouTube Music SOUNDCLOUD, ONEDRIVE, ... }

Response

You're **never** going to catch *every* platform on earth (there's thousands, half of them die every year), so do the sensible thing: cover **(1) big global**, **(2) regional giants (Russia/China etc.)**, **(3) DJ stores/pools**, **(4) cloud/storage/import paths**, and keep **UNKNOWN / OTHER** as escape hatches. Here's a solid enum you can actually ship:

```gdscript
enum E_Track_Origin_Type {
    # ---- fallback / misc ----
    UNKNOWN = 0,
    OTHER = 1,

    # ---- local / self-hosted ----
    LOCAL_STORAGE = 10,
    LOCAL_NETWORK_SMB = 11,   # NAS / Windows share
    LOCAL_NETWORK_DLNA = 12,  # UPnP / DLNA media servers
    LOCAL_HTTP_URL = 13,      # direct URL (http/https)
    LOCAL_FTP = 14,
    LOCAL_SFTP = 15,
    LOCAL_WEBDAV = 16,

    # ---- cloud storage ----
    ONEDRIVE = 100,
    GOOGLE_DRIVE = 101,
    DROPBOX = 102,
    ICLOUD_DRIVE = 103,
    BOX = 104,
    MEGA = 105,
    NEXTCLOUD = 106,

    # ---- major global streaming ----
    SPOTIFY = 200,
    APPLE_MUSIC = 201,
    ITUNES_STORE = 202,       # purchases / library
    AMAZON_MUSIC = 203,
    YOUTUBE_MUSIC = 204,
    DEEZER = 205,
    TIDAL = 206,
    QOBUZ = 207,
    PANDORA = 208,
    IHEARTRADIO = 209,
    NAPSTER = 210,

    # ---- UGC / creator platforms ----
    SOUNDCLOUD = 300,
    BANDCAMP = 301,
    MIXCLOUD = 302,
    AUDIOMACK = 303,
    HEARTHIS_AT = 304,
    REVERBNATION = 305,

    # ---- DJ download stores ----
    BEATPORT = 400,
    BEATSOURCE = 401,
    TRAXSOURCE = 402,
    JUNO_DOWNLOAD = 403,
    TRACKITDOWN = 404,
    SEVENDIGITAL = 405,
    BLEEP = 406,
    BOOMKAT = 407,

    # ---- DJ record pools ----
    BPM_SUPREME = 450,
    DJCITY = 451,
    ZIPDJ = 452,
    DIGITAL_DJ_POOL = 453,

    # ---- Russia / CIS ----
    YANDEX_MUSIC = 500,
    VK_MUSIC = 501,
    ZVUK = 502,       # (ex SberZvuk branding in the wild)
    MTS_MUSIC = 503,
    UMA_BOOM = 504,  # BOOM/UMA ecosystem

    # ---- China (mainland) ----
    QQ_MUSIC = 600,
    NETEASE_CLOUD_MUSIC = 601,
    KUGOU_MUSIC = 602,
    KUWO_MUSIC = 603,
    MIGU_MUSIC = 604,
    XIAMI_MUSIC_LEGACY = 605, # dead, but old metadata still exists

    # ---- Japan / Korea / Taiwan ----
    LINE_MUSIC = 650,
    AWA = 651,
    RECOCHOKU = 652,
    KKBOX = 653,
    MELON = 654,
    GENIE_MUSIC = 655,
    FLO = 656,

    # ---- India / South Asia ----
    JIO_SAAVN = 700,
    GAANA = 701,
    WYNK_MUSIC = 702,
    HUNGAMA_MUSIC = 703,

    # ---- MENA ----
    ANGHAMI = 750,
    ROTELA? = 751, # (delete if you don't want questionable ones)
    ROTELA_MUSIC = 752, # keep one name only if you use it
```

A couple of reality checks:

- The big Russian players you specifically asked for (**Yandex Music, VK Music, Zvuk**) are genuinely mainstream over there.
- China's big streaming brands in practice include **QQ Music, NetEase Cloud Music, KuGou**.
- If you're supporting "DJ sources", stores like **Beatport/Traxsource/Juno Download** are common.
- **Boomplay** and **Joox** are legit regional platforms you'll run into in distribution lists.

If you're saving this enum to disk as an int: **don't reorder entries later** (or keep explicit numeric IDs like above), otherwise you'll corrupt older saves.

Next prompt
Could you do the same but for a stringname array now like ["Spotify", "YouTube Music", "Yandex Music" .....
Response

gdscript

```
# Godot 4.x: StringName literals use the & prefix.
const TRACK_ORIGIN_NAMES: Array[StringName] = [
    &"Unknown",
    &"Other",

    # local / self-hosted
    &"Local Storage",
    &"Local Network (SMB)",
    &"Local Network (DLNA/UPnP)",
    &"Direct URL (HTTP/HTTPS)",
    &"FTP",
    &"SFTP",
    &"WebDAV",

    # cloud storage
    &"OneDrive",
    &"Google Drive",
    &"Dropbox",
    &"iCloud Drive",
    &"Box",
    &"MEGA",
    &"Nextcloud",

    # major global streaming
    &"Spotify",
    &"Apple Music",
    &"iTunes Store",
    &"Amazon Music",
    &"YouTube Music",
    &"Deezer",
    &"TIDAL",
    &"Qobuz",
    &"Pandora",
    &"iHeartRadio",
    &"Napster",

    # UGC / creator platforms
    &"SoundCloud",
    &"Bandcamp",
    &"Mixcloud",
    &"Audiomack",
    &"hearthis.at",
    &"ReverbNation",

    # DJ stores
    &"Beatport",
    &"Beatsource",
    &"Traxsource",
    &"Juno Download",
    &"Trackitdown",
    &"7digital",
    &"Bleep",
    &"Boomkat",
```

gdscript

```
    # DJ pools
    &"BPM Supreme",
    &"DJcity",
    &"ZIPDJ",
    &"Digital DJ Pool",

    # Russia / CIS
    &"Yandex Music",
    &"VK Music",
    &"Zvuk",
    &"MTS Music",
    &"BOOM (VK/UMA)",

    # China
    &"QQ Music",
    &"NetEase Cloud Music",
    &"KuGou Music",
    &"KuWo Music",
    &"Migu Music",
    &"Xiami (legacy)",

    # Japan / Korea / Taiwan
    &"LINE MUSIC",
    &"AWA",
    &"Recochoku",
    &"KKBOX",
    &"Melon",
    &"Genie Music",
    &"FLO",

    # India / South Asia
    &"JioSaavn",
    &"Gaana",
    &"Wynk Music",
    &"Hungama Music",

    # MENA
    &"Anghami",

    # Southeast Asia
    &"Joox",
    &"Zing MP3",
    &"NhacCuaTui",

    # Africa
    &"Boomplay",
    &"Mdundo",

    # niche / classical
    &"IDAGIO",
    &"Naxos Music Library",
]
```

6. CHAT 6: WAVEFORM TEXTURE RESEARCH

Prompt
One of the features I want is that my 2 main tracks will be displayed on a "Hub" interface that will show the song information like, BPM, music key, artist/album details, and the playback position of the song of the relevant track. What I want is, similar to Rekordbox, is a preview of the upcoming audio waveform so that you can see the upcoming spectrum... In godot 4.6, tell me how I could do this?

Response
Godot doesn't give you a Rekordbox-style "upcoming waveform" UI out of the box. You have to **generate waveform data from the track** (ideally as PCM), **cache it**, then **draw a moving window** based on the deck's current playback time. Below is a sane way to do it in **Godot 4.6**. **The approach that actually works** **1) Load the track as WAV (PCM) so you can read samples** For runtime-loaded user files, **WAV is the easiest** because AudioStreamWAV exposes the raw sample bytes (data), plus sample-rate + stereo flags, and even has load_from_file() in 4.x. If you insist on MP3/OGG: Godot decodes those internally and doesn't hand you "all decoded PCM upfront" in a nice way, so you'll either: • convert to WAV on import (recommended for your project stage), or • use a native decoder (GDExtension/ffmpeg) and feed PCM yourself. **2) Build a lightweight "peak cache" (min/max per time slice)** Rekordbox isn't drawing every sample. It stores "peaks" per small chunk (e.g. 256–2048 frames per peak), so drawing is cheap. This fits your own plan of analysing waveforms *before* effects/bus processing. **3) Draw a scrolling window that looks ahead** Every f-ish playback time, • draw [now - behind, now + ahead] from the cached peaks, • draw a playhead line at now. For accurate playback time, Godot recommends compensating for mixing + output latency: get_playback_position() + AudioServer.get_time_since_last_mix() - AudioServer.get_output_latency() **4) Put it on your XR "Hub"** Make the waveform a normal Control UI, put it inside a SubViewport, then display the viewport texture on a quad/3D panel in front of the user. **"Upcoming spectrum" (coloured waveform) options** **Option A (easy): colour by *live* spectrum (not upcoming)** Attach an AudioEffectSpectrumAnalyzer to the deck's bus and query it via AudioServer.get_bus_effect_instance(). That gives you magnitudes for frequency ranges. **Downside:** it only reflects what's playing *now*, not what's coming next. **Option B (Rekordbox-like): precompute colours per peak (upcoming)** When you build peaks, also compute **3-band energy** per peak (low/mid/high) and store 3 floats. Then draw each column with a colour based on those values. • This needs an FFT (or a simpler filterbank). Do it **offline/background thread**, not every frame.

Where this plugs into your project cleanly

- Your AudioTrack metadata can carry:
  - stream_path, bpm, key, artist, etc.
  - waveform_cache (Resource) or a cached file path in user://waveforms/...
- Since you're already thinking in terms of "input pre-FX vs post-FX" busses, the waveform preview belongs to the *track input* side.

Next Prompt

I want to use FFMPEG instead to make this texture. What I'll do is compile FFMPEG as android binary that can then be run on the Meta quest 3s by a background worker thread and will supply that waveform texture to the scene. Tell me how I could get ffmpeg set up to do this, does meta quest 3s support his, provide references to back up your answers

Response

Yeah, you *can* use FFmpeg on a Quest 3S — **but doing it as a separate "ffmpeg" executable you spawn at runtime is the annoying, fragile way**. The solid way is **FFmpeg as native libs (.so)** called from an Android plugin / JNI.

Here's what's actually supported and how to set it up.

- **Does Meta Quest 3S support this?**

Yes, in the only way Android actually expects: native code via the Android NDK.

- Meta's own native dev docs for Quest explicitly talk about **VR development on Android** using their Mobile SDK samples.
- Quest 3S is **Snapdragon XR2 Gen 2** (so you're targeting **ARM64 / arm64-v8a**).
- Android supports native libraries through the **NDK** (that's the whole point of it).

What's *not* reliably supported: "ship an ffmpeg ELF binary, extract it, chmod it, and spawn it". That *sometimes* works, sometimes fails depending on device/security quirks, and Godot users regularly hit walls with OS.execute() on Android.

So: Quest supports FFmpeg **as libs**, not "random CLI program you spawn" as your main plan.

- **Recommended architecture (do this, not the cursed binary-spawn plan)**
- **Option A (recommended): FFmpeg as .so libraries + Godot Android plugin**

1. **Cross-compile FFmpeg to shared libs** (libavcodec.so, libavformat.so, libavfilter.so, etc.).
2. Wrap what you need in **Kotlin/Java** (or JNI) via a **Godot Android plugin**.
3. Call it from GDScript with Engine.get_singleton(...).

This is the cleanest route because you're not fighting Android process execution weirdness.

- **Option B: Use FFmpegKit (wrapper that runs FFmpeg/FFprobe commands)**

FFmpegKit exists specifically to run FFmpeg commands in-app.
Caveat: it's been in a messy maintenance situation (archived at one point), and Android platform changes (like page-size requirements) can bite you.

- **Option C (your plan): ship an ffmpeg executable and spawn it**

Possible, but expect pain:

- You must **extract from APK assets to internal storage**, then chmod +x, then spawn with ProcessBuilder.
- Godot OS.execute() on Android is a known headache.
  If you insist, I'll still show you how below.

Next Prompt
Ok so Ill make ffmpeg a seperate .SO I dont want to bundle EVERYTHING with FFMPEG so just tell me EXACTLY what components of ffmpeg I need to compile for andorid so that I can run "ffmpeg -i input -filter_complex "aformat=channel_layouts=mono,showwavespic=s=640x120" -frames:v 1 output.png " with no issues I ONLY want the waveforms
Response

You *can't* run that exact command with "just one .so" unless you also ship the **ffmpeg CLI binary** (or you re-implement what the CLI does in your own code). The command you wrote is the **ffmpeg program** driving **libavformat/libavcodec/libavfilter** etc.

So, for **"ffmpeg -i ... -filter_complex ... output.png with only waveforms"**, the **minimum moving parts** are:

- **The FFmpeg pieces you must build (minimum)**
- **1) Libraries (these become .so)**

These are the core libraries needed for *read audio → filtergraph → encode PNG → write file*:

- libavutil (core types/options)
- libavcodec (decoders + PNG encoder)
- libavformat (demuxers + writing output.png via muxer)
- libavfilter (your aformat + showwavespic, plus any auto-inserted conversions)
- libswresample (channel remixing / resampling used by conversion paths)

Strongly recommended (prevents stupid "works on one file, fails on another" cases):

- libswscale **+** enable the scale filter, because FFmpeg may auto-insert video conversion filters when encoding the generated frame.
- **2) Components you must enable inside those libs**

Your command needs these *features enabled* (because you're going to use --disable-everything):

Filters

- showwavespic (the whole point)
- aformat (your command uses it)
- aresample (often auto-inserted for format/channel conversions)
- scale (recommended safety net for pixel format conversion)

Encoder + muxer

- encoder=png
- muxer=image2 (PNG sequence/single image muxer)

Protocols

- protocol=file (because -i input and output.png are files)

Input formats (demuxer + decoder)
This depends entirely on what your "input" can be. There is no "waveform-only" switch that magically reads everything.

Pick ONE of these sets:

- **A) Smallest possible: WAV-only**

If you guarantee you feed FFmpeg **PCM WAV**:

- demuxer=wav

- decoders: at least pcm_s16le (add pcm_f32le, pcm_s24le if you want fewer surprises with WAVs)
- **B) Still small but usable: common music files**

If you want “no issues” for typical DJ-library files, enable at least:

- demuxers: wav, mp3, ogg, flac, mov (for .m4a/.mp4)
- decoders: pcm_* you care about, mp3, vorbis, flac, aac
- parsers (keep it simple): mpegaudio, vorbis, flac, aac
- **3) External dependency you must not forget**

PNG support needs **zlib** available at build time and enabled, otherwise PNG encoding/decoding support can be missing/broken.

Practical Android note (so you don’t waste your time)

- If you truly want “FFmpeg as .so”, you’ll ship **multiple** .so (libavcodec.so, libavformat.so, …). That’s normal.
- To *run the command*, you still ship the ffmpeg binary (linked against those .so) **or** call the libs directly from your plugin and generate the PNG yourself.

7. CHAT 7: AUDIO EFFECTS ORDERING RESEARCH

Prompt
Assuming I want perfect emulation of the Pioneer DDJ-FLX4, does the order of the FX's in the indivisual godot audio bus matter? like I have channel 1 input (the raw audio strea,) going to the channel 1 fx bus (where I will have all the FX with different effects) then being routed to mixer, routed to master Does it make a difference if I do Trim FX, then Low EQ, then mid, then high, does it change if I switch the order around? provide references to support your answer, the exact lines please

Response
Yeah, **the order matters in Godot** — the bus effects are **processed sequentially**. Godot's own docs literally say it: "Audio buses can contain all sorts of effects... and are **applied in order**." (L9206) Same wording in older docs too: "**applied in order**." (L5355) So if you swap effects around, you're changing the signal chain. **For your specific "Trim → EQ" question** If you're doing *perfect-ish FLX4-style channel strip emulation*, the **sensible order** is: **TRIM (input gain) → EQ (HI/MID/LOW) → CFX/Filter → (other FX) → channel fader/crossfader → master (limiter last if you use one)** Why: • **Trim changes the level feeding everything after it**. Pioneer/AlphaTheta defines it as input level: "**TRIM adjusts the volume of the audio coming into the channel** ..." (L21) • And the fader is *output* level: "The **channel fader adjusts the volume of the audio going out of the channel**." (L25) Also: in Godot, gain staging matters because clipping is a thing if you drive stuff too hot: "Setting the level too high can make the sound **clip** ..." (L5363) "To avoid clipping... output of the master bus... never exceeds **0 dB**." (L9170) So **Trim-before-EQ** matches how mixers are *conceptually* laid out (input gain before tone shaping), and it avoids you boosting EQ into clipping later. **"Low EQ then mid then high — does swapping those matter?"** Inside an EQ-only block: **usually not in any audible way** *if* they're just linear EQ filters. But in a real chain (and in Godot), it **can** matter if: • you have **any non-linear/dynamic FX** anywhere (distortion, compressor, limiter, saturator), • or you're hitting **clipping/headroom limits** depending on where the gain happens. And yes, "the way you sequence plug-ins matters" is standard audio practice too: "The way you **sequence plug-ins matters**... order... will change the sound..." (L36) **About your bus routing layout** Your "Channel 1 input bus → Channel 1 FX bus → Mixer → Master" approach is fine — just remember Godot routes **right-to-left**: "Routing always passes audio from buses on the right to buses further to the left." (L9183) So set up your bus list accordingly. **FLX4 control grouping (what you're trying to mirror)** Pioneer's hardware diagram groups the mixer controls like this: "3 **TRIM** ... 4 **EQ (HI, MID, LOW)** ... 5 **CFX** ... 7 **CHANNEL FADER** ... 8 **CROSSFADER**" (L61–L66) That's not a signal-flow chart, but it matches the "gain → EQ → filter/CFX → faders" mental model you're building.

8. CHAT 8. GENERATING TRANSLATIONS

Prompt
For context, I am working on an XR project in godot that is supposed to be like having a Pionneer ddj-flx4, hooked up to my own version of Rekordbox called “Librebox”, theres going to be loads of labels like “EQ hi”, What I want you to do is to go through the attached language csv file that the project uses and using the context Ive given you now, please provide the translations in French, mandarin and irish. The header of the CSV is already done we key (the key of the string to translate),en,fr,zh_CN,ga Check each string you’re translating by using the key name and the English version as a hint sense to determine what the appropriate translation is. If I have “EQ hi” that obvisously means EQ High, but in the context of a DJ Controller, where space is tight, you can use shortened versions of words. Like in French you don’t got to say “S’il vous plait” you can use “SVP” Look at this little snippet here for reference key,en,fr,zh_CN,ga KEY_LEFT,LEFT,GAUCHE,左,CLÉ KEY_RIGHT,RIGHT,DROITE,右,DEAS KEY_TRACK_1_SELECTION,Track 1 Selection,Sélection piste 1, KEY_TRACK_2_SELECTION,Track 2 Selection,Sélection piste 2, Notice how for Left/Right I say gauche/droite, instead of á gauche or á droite, give me short version, not the big full formal version... Give me just the completed CSV afterwards, any text I give you in that’s in the format “KEY_ADD_NEW_TRACK,+ Add New Track”, assume that I’m giving you more text to translate.
Response
ChatGPT proceeded to generate all missing fields. Translations can be found in the final build in the project path “..**/RaveSpin/Godot/Language/LANG_Translation.csv**” [64]

XIII. APPENDIX 5. CODE SAMPLES

A. FFMPEG GD-EXTENSION

1. IMPORTS

```
package org.godotengine.plugin.waveform

import android.util.Log
import com.arthenica.ffmpegkit.FFmpegKit
import com.arthenica.ffmpegkit.FFmpegSession
import com.arthenica.ffmpegkit.FFprobeKit
import com.arthenica.ffmpegkit.ReturnCode
import org.godotengine.godot.Godot
import org.godotengine.godot.plugin.GodotPlugin
import org.godotengine.godot.plugin.UsedByGodot
import java.util.concurrent.Executors
```

2. WAVEFORM IMAGE GENERATION

```
/**
 * Generate a waveform PNG from an audio file
 * Runs ffmpeg in-process via ffmpeg-kit... Blocks until done
 *
 * @param audioPath Absolute path to the audio file (e.g. user:// or content path)
 * @param outputPngPath Absolute path where the PNG will be saved
 * @param width Width of the waveform image (Please no more than 8192)
 * @param height Height of the waveform image (200px is plenty)
 * @return 0 on success, non-zero on failure
 */
@UsedByGodot
fun generate(audioPath: String, outputPngPath: String, width: Int, height: Int): Int {
    if (audioPath.isBlank() || outputPngPath.isBlank()) {
        Log.e( tag = pluginName, msg = "generate: empty path")
        return -1
    }

    val img_width = width.coerceIn(1, 8192) // No wider than 8192px
    val img_height = height.coerceIn(1, 512) // Should only ever be 200px high ideally
    val cmd = "-y -i \"$audioPath\" -filter_complex aformat=channel_layouts=mono,showwavespic=s=${img_width}x${img_height} -frames:v 1 \"$outputPngPath\""

    return runBlockingOnExecutor {
        val session : FFmpegSession = FFmpegKit.execute( command = cmd)

        if (ReturnCode.isSuccess( returnCode = session.getReturnCode() ))
        {
            Log.e( tag = pluginName, msg = "ffmpeg succeeded making the Waveform image! ")
            0
        }
        else {
            Log.e( tag = pluginName, msg = "ffmpeg failed: ${session.failStackTrace}")
            -1
        }
    }
}
```

3. METADATA EXTRACTION

```
 * Returns ffprobe JSON metadata for an audio file
 * Returns "{}" when metadata cant be read for whatever reason.
 */
@UsedByGodot
fun getMetadataJson(audioPath: String): String {
    if (audioPath.isBlank()) {
        return "{}"
    }

    val cmd = "-v quiet -print_format json -show_format -show_streams \"$audioPath\""

    return runBlockingOnExecutor {
        val session = FFprobeKit.execute( command = cmd)

        if (ReturnCode.isSuccess(session.returnCode)) {
            val output = session.output ?: ""
            if (output.trim().startsWith( prefix = "{")) output else "{}"
        }

        else {
            Log.e( tag = pluginName,  msg = "ffprobe failed: ${session.failStackTrace}")
            "{}"
        }
    }
}

/**
 * Try to get the embedded artwork to output image path
 * Returns 0 on success, failure gonna be something else...
 */
@UsedByGodot
fun extractArtwork(audioPath: String, outputImagePath: String): Int {
    if (audioPath.isBlank() || outputImagePath.isBlank())
    {
        return -1
    }
    val cmd = "-y -i \"$audioPath\" -an -vcodec copy \"$outputImagePath\""
    return runBlockingOnExecutor {
        val session = FFmpegKit.execute( command = cmd)
        if (ReturnCode.isSuccess(session.returnCode)) 0 else -1
    }
}
```

B. BUS MANAGER

1. WEIGHTS OF EQ KNOBS TO EQ BANDS

```
# Im explicitingly setting the int values cause I want to make sure that band[0] IS 32hz
enum E_BAND_HZ
{
    HZ_32       = 0,
    HZ_100      = 1,
    HZ_320      = 2,
    HZ_1000     = 3,
    HZ_3200     = 4,
    HZ_10000    = 5,
}
```

```
# So EQ bands are made up of 6+ different freqs bands, but we only have "high" "mid" "low"
# each knob will have different weights for how they affect the other hz
# these weights are pulled out of thin air
const EQ6_LOW_WEIGHTS : Dictionary[E_BAND_HZ, float] = {
    E_BAND_HZ.HZ_32 : 1.0,
    E_BAND_HZ.HZ_100 : 0.8,
    E_BAND_HZ.HZ_320 : 0.3,
    E_BAND_HZ.HZ_1000 : 0.0,
    E_BAND_HZ.HZ_3200 : 0.0,
    E_BAND_HZ.HZ_10000 : 0.0
    }
```

```
const EQ6_MID_WEIGHTS : Dictionary[E_BAND_HZ, float] = {
    E_BAND_HZ.HZ_32 : 0.0,
    E_BAND_HZ.HZ_100 : 0.2,
    E_BAND_HZ.HZ_320 : 0.7,
    E_BAND_HZ.HZ_1000 : 1.0,
    E_BAND_HZ.HZ_3200 : 0.4,
    E_BAND_HZ.HZ_10000 : 0.0
}
```

```
const EQ6_HIGH_WEIGHTS : Dictionary[E_BAND_HZ, float] = {
    E_BAND_HZ.HZ_32 : 0.0,
    E_BAND_HZ.HZ_100 : 0.0,
    E_BAND_HZ.HZ_320 : 0.0,
    E_BAND_HZ.HZ_1000 : 0.0,
    E_BAND_HZ.HZ_3200 : 0.6,
    E_BAND_HZ.HZ_10000 : 1.0
}
```

```gdscript
# So Give me the low, mid, hi alpha floats (0-1) and then spit out a dictionary of the new HZ but in 0-1 form
static func Calculate_Weights_Normal(low_alpha : float = 0.5, mid_alpha : float = 0.5, high_alpha : float = 0.5) -> Dictionary[E_BAND_HZ, float]:
    low_alpha = clampf(low_alpha, 0.0, 1.0)
    mid_alpha = clampf(mid_alpha, 0.0, 1.0)
    high_alpha = clampf(high_alpha, 0.0, 1.0)

    var new_weights : Dictionary[E_BAND_HZ, float] = {}
    for band in E_BAND_HZ.values():
        var weighted_alpha : float = (
            low_alpha * EQ6_LOW_WEIGHTS[band] +
            mid_alpha * EQ6_MID_WEIGHTS[band] +
            high_alpha * EQ6_HIGH_WEIGHTS[band]
        )
        new_weights[band] = clampf(weighted_alpha, 0.0, 1.0)

    return new_weights
```

```gdscript
# Same thing as Calculate_Weights_Normal, except returns raw dB values for each EQ6 band.
# Input is still low/mid/high alpha (0-1), which is remapped to EQ_DB_MIN...EQ_DB_MAX first.
static func Calculate_Weights_DB(low_alpha : float = 0.5, mid_alpha : float = 0.5, high_alpha : float = 0.5) -> Dictionary[E_BAND_HZ, float]:
    low_alpha = remap(low_alpha, 0.0, 1.0, 0.0, 2.0)
    mid_alpha = remap(mid_alpha, 0.0, 1.0, 0.0, 2.0)
    high_alpha = remap(high_alpha, 0.0, 1.0, 0.0, 2.0)

    var low_db : float =    clampf(linear_to_db(low_alpha), BUS_MANAGER.EQ_DB_MIN, BUS_MANAGER.EQ_DB_MAX)
    var mid_db : float =    clampf(linear_to_db(mid_alpha), BUS_MANAGER.EQ_DB_MIN, BUS_MANAGER.EQ_DB_MAX)
    var high_db : float =   clampf(linear_to_db(high_alpha), BUS_MANAGER.EQ_DB_MIN, BUS_MANAGER.EQ_DB_MAX)

    var new_band_dbs : Dictionary[E_BAND_HZ, float] = {}
    for band in E_BAND_HZ.values():
        var band_db : float = (
            low_db * EQ6_LOW_WEIGHTS[band] +
            mid_db * EQ6_MID_WEIGHTS[band] +
            high_db * EQ6_HIGH_WEIGHTS[band]
        )
        new_band_dbs[band] = clampf(band_db, EQ_DB_MIN, EQ_DB_MAX)

    return new_band_dbs
```

2. ADDING AUDIO EFFECTS

```gdscript
## Adds or replaces beat FX on the selected track bus
## If Poly FX is off we can ONLY have 1 FX active
static func add_beat_fx(fx_type: E_BEAT_FX_TYPE, channel: int) -> void:
    channel = Utility.Clamp_to_Valid_TrackID(channel)
    var bus_index: int = Get_Channel_Index_i(channel)
    if bus_index < 0:
        return
    var active_fx : Dictionary = _get_active_effects_for_channel(channel)

    if ONE_FX_AT_A_TIME:
        # Only slot 4's used... replace whats is there.
        if AudioServer.get_bus_effect_count(bus_index) > BEAT_FX_SLOT_SINGLE:
            AudioServer.remove_bus_effect(bus_index, BEAT_FX_SLOT_SINGLE )
            active_fx.clear()
        var effect: AudioEffect = _create_beat_fx_instance(fx_type)
        if effect != null:
            AudioServer.add_bus_effect(bus_index, effect, BEAT_FX_SLOT_SINGLE)
            AudioServer.set_bus_effect_enabled(bus_index, BEAT_FX_SLOT_SINGLE, true)
            active_fx[fx_type] = BEAT_FX_SLOT_SINGLE
    else:
        # Multiple: set next free slot and add effect there and pray
        if active_fx.has(fx_type):
            return  # already on
        var slot: int = _get_next_free_slot(channel)
        var effect: AudioEffect = _create_beat_fx_instance(fx_type)
        if effect:
            AudioServer.add_bus_effect(bus_index, effect, slot)
            AudioServer.set_bus_effect_enabled(bus_index, slot, true)
            active_fx[fx_type] = slot
```

3. SET AUDIO EFFECT LEVEL

```gdscript
## Each effect is slightly different... alpha 0 = inaudible, alpha 1 = full, but effects got different parameter scaling... yay
static func _set_beat_fx_effect_level(effect: AudioEffect, fx_type: E_BEAT_FX_TYPE, alpha: float, channel: int = 0) -> void:
    alpha = clampf(alpha, 0.0, 1.0)

    if effect is AudioEffectDelay:
        var delay: AudioEffectDelay = effect as AudioEffectDelay
        var db_off: float = -80.0
        delay.tap1_level_db = lerpf(db_off, 0.0, alpha)
        delay.tap2_level_db = lerpf(db_off, -6.0, alpha)
        delay.feedback_level_db = lerpf(db_off, -6.0, alpha)

    elif effect is AudioEffectReverb:
        var reverb: AudioEffectReverb = effect as AudioEffectReverb
        reverb.dry = 1.0 - alpha
        reverb.wet = alpha

    elif effect is AudioEffectChorus:
        var choris: AudioEffectChorus = effect as AudioEffectChorus
        for i in range(choris.voice_count):
            choris.set_voice_depth_ms(i, lerpf(0.0, 3.0, alpha))
            choris.set_voice_level_db(i, lerpf(-80.0, 0.0, alpha))

    elif effect is AudioEffectPhaser:
        var phasor: AudioEffectPhaser = effect as AudioEffectPhaser
        phasor.depth = lerpf(0.0, 1.0, alpha)

    elif effect is AudioEffectPitchShift:
        var pitshshift: AudioEffectPitchShift = effect as AudioEffectPitchShift
        var override_scale: float = _get_pitch_shift_scale_override(channel)
        pitshshift.pitch_scale = override_scale if override_scale > 0.0 else lerpf(1.0, 1.5, alpha)

    elif effect is AudioEffectDistortion:
        var distort: AudioEffectDistortion = effect as AudioEffectDistortion
        distort.drive = lerpf(0.0, 1.0, alpha)

    elif effect is AudioEffectCompressor:
        var compres: AudioEffectCompressor = effect as AudioEffectCompressor
        compres.ratio = lerpf(1.0, 4.0, alpha)

    elif effect is AudioEffectLimiter:
        var limtor: AudioEffectLimiter = effect as AudioEffectLimiter
        limtor.threshold_db = lerpf(6.0, -12.0, alpha)

    elif effect is AudioEffectBandPassFilter:
        var bandpass: AudioEffectBandPassFilter = effect as AudioEffectBandPassFilter
        bandpass.resonance = lerpf(0.1, 2.0, alpha)

    elif effect is AudioEffectPanner:
        var paner: AudioEffectPanner = effect as AudioEffectPanner
        paner.pan = lerpf(0.0, 1.0, alpha)

    elif effect is AudioEffectStereoEnhance:
        var steroEnchase: AudioEffectStereoEnhance = effect as AudioEffectStereoEnhance
        steroEnchase.pan_pullout = lerpf(0.0, 1.0, alpha)
```

4. POLY FX AND FX POLICY

```
# ---- Beat FX: single vs multiple, and active state ---
static var ONE_FX_AT_A_TIME: bool = true # FLX4 style (one FX at a time) or allow stacking (AWFUL PERFORMANCE)?

## Returns true when stacking more than one beat FX at once is allowed.
static func Allow_Multiple_FX_at_same_time() -> bool:
    return ONE_FX_AT_A_TIME == false


## Enables or disables beat FX stacking mode.
static func Set_Allow_Multiple_FX_at_same_time(enabled : bool):
    ONE_FX_AT_A_TIME = not enabled


## false means we cant do multiple FX at the same time now
static func Toggle_Allow_Multiple_FX_at_same_time() -> bool:
    ONE_FX_AT_A_TIME = not ONE_FX_AT_A_TIME
    return not ONE_FX_AT_A_TIME


enum E_Track_FX_Policy
{
    FX_Disabled,
    Only_Track_1,
    Only_Track_2,
    Both_Tracks,
}
```

C. CONTROLLER

1. GET SONG COMPLETION PERCENTAGE

```
## Returns 0..1 progress through current track stream, or -1 when missing stream
static func Get_Track_Playback_Alpha(which_track : int) -> float:
    which_track = Utility.Clamp_to_Valid_TrackID(which_track)
    if Controller_Instance.AudioPlayerList[which_track].stream:
        var total_seconds : float = Controller_Instance.AudioPlayerList[which_track].stream.get_length()
        var played_for_seconds : float =  Controller_Instance.AudioPlayerList[which_track].get_playback_position()
        return remap(played_for_seconds, 0.0, total_seconds, 0.0, 1.0)
    else:
        #push_warning("HEY! Audio Player #" + str(which_track) + " doesn't have a stream assigned to it... HOW DID THIS GET MESSED UP?")
        return -1.0
```

2. ON HOT CUE ACTIVATED

```
func _performance_pad_handle_hot_cue(which_track: int, pad_index: int) -> void:
    if AudioPlayerList[which_track] == null or AudioPlayerList[which_track].stream == null:
        return

    if Hot_Cue_Add_Mode_By_Track[which_track]:
        if Has_Hot_Cue(which_track, pad_index):
            var cue_sec: float = Hot_Cue_Sec_By_Track[which_track][pad_index]
            _apply_track_seek(which_track, cue_sec)
        else:
            Hot_Cue_Sec_By_Track[which_track][pad_index] = Utility.Return_Valid(AudioPlayerList[which_track].get_playback_position(), 0.0)
    else:
        Hot_Cue_Sec_By_Track[which_track][pad_index] = HOT_CUE_UNSET_SEC
```

3. JOG WHEELS ACTIVATED

```
func _update_jogwheel_track(which_track: int, delta: float) -> void:
    _try_acquire_jogwheel_finger_from_overlap(which_track)
    var active_finger: Player_Finger = _get_jogwheel_finger_for_track(which_track)
    var target_mesh: MeshInstance3D = _get_jogwheel_mesh_for_track(which_track)
    if target_mesh == null:
        return

    if active_finger != null:
        if not is_instance_valid(active_finger) or not _is_pose_allowed_for_jogwheel(active_finger):
            _set_jogwheel_finger_for_track(which_track, null)
            _clear_jog_angle_tracking(which_track)
            active_finger = null
        else:
            _release_jogwheel_finger_if_out_of_snap_range(which_track)
            active_finger = _get_jogwheel_finger_for_track(which_track)

    if active_finger != null:
        if not _jog_touch_active[which_track]:
            _jog_touch_active[which_track] = true
            _jog_resume_after_touch[which_track] = AudioPlayerList[which_track] != null and AudioPlayerList[which_track].playing and not Aud
            _jog_virtual_position_seconds[which_track] = Utility.Return_Valid(AudioPlayerList[which_track].get_playback_position(), 0.0)
            _jog_has_virtual_position[which_track] = true

        var current_angle_radians: float = _get_jogwheel_touch_angle_radians(which_track, active_finger)
        if not _jog_has_last_angle[which_track]:
            _jog_last_angle_radians[which_track] = current_angle_radians
            _jog_has_last_angle[which_track] = true
            if AudioPlayerList[which_track] != null and AudioPlayerList[which_track].playing:
                AudioPlayerList[which_track].stream_paused = true
            return

        var delta_angle_radians: float = wrapf(current_angle_radians - _jog_last_angle_radians[which_track], -PI, PI)
        _jog_last_angle_radians[which_track] = current_angle_radians
```

```
        var delta_angle_radians: float = wrapf(current_angle_radians - _jog_last_angle_radians[which_track], -PI, PI)
        _jog_last_angle_radians[which_track] = current_angle_radians

        if absf(delta_angle_radians) <= 0.0001:
            # Keep hard-holding current sample position while hand is touching.
            if _jog_has_virtual_position[which_track]:
                _apply_track_seek(which_track, _jog_virtual_position_seconds[which_track])
            return

        # clockwise spin should move audio forward
        delta_angle_radians *= -1.0

        _rotate_jogwheel_mesh_visual(which_track, delta_angle_radians)
        _warp_track_from_jogwheel(which_track, delta_angle_radians)
        return
    elif _jog_touch_active[which_track]:
        _resume_track_after_jog_touch(which_track)
        _jog_touch_active[which_track] = false
        _jog_resume_after_touch[which_track] = false
        _jog_has_virtual_position[which_track] = false
        _clear_jog_angle_tracking(which_track)

    if AudioPlayerList[which_track] != null and AudioPlayerList[which_track].stream != null and AudioPlayerList[which_track].playing
        var track_speed_multiplier: float = maxf(0.0, AudioPlayerList[which_track].pitch_scale)
        var auto_spin_radians: float = TAU * Jogwheel_Auto_Spin_Rotations_Per_Second_At_Normal_Speed * track_speed_multiplier * delta
        _rotate_jogwheel_mesh_visual(which_track, auto_spin_radians)
```

4. PROCESS AND PHYSICS PROCESS

```
func _process(delta: float) -> void:

    # Just try cache the playback pos cause it's expensive to run get_playback_position()
    if i % 3 == 0:
        Track_1_playback_pos = AudioPlayerList.get(0).get_playback_position()
        Track_2_playback_pos = AudioPlayerList.get(1).get_playback_position()

        Track_1_playback_length = AudioPlayerList.get(0).stream.get_length() if AudioPlayerList.get(0).stream else -1.0
        Track_2_playback_length = AudioPlayerList.get(1).stream.get_length() if AudioPlayerList.get(1).stream else -1.0
    i += 1

func _physics_process(delta: float) -> void:
    if not Use_Multi_threaded_looping:
        _loop_thread_main(0)
        _loop_thread_main(1)

    Update_Channel_Tempo_Adjusts()
    Update_Channel_DBs() # crossfades and stuff
    Update_Channel_Trim_EQ_CFX() # trim, EQ hi/mid/low, CFX per track
    # Beat FX "power": 0 = inaudible, 1 = full effect
    BUS_MANAGER.Apply_Beat_FX_Level(0, Beat_FX_Knob.Value)
    BUS_MANAGER.Apply_Beat_FX_Level(1, Beat_FX_Knob.Value)
    _update_jogwheel_track(0, delta)
    _update_jogwheel_track(1, delta)
    _update_performance_pad_feedback(delta)

    $"General Status".text = tr("KEY_CROSSFADE_STATUS") + ": " + str("%0.3f" % remap(Crossfade_Alpha, 0.0, 1.0, -1.0, 1.0))
```

5. SNAP TO DOWNBEAT (USED IN LOOPS AND BEAT-SYNC)

```
## Aligns moved track beat phase to reference track
## Gonna make sure that the tracks both match the downbeat
func _seek_track_phase_to_match(track_to_move: int, reference_track: int):
    var ref_bpm : float = 0.0
    var move_bpm : float = 0.0
    var stream_length: float
    var ref_beat_stream: float
    var move_beat_stream: float
    var latency_correction: float
    var ref_pos: float
    var move_pos: float
    var phase_ratio: float
    var move_beat_index: int
    var new_pos: float

    track_to_move = Utility.Clamp_to_Valid_TrackID(track_to_move)
    reference_track = Utility.Clamp_to_Valid_TrackID(reference_track)
    if AudioPlayerList[reference_track].stream == null or AudioPlayerList[track_to_move].stream == null:
        return

    stream_length = AudioPlayerList[track_to_move].stream.get_length()

    # Beat boundaries in the file are fixed by metadata BPM
    # so we need beat length in stream seconds (60 / base BPM), not real-time
    ref_bpm = LibreBox.Get_Track_BPM(reference_track)
    move_bpm = LibreBox.Get_Track_BPM(track_to_move)

    if ref_bpm <= 0.0 or move_bpm <= 0.0:
        return
```

```
    ref_beat_stream = 60.0 / ref_bpm   # seconds of reference stream per beat
    move_beat_stream = 60.0 / move_bpm # seconds of move stream per beat

    # What Godot hears... and the user hears is different, whack on some latency compensation
    latency_correction = AudioServer.get_time_since_last_mix() - AudioServer.get_output_latency()
    ref_pos = Utility.Return_Valid(AudioPlayerList[reference_track].get_playback_position(), 0.0)

    if AudioPlayerList[reference_track].playing:
        ref_pos += latency_correction

    move_pos = Utility.Return_Valid(AudioPlayerList[track_to_move].get_playback_position(), 0.0)
    if AudioPlayerList[track_to_move].playing:
        move_pos += latency_correction

    # Phase as a ratio 0...1 within the beat (same for any BPM)
    phase_ratio = fmod(ref_pos, ref_beat_stream) / ref_beat_stream

    # Keep the moved track in the same beat region of its own song
    move_beat_index = int(floor(move_pos / move_beat_stream))
    new_pos = move_beat_index * move_beat_stream + phase_ratio * move_beat_stream
    new_pos = clampf(new_pos, 0.0, maxf(0.0, stream_length - 0.001))

    if AudioPlayerList[track_to_move].has_stream_playback():
        AudioPlayerList[track_to_move].stream_paused = false
        AudioPlayerList[track_to_move].seek(new_pos)
    else:
        AudioPlayerList[track_to_move].play(new_pos)
```

D. LIBREBOX

1. STARTUP

```
## Starting up LibreBox scene and first-time song load into the controller
func _ready() -> void:
    _apply_viewport_space_labels()
    HUB_Menu_ref.Track_1 = Track_1_Song
    HUB_Menu_ref.Track_2 = Track_2_Song
    Track_1_Selection_ref.track_selected.connect(On_Song_Change_Track_1)
    Track_2_Selection_ref.track_selected.connect(On_Song_Change_Track_2)
    LibreBox_instance = self
    var success = Refresh()

    await DJ_Controller.Get_Instance_await()
    await get_tree().process_frame
    _configure_external_db_meter_viewports()
    if Track_1_Song:
        DJ_Controller.Get_Instance().LoadTrackIntoMemory(0, Track_1_Song)
    if Track_2_Song:
        DJ_Controller.Get_Instance().LoadTrackIntoMemory(1, Track_2_Song)
    #DJ_Controller.Get_Instance().Sync_LeftTrackBPM_to_RightTrackBPM.connect()

    #JANK... but works
    await get_tree().create_timer(0.1).timeout
    Utility.set_all_is_ready(true)
    if not success:
        Refresh()
```

2. ON TRACK DELETION

```
## Remove song from active decks if that song was just deleted
static func apply_song_deleted_from_library(deleted_metadata_path: String) -> void:
    if deleted_metadata_path.is_empty():
        return

    if LibreBox_instance.Track_1_Song != null and LibreBox_instance.Track_1_Song.resource_path == deleted_metadata_path:
        LibreBox_instance.Track_1_Song = null
        DJ_Controller.Get_Instance().LoadTrackIntoMemory(0, null)
        LibreBox_instance.HUB_Menu_ref.Track_1 = null
        LibreBox_instance.HUB_Menu_ref.Refresh(true)

    if LibreBox_instance.Track_2_Song != null and LibreBox_instance.Track_2_Song.resource_path == deleted_metadata_path:
        LibreBox_instance.Track_2_Song = null
        DJ_Controller.Get_Instance().LoadTrackIntoMemory(1, null)
        LibreBox_instance.HUB_Menu_ref.Track_2 = null
        LibreBox_instance.HUB_Menu_ref.Refresh(false)
```

3. CONFIGURE DB METERS

```
func _configure_db_meter_viewport(viewport_node: XRToolsViewport2DIn3D, meter_bus_name: StringName, use_microphone_input: bool) -> void:
    if viewport_node == null:
        return

    # Force continuous redraw for live level meters... will stay frozen otherwise
    viewport_node.update_mode = XRToolsViewport2DIn3D.UpdateMode.UPDATE_ALWAYS

    # On meters dont have aa consistent init order... got to make sure it's valid
    var meter_scene: Node = viewport_node.get_scene_instance()

    if meter_scene == null:
        return

    meter_scene.set("bus_name", meter_bus_name)
    meter_scene.set("Use_Microphone_Input", use_microphone_input)


func _configure_external_db_meter_viewports() -> void:
    _configure_db_meter_viewport(get_node_or_null("Track 1 DB Level") as XRToolsViewport2DIn3D, &"Channel 1 Input", false)
    _configure_db_meter_viewport(get_node_or_null("Track 2 DB Level") as XRToolsViewport2DIn3D, &"Channel 2 Input", false)
    _configure_db_meter_viewport(get_node_or_null("Mic Level") as XRToolsViewport2DIn3D, &"Microphone Input", true)
```

4. LIBREBOX HUB: STARTUP

```
## Initialises shader material instances and rhythm notifier nodes
func _ready() -> void:

    # gotta do this horribleness because "local to scene" did nothing for some reason
    _waveform_mat_1 = M_HORIZONTAL_PAN.duplicate()
    Track_1_waveformVis.material = _waveform_mat_1
    _configure_hot_cue_shader_colours(_waveform_mat_1)

    _waveform_mat_2 = M_HORIZONTAL_PAN.duplicate()
    Track_2_waveformVis.material = _waveform_mat_2
    _configure_hot_cue_shader_colours(_waveform_mat_2)

    $"VBoxContainer/Track 1 Container/Track 1 Card".Song_Resource = Track_1
    $"VBoxContainer/Track 2 Container/Track 2 Card".Song_Resource = Track_2
    Refresh(true)
    Refresh(false)

    _setup_rhythm_notifiers()
```

```
## Refreshes one side of the hub (track card + waveform texture + BPM + everything)
#TODO: Code duplication here... Fix this to just like Target_Track = 1 ? Set_Track_1 : 2
func Refresh(Set_Track_1 : bool) -> bool:
    var success = true
    if Set_Track_1:
        $"VBoxContainer/Track 1 Container/Track 1 Card".Set_New_Song(Track_1)
        if Track_1:
            if Track_1.Audio_File_Waveform == null:
                Track_1.Attempt_Find_waveform_from_audio_file_path()
        if Track_1 and Track_1.Audio_File_Waveform:
            if not Track_1_waveformVis:
                success = false
            else:
                Track_1_waveformVis.texture = Track_1.Audio_File_Waveform
        else:
            if not Track_1_waveformVis:
                success = false
            else:
                Track_1_waveformVis.texture = null
        if _rhythm_1:
            _rhythm_1.bpm = Track_1.Track_BPM if Track_1 else 120.0
    else:
        $"VBoxContainer/Track 2 Container/Track 2 Card".Set_New_Song(Track_2)
        if Track_2:
            if Track_2.Audio_File_Waveform == null:
                Track_2.Attempt_Find_waveform_from_audio_file_path()
        if Track_2 and Track_2.Audio_File_Waveform:
            if not Track_2_waveformVis:
                success = false
            else:
                Track_2_waveformVis.texture = Track_2.Audio_File_Waveform
        else:
            if not Track_2_waveformVis:
                success = false
            else:
                Track_2_waveformVis.texture = null
        if _rhythm_2:
            _rhythm_2.bpm = Track_2.Track_BPM if Track_2 else 120.0
    return success
```

XIV. APPENDIX 6. WEEKLY LOGS

As the project progressed, I kept weekly logs detailing the progress made that week on the project.

A. SEMESTER 1

1. WEEK 1 – AWAITING PROJECT APPROVAL & STARTED RESEARCH

In the previous weeks, I prepared the final year project proposal. I completed a great deal of research to find out if previous solutions existed already and preformed a comparative analysis between my idea and what others have already done.

After a couple of weeks and some issues regarding supervisors, I was finally assigned my FYP supervisor, and I have sent my proposal now to be signed off.

I am still waiting to receive my signed-off proposal, but this should be coming soon.

2. WEEK 2 – FURTHER INITIAL RESEARCH

This week I completed more research into the existing solutions that already exist and made particular note of what could take and learnt from these solutions. The main area of focus this week was researching more about the various effects that will be required, such as reverb, stereo separation, high/low pass filtering, and various other effects and how they work.

For next week I'm going to create some design documents on how the app will function, with focus being made on minimum viable product specifications and optional specifications.

Unfortunately, that's all that's in the pipeline for now as I have work in other subjects that require more urgent attention.

3. WEEK 3 – INITIAL DRAFT DESIGN

For this week I put a great deal of effort into making the designs for how Rave Spin would operate, such as the Main menu, the socials features menu, and a mock-up of how the main system would work with the audio wave visualisers and more.

I used Unreal Engine to import the 3D model of the DDJ-FLX6 and make the UI components as it was a faster workflow than drafting something in 2D using Adobe Photoshop or by hand.

4. WEEK 4 – INITIAL PROTOTYPE DESIGN

This week primarily focused on making a working prototype in Godot and learning more about audio. I ran into several issues mostly with GD script and its lack of explicit pointers which lead to loads of initialisation issues.

The prototype is still in progress and is coming along nicely. Audio playback and loading is working, and the Audio Bus Layout has been finalised to allow for either 2 tracks (DDJ-FLX4) or 4 tracks (FLX10) at once. Work is being made as of writing this to make a modular script that will work not only for the FLX4 but other components that may be needed.

5. WEEK 5 – FURTHER WORK & INTERIM REPORT

This week I worked primarily on the Interim write up. Further work also went onto the prototype as I have finally obtained a Meta Quest 3S so testing of the application can now proceed properly.

It took a great deal of effort getting Slider controls working as I was facing issues between local vs global positions but ultimately everything worked out.

Buttons work with little to no issue. As of writing this they just require some further finesse and polish to look nicer. Buttons also coincide with actual functionality *(pausing/playing playback)*, sliders haven't had any functionality linked just yet due to the fact that transition logic between global and local space is not working.

6. WEEK 6 – FIRST ENCOUNTER OF GITHUB DESKTOP CORRUPTION BUG

This week I focused more on developing the prototype. Controls were working, sliders and buttons were operating as expected.

But for reasons that are lost on me, none of that is working at the moment. Godot keeps getting my scenes mixed up. Rolling back to a previous commit shows that the functionality can work but my current efforts are spent now to complete the presentation slides that are due for the interim presentation.

Debugging this Godot scene jumbling will be the cause for a major headache.

7. WEEK 7 – INTERIM PRESENTATION

Added more controls to the FLX4.

Not much work was able to be completed due to the preparation for interim presentation and multiple other CA's all due in the same timeframe.

Audio is now responsive to slider changes. Adjusting the channel faders immediately affects volume, crossfade works instantly, etc... Very pleased!

Slider collision seems to be a bit inconsistent and “Jumpy” so will need to consult with Bryan as to what could fix this??

B. SEMESTER 2

1. WEEK 1 – FFMPEG & SECOND ENCOUNTER WITH GITHUB DESKTOP CORRUPTION BUG

My original plan for adding waveform preview for any provided song was to compile my own GD-Extension using C++ and then sideload waveform preview via an OS shell call. I’m going to try compile FFMPEG because it seems to be the most solid for this?

Since decided to add the "Skeleton" for this functionality *(Will attempt to implement it in the future)*, but ultimately, I will use hard-coded pre-rendered waveform images instead so that the project may stay on track and within the current timeframe.

All progress on Monday (7-9 hours) was lost due to an obscure bug so I will be committing progress ideally every 20-ish minutes to avoid such a headache ever occurring again. “Frustration” is an inadequate term to describe what I am currently feeling.

2. WEEK 2 – HAND POSE DETECTION

Finally got hand pose detection working.

Through much experimentation, pinching doesn’t work as the Pose Detection then fumbles and thinks it’s a different pose that is being performed, tried using quaternions to track hand rotation but this didn’t solve it.

To counteract this, I am going to be using a slider that spawns when the user tries to interact with a knob, and ideally use simpler poses like fists and thumbs-up for grabbing/confirmation of UI elements as these poses proved to be the most reliable and kept getting correctly identified.

3. WEEK 3 – HAND POSE LIMITATIONS

While prototyping I discovered that the headsets, in this case the meta quest 3S, struggles reading hand poses at certain angles.

To combat this issue, I figured out what poses produce the most replicable results, in this case it was "Fist" and "Thumbs Up" that consistently registered no matter the hand angle relative to the headset.

My project will now use Fist to activate a pointer that can select UI elements in 3D XR space, and a thumbs up to emit Button pressed events. I will also be using this for Knobs as the turning of knobs is currently unreliable and inconsistent, so I'll be converting knobs to sliders.

4. WEEK 4 – BEAT SYNC

For this week I worked on the "Beat Sync" functionality and some code refactoring.

So far it is working and although it took a good deal of work but now songs can have their BPMs match and play in sync with each other.

The issue now is that when syncing songs, the beat is a multiple of ¼ beats off. Another student, Kim had a similar issue with her Android App, so I'll be in contact with her to devise an effect solution.

Refactoring of code is the next priority and will hopefully take no longer than a day as much of the code was made back to the start of this project when I had no solid understanding of Godot or GD-Script.

5. WEEK 5 – EQ COMPLETED, FFMPEG, AUDIO BUS LAYOUT SIMPLIFICATION, AND RELOCATION

This week has been a very productive one. Firstly, the core effects of Trim *(Amplify)* and setting the High, mid, and low band EQs on the FLX4 are now done. The next step will be the Performance pads, the Jog Wheels, and FX selection.

There wasn't any FFMPEG plugin ready to use for Godot for generating audio waveform images so I very quickly made my own one and thankfully android studio already had FFMPEG binaries pre-included which can be automatically retrieved.

We now also have the option to relocate the scene based on where the user is standing so this should make using the software in open spaces much easier.

The bus layout has also been streamlined and changes, such as any effects or volume adjustments in one bus, despite being routed to another bus, did NOT carry over any change of state to that bus. So, what I have done now is by combining "Channel X" input and "Channel X" FX into just one big "Channel X" this now finally works. Progress is coming along at a really good pace!

UI interaction is now substantially improved! UI interaction is touching the XR Viewports with hand, no more fist -> laser -> Thumbs Up -> simulate click required, just hand touch!

6. WEEK 6 – TRACK AND FILE IO

For this week I got working on Adding tracks at runtime.

A 2D menu has been added which will in the near future allow users to add songs from local storage at runtime, with the option of adding songs from Spotify or Soundcloud but further work will be required if I want to check if the user is connected to Wi-Fi because API's *(like the Spotify API)* don't allow for users to download the song file, it is only streamed in so work will have to done to complete this.

The original plan was to route Spotify through SpotDL or something similar, but this is impossible now. This issue isn't unique to just me; after looking online, multiple people seem to be having issues with the Spotify API.

7. WEEK 7 – ALL ITEMS ARE COMPLETED

All requirements are now met!

We have a language selection, FX selection, Track selection, adding tracks, track importing is now done with native file picker dialog instead of the Godot one, Loop management works, Performance pads work, Jog wheels now work, Today is a good day!

Project is at a good place now, but polish is required.

8. WEEK 8 – PROJECT COMPLETED

Finished polish on all aspects.

Multithreading on DJ controller is making the jitter issue disappear.

Rendering is now done on a separate thread instead of the main one. Any performance issues no longer affect the rendering so the headset can keep a consistent frame rate. The DJ Controller and Librebox can still stutter a bit but no where near as bad as it was.

I have also finally discovered why *"I think I like it"* by Fake Blood had the best performance as opposed to the other songs. Fake Blood was in .WAV format, whereas everything else was downloaded in MP3. The MP3 decoding step is not included in the Godot profiler.

After making a custom tool to find all Mp3 files in the project and converting them in place to .wav, EVERYTHING WORKS FLAWLESSLY.

Will get started on writing report now.